

序言 人工智能通识

古老的梦想：会思考的机器

飞速发展

- 1956年 正式提出“人工智能” (Artificial Intelligence) 概念
- 1997年 IBM深蓝依靠强大算力击败国际象棋冠军
- 2016年 DeepMind AlphaGo 通过策略网络+强化学习击败围棋冠军
- 2020年广泛落地于人脸识别、智能驾驶等领域

局限性

专用人工智能 ANI

只在**特定任务**上表现卓越，但很难**跨任务、跨场景**迁移能力

什么是智能？

- 现象：举一反三
小孩被开水烫过一次后，不仅知道避开那个杯子，还会警惕所有冒热气的物体
- 泛化 Generalization
从有限经验中抽取规律，并在陌生环境中做出正确判断的能力

[!TIP]

智能 = 学习能力 + 迁移能力

这就是通用人工智能 (Artificial Generalized Intelligence, AGI)

一种具备类人理解、推理与学习能力，并能在**不同领域**中解决问题的系统

现代AI模仿的学习方式

1. 收集样本
2. 提取特征
3. 迁移学习

AI发展的路线

1. 规则驱动——符号主义

```
if 圆 and 红 and 有柄:
    return “苹果”
```

无法完全描述所有规则

2. 学习驱动——联结主义

Machine learning is the field of study that gives computers the ability to learn without being explicitly programmed.

让机器像人一样“看”、“学”。1959年计算机科学家亚瑟·塞缪尔定义机器学习

机器不再依赖人类硬编码的规则，而是开始从海量数据中自主归纳特征

AI发展的爆发

1. 海量数据 Data
2. 超强算力 Compute

模型的本质

在数学中，通过函数曲线来拟合坐标系中的数据点

机器学习的本质，是在海量的参数空间里，通过不断试错与优化，搜索最能解释数据背后的规律的函数

通过拟合得到的函数，使之可以：

- 输入一张图片 $x \rightarrow$ 输出其代表的分类 y
- 输入一段语音 $x \rightarrow$ 输出对应的文字 y
- 输入前半语句 $x \rightarrow$ 输出后半语句 y

AI发展的阶段

1. 机器学习

统计学的胜利

- 数据驱动
不依赖预设规则，从数据中学习概率
- 典型案例
垃圾邮件分类（统计词频组合）
- 局限
依赖人工特征工程，在“人画的框”里优化

2. 深度学习

感知智能的觉醒

- 自动特征
神经网络模拟人脑，学会“看”、“听”
- 典型案例
AlphaGo、人脸识别（直觉式策略）
- 局限
专用智能，缺乏跨任务迁移能力

3. 大语言模型

生成AI的兴起

- 生成范式
从“选择题”到“作文题”（推理/创造）
- 核心机制
Transformer + Next Token Prediction
- 展望
跨领域通用能力，迈向AGI

为什么是语言模型？

为什么语言模型率先通向通用智能？

我的语言之界限，即我世界之界限
——维特根斯坦

语言是世界的压缩。人类的文字记录，编码了对世界全量的认知。

“因为...所以...” 编码因果律

“虽然...但是...” 编码逻辑转折

讲述一个故事编码时间、空间、社会规范

预测下一个字，就是理解世界

LLM 的训练目标看似简单：

预测下一个字(Next Token Prediction)

为了准确预测，模型被迫去“理解”深层逻辑：

- 要预测物理题目答案，必须学会物理规律
- 要预测代码的下一行，必须学会编程逻辑
- 要预测故事的结局，必须学会人物动机

通过学习人类所有的文字，LLM 实际上是在学习人类对世界的全量认知。

进而“涌现”出惊人的能力——理解逻辑、因果和常识，能够进行推理、创作代码、甚至模仿人类的情感

空间智能：AGI 另一路径

斯坦福李飞飞的反共识观点：

语言模型无法通往AGI，空间智能才是最有路径

空间智能 Spatial

源于寒武纪，经历5亿年演化史。“看见”驱动大脑的爆发。
生存需要感知三维物理（距离、速度、重力）

缺失的拼图：物理直觉

如果AI不懂“球被推下桌子会掉落”，它就无法真正理解世界。
World Lab 致力于构建具有物理一致性的世界模型。
“语言智能与空间智能，如同认知的双翼，缺一不可”

前言 机器学习是什么

本节内容

1. AI 的定义
2. 机器学习的核心与价值
3. ML 与编程的区别
4. ML 与 DL 的关系
5. 模型：学习结果的抽象
6. 算法与模型的区别
7. 建模与特征演进实例
8. 特征与权重分配
9. 学习前提与工具准备

通过数学学习规律，对未来预测

人工智能

人工智能(Artificial Intelligence, AI)

- 是计算机科学的一个分支，致力于创建能够模拟、延伸和扩展人类智能的理论、方法、技术及应用系统。
- 是让机器具备感知、学习、推理和决策的能力

日常语境中的双重定义

1. 理论与方法
实现机器智能的科学原理与技术手段
2. 应用载体
呈现出智能特征的具体产品（ChatGPT、智能机器人等）

机器学习

机器学习(Machine Learning, ML) 研究如何让计算机在 **无需显式编程的情况下，通过数据自动学习规律**，对未来进行预测

机器学习的意义在于，**由计算机来自动完成寻找规律的过程**

- AI 与编程的关系
机器学习依旧需要编写程序，但不是编写最终答案，只是告诉计算机如何进行学习

深度学习

深度学习：基于神经网络的ML

深度学习(DL)本质仍是机器学习，但它使用了**多层神经网络架构**，能够自动提取更深层次的抽象特征

- 传统机器学习
依赖人工完成特征工程
擅长处理结构化数据（表格等）
算力需求较小，在金融风控、商品推荐中广泛应用
- 深度学习
输入原始数据，模型自动逐层抽象(边缘 → 纹理 → 整体语义)
擅长处理非结构化数据（图/文/音）
需要海量数据以及庞大算力（大语言模型、自动驾驶）

AI, ML, DL 的关系

$$DL \subset ML \subset AI$$

- 深度学习
使用神经网络，提取深层抽象模式
计算机使用神经网络处理数据做出决策
- 机器学习
不依赖人工规则，从数据中自动寻找规律
计算机基于数据做出决策任务
- 人工智能
让机器表现出类似人类的智能
计算机执行各种决策任务的能力合集

机器根据数据决策

计算机基于数据决策的原理，完美映射人类基于经验进行预测的过程

1. 记住历史数据
 2. 学习通用规律
 3. 基于规律预测
- 广义的模型
对现实世界复杂事物的 **简化与抽象**
以清晰地理解、分析、预测或控制错综复杂的现象
 - 狭义的模型
在机器学习领域，模型是计算机通过学习大量数据后，自主总结出的一套 **规则集合**，用于对未来的新数据进行预测或决策

算法与模型的区别

算法负责构建模型，模型负责预测未来

- 算法 Algorithm
机器学习的方法
用于解决问题或执行计算的一套明确逻辑步骤
指导计算机去观测并找出共同特征
- 模型 Model
机器学习的结果
计算机学习大量数据后自主总结的一套规则集合
对现实规律极简表示，用于预测新数据

建模与特征演进实例

垃圾邮件分类

场景：周末收到小张的邮件，在不打开邮件的前提下如何判断垃圾邮件？

1. 基于历史概率进行盲猜
 - 历史数据
最近10封邮件，其中6封是垃圾邮件
 - 模型1.0
小张的每10封邮件，包含6封垃圾邮件
 - 预测结果
新邮件60%的概率是垃圾邮件，盲猜其为垃圾邮件
2. 寻找单一特征规律
 - 发送时间特征
 1. 周一  正常
 2. 周六  垃圾
 3. 周二  正常
 4. 周六  垃圾
 5. 周三  正常
 6. 周四  正常
 7. 周日  垃圾

- 模型2.0
 - 工作日邮件 → 正常邮件
 - 休息日邮件 → 垃圾邮件
 - 预测结果
 - 今天是周末，推断是垃圾邮件
3. 构建多维特征组合
- 文件大小特征
 - 1. 1~7KB ✔ 正常
 - 2. 16KB ✘ 垃圾
 - 3. 3KB ✔ 正常
 - 4. 20~35KB ✘ 垃圾
 - 模型3.0
 - 如果 休息日发送 且 文件 > 10KB
 - 那么 推断垃圾邮件
 - 否则 推断正常邮件
 - 预测结果
 - 拦截周六发送的大附件广告
 - 接收周日发送的3KB通知

特征与权重分配

机器学习本质：寻找最优权重

“发送时间”、“文件大小”称为预测的 *特征 (Feature)*

人类一次只能处理有限的少量特征，计算机能快速分析大量特征

算法会为不同特征分配 *权重 (Weight)*

$$\text{得分} = (w_1 \times \text{周末发送}) + (w_2 \times \text{文件超标}) + (w_3 \times \text{包含免费}) + \dots$$

机器学习的核心问题

如何精准确定每个特征的权重？

让计算机通过大量历史数据，不断自动调整直到寻得 **最优权重**

学习前提与工具准备

学习路线与前置知识

- 数学基础
 - 1. 线性代数
 - 向量与矩阵的基础概念
 - 加减/点积/乘法
 - 2. 概率统计
 - 均值
 - 方差
 - 概率分布
 - 贝叶斯定理
 - 3. 微积分
 - 导数
 - 偏导数
 - 链式法则
- 编程技能
 - 1. Python
 - 机器学习的事实标准
 - 2. Jupyter Notebook
 - 可视化的实验环境
 - 3. 生态工具箱
 - 数据处理 Numpy / Pandas
 - 可视化 Matplotlib / Seaborn
 - 算法库 Scikit-learn

01. 机器学习的类型

本节内容

- 1. 机器学习分类引入
- 2. 基础概念：数据集与标签
- 3. 机器学习三大类型
- 4. 监督学习：回归与分类
- 5. 无监督学习：聚类降维生成
- 6. 强化学习原理与应用
- 7. 总结

机器学习的应用场景

- 1. 根据各项因素预测资产价格
- 2. 根据邮件内容预测垃圾邮件
- 3. 根据像素识别人脸或动物
- 4. 向用户推荐偏好内容
- 5. 处理长文本文档为摘要
- 6. 训练自动驾驶汽车在城市穿行
- 7. 进行棋牌对弈

虽然场景各异，但是多种问题的解决方式是通用的。
例如，房市价格预测与股市价格预测属于同类问题；垃圾邮件拦截与信用风险控制用相似模型解决

数据、特征与标签

数据是用于训练和测试机器学习模型的原始信息。
对于结构化数据，诸如表格形式存储的数据，每一行代表一个具体的实例，每一列代表一项特定的属性

面积(m^2)	卧室数量	浴室数量	地段评分	房龄(年)	房价(万元)
85	2	1	6	15	180
110	3	1	7	8	320
142	4	2	9	3	510
73	2	1	4	20	120
128	3	2	8	5	450

- 数据集 Dataset
指整张数据表格
- 数据点 Data Point
也称样本，数据集中的单条记录，表格的每一行就是一个数据点
- 特征 Feature
也称维度 Dimension，表示数据的属性，表格的列
- 标签 Label
模型需要重点预测的“目标变量”(常是表格的最后一列)。
所谓预测，本质是机器通过学习特征用以推算数据中的标签

机器学习的三大分支

根据数据集有无标签，以及学习方式的差异，机器学习衍生为三大分支：

- 1. 监督学习 Supervised Learning
数据点 **包含标签**，机器通过学习特征与标签的关系进行预测
- 2. 无监督学习 Unsupervised Learning
数据点 **不含标签**，机器需要在数据中自行寻找隐藏的结构或规律
- 3. 强化学习 Reinforcement Learning
在动态环境中**不断试错**，学习如何更有决策获得最大奖励

监督学习：处理标签数据

监督学习覆盖图像识别、文本处理、推荐系统等众多领域

- 核心目标
根据已有数据预测标签

针对不同类型的标签数据，监督学习衍生两种模型：

1. 回归模型 Regression

用于预测**连续的数值型数据**，模型的输出是具体的数值

- 应用场景
预测价格走势
- 常见算法
 1. 线性回归
 2. 决策树
 3. 随机森林
 4. XGBoost

2. 分类模型 Classification

用于预测**离散的分类型数据**

- 应用场景
 1. 图像识别
 2. 垃圾邮件识别
 3. 用户画像识别
- 常见算法
 1. 感知器
 2. 逻辑回归
 3. 朴素贝叶斯
 4. K近邻投票 KNN
 5. 支持向量机 (Support Vector Machine, SVM)

无监督学习：处理无标签的数据

无监督学习的目标不是预测具体的数值或类别，而是让模型自行探索，发现数据内部隐藏的结构、模式或规律

无监督学习主要包括3个方向：

1. 聚类算法
2. 降维算法
3. 生成算法

聚类：对数据集分组

- 核心作用
将数据集分组分类
聚类算法会计算数据之间的相似度，将相似数据归拢为组/簇，实现物以类聚
- 应用场景
梳理用户画像用于电商精准营销
- 代表算法
K-means

降维：信息整合与特征重构

- 核心作用
信息整合与特征重构
在尽量不丢失关键信息的前提下，极大地简化数据集
- 应用场景
捕捉冗余特征的相关性，合并为新特征
房价预测中将“地铁距离”、“学校数量”、“商业配套”等多个周边环境特征，整合为“地段综合评分”
- 代表算法
主成分分析 (Principal Component Analysis, PCA)

生成算法 AIGC

- 核心作用
从数据中创造全新内容
生成算法通过深度学习真实数据的分布规律，生成**现实中不存在的全新数据点**，解决数据匮乏问题
- 应用场景
低成本扩充模型训练集
AI 绘画
- 代表算法
生成对抗网络 (Generative Adversarial Network, GAN)

强化学习：在动态环境中试错

在复杂的现实场景中（在自动驾驶实时决策、围棋落子），面临高度动态的决策，缺少现成的静态历史数据集

- 核心目标
在不断试错的过程中，学习如何做出更有决策
- 运行机制
在环境中设置智能体(Agent)，根据智能体采取动作的结果好坏，实时反馈奖励(Reward)或惩罚(Penalty)。最终形成在长期过程中取得最大累计奖励的策略。
- 应用场景
 1. 棋牌或电子游戏
 2. 机器人姿态控制
 3. 工业机械臂调度
 4. 量化交易策略制定
 5. 大型语言模型的人类意图对齐
- 关键技术
 1. Q-Learning 算法
 2. 结合神经网络的深度强化学习 DRL

总结

核心分类

1. 监督学习
机器基于标签数据进行学习
包括**分类模型**和**回归模型**
2. 无监督学习
探索无标签数据的内在结构
包含**聚类**、**降维**与**生成算法**
3. 强化学习
在动态环境中，通过**不断试错**与**奖惩机制**，学习更优决策

02. 线性回归

本节内容

1. 核心概念
2. 模型结构
3. 损失函数
4. 梯度下降
5. 向量表示
6. 逻辑回归与深度学习
7. 完整流程与意义总结

房价预测案例

数据集

假设房价数据

面积(m^2)	总价(万元)
50	122

面积(m^2)	总价(万元)
70	157
90	203
110	255
130	303

预测100平方米的房价

可视化

```
import matplotlib.pyplot as plt

# 1 prepare data
area = [50, 70, 90, 110, 130] # 面积 m^2
price = [122, 157, 203, 255, 303] # 总价 万元

# 2 draw plot
plt.figure(figsize = (8, 6))
plt.scatter(area, price, color = 'blue', s = 80)

# 3 set axis
plt.xlabel('面积(m^2)')
plt.ylabel('总价(万元)')
```

趋势线

尝试绘制一条直线，尽量穿过所有的数据点，以此描述面积和总价之间的关系

直线方程可能是 $price = area \times 2.2 + 10$

代入面积100平方米，算得总价230万元

线性回归

线性回归 (Linear Regression)

是机器学习最经典的模型之一

通过构建线性方程来拟合数据的内在分布规律，进而实现对连续数值的预测

线性

- 线性 $\neq$ 直线
- 齐次性
$$f(ax) = a \cdot f(x)$$
- 可加性
$$f(x + y) = f(x) + f(y)$$
- 几何解释
在一维空间中，线性关系表现为一条直线；
在高维空间中，线性关系表现为平面或超平面

回归

高尔顿的“回归均值”

回归 (Regression) 源于统计学家弗朗西斯·高尔顿 (Francis Galton) 的经典研究

研究发现，身高极高或极矮的父母，子女身高常向群体均值靠拢，称该现象“回归均值”

在机器学习领域，回归被引申为

旨在通过数据建模变量间的映射关系，进而实现对连续数值的预测

模型公式

最简单的线性回归模型只有一个特征，公式如下

$$\hat{y} = wx + b$$

- $\hat{y}$: 模型输出的预测值
- x : 输入模型的特征值
- w : 权重 (weight), 也称斜率 (slope)
- b : 偏置 (bias), 也称截距 (intercept)

w 和 b 是模型需要学习的参数
训练模型即是寻得一组最合适的 w 和 b

损失函数：度量模型偏差

损失函数 (Loss Function) 用于评估模型参数，度量预测值与真实值之间的偏差
预测值距离真实值越远，损失越大
模型训练要让损失函数的值尽量变小

- 均方误差 (Mean Squared Error, MSE)
 - 1. 对每个样本的误差取平方，消除正负号、缩放极端误差
 - 2. 对所有样本的误差取平均

梯度下降：寻找最优参数

- 梯度下降 (Gradient Descent)
沿着损失函数下降最快的方向，逐步调整参数，直至找到最优解

循环步骤

1. 计算梯度
2. 更新参数
3. 迭代优化
4. 模型收敛

以蒙眼下山比喻优化模型

- 当前海拔，对应损失 (Loss)
- 下坡方向，对应梯度 (Gradient)
- 下山步幅，对应学习率 (Learning Rate)

训练流程

1. 输入特征值
2. 输出预测值
3. 计算损失
4. 计算梯度
5. 更新参数

每轮循环旨在优化参数，模型过多轮迭代逐渐收敛至最优解

多特征线性回归

对于房价额外引入其他特征

1. 楼层
2. 地铁距离
3. 房龄

面积(m^2)	楼层	地铁距离(km)	房龄(年)	总价(万元)
50	3	0.5	5	180
70	15	1.2	10	210
90	8	0.3	2	320
110	20	2.0	15	280
130	12	0.8	8	380

扩展模型

现有 d 个特征，模型扩展如下

$$\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + \cdots + w_dx_d + b$$

$$\hat{y} = w_1 \cdot x_{\text{面积}} + w_2 \cdot x_{\text{楼层}} + w_3 \cdot x_{\text{地铁距离}} + w_4 \cdot x_{\text{房龄}} + b$$

- w_1 (面积): $+$ 正相关, 越大越贵
- w_2 (楼层): $+$ 正相关, 越高越贵
- w_3 (地铁距离): $-$ 负相关, 越近越贵
- w_4 (房龄): $-$ 负相关, 越新越贵

代码实现

```
import numpy as np
from sklearn.linear_model import LinearRegression

# 1 Prepare Data
## 面积, 楼层, 地铁距离, 房龄
X = np.array([
    [ 50,  3, 0.5,  5],
    [ 70, 15, 1.2, 10],
    [ 90,  8, 0.3,  2],
    [110, 20, 2.0, 15],
    [130, 12, 0.8,  8],
])
y = np.array([180, 210, 320, 280, 380]) # 总价(万元)

# 2 Train Model
model = LinearRegression()
model.fit(X, y)

# 3 View Parameters
feature_names = ['面积', '楼层', '地铁距离', '房龄']
print("各项特征权重: ")
for name, w in zip(feature_names, model.coef_):
    print(f"    {name}: {w:+.2f}")
print(f"    偏置 b: {model.intercept_: .2f}")

# 4 Predict
## New Data [100, 10, 0.6, 5]
new_house = np.array([[100, 10, 0.6, 5]])
pred = model.predict(new_house)
print(f"\n预测房价: {pred[0]:.1f} 万元")
```

向量表达

$$\hat{y} = w^T x + b$$

其中:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}, \quad w = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix}$$

多项式回归

引入多项式回归用于曲线的拟合

$$\hat{y} = w_1x + w_2x^2 + \cdots + w_dx^d + b$$

- 当 $d = 1$ 时, 为普通的线性回归

- 当 $d = 2$ 时，拟合抛物线回归
- 阶数 d 越高，曲线越灵活

尽管特征 (x^2, x^3) 出现高阶非线性项，
但是模型对参数 $(w_1, w_2, \dots, w_d, b)$ 仍是线性的
即 $\hat{y}$ 是 $w_1, w_2, \dots, w_d, b$ 的线性组合
因此多项式回归本质上仍是广义线性模型的范畴，可以用线性回归的方法求解

逻辑回归与深度学习

由线性回归延伸出两个重要方向：

1. 逻辑回归

线性回归 + 激活函数

用于解决分类问题，输出概率

$$\begin{aligned} P(y = 1 \mid x) &= \sigma(wx + b) \\ &= \frac{1}{1 + e^{-(wx+b)}} \end{aligned}$$

2. 深度学习

多层逻辑回归的叠加

将多个“线性变化+激活函数”逐层堆叠构成神经网络：

输入 $\xrightarrow{\text{线性+激活}}$ 隐藏层₁ $\xrightarrow{\text{线性+激活}}$ 隐藏层₂ $\rightarrow \dots \rightarrow$ 输出

总结

线性回归包含了机器学习的核心思想

1. 用数据寻找规律
2. 用模型表达规律
3. 用损失函数衡量误差
4. 用梯度下降优化参数
5. 循环训练迭代准确模型

几乎所有机器学习和深度学习模型，都是对上述基本范式的扩展和升级

03. 梯度下降

本节内容

1. 梯度下降核心地位
2. 梯度数学原理
3. 梯度公式推导
4. 房价预测实例
5. 学习率调优
6. 优化器演进
7. 最小批量梯度下降
8. 核心总结

模型训练

以简单的线性回归为例

$$\hat{y} = wx + b$$

- w : weight 权重 / 斜率
- b : basis 偏置 / 截距

训练模型的本质

- 是搜索最佳的 w 和 b

[!TIP]

训练闭环

输入数据 → 输出预测 → 计算损失 → 计算梯度 → 更新参数

用损失函数来度量预测值 $\hat{y}$ 与真实值 y 之间的偏差
以均方差 (MSE) 为例

$$L = MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

训练模型的目标

- 实现最小损失函数
- 此时取得前述的最佳 w 和 b

梯度下降 (Gradient Descent)

- 是一种自动调参的算法
- 回答如何调参实现最小损失函数的问题

数学基础

两个关键数学概念:

1. 导数
函数在某点的变化率
2. 偏导数
多变量函数的变化率

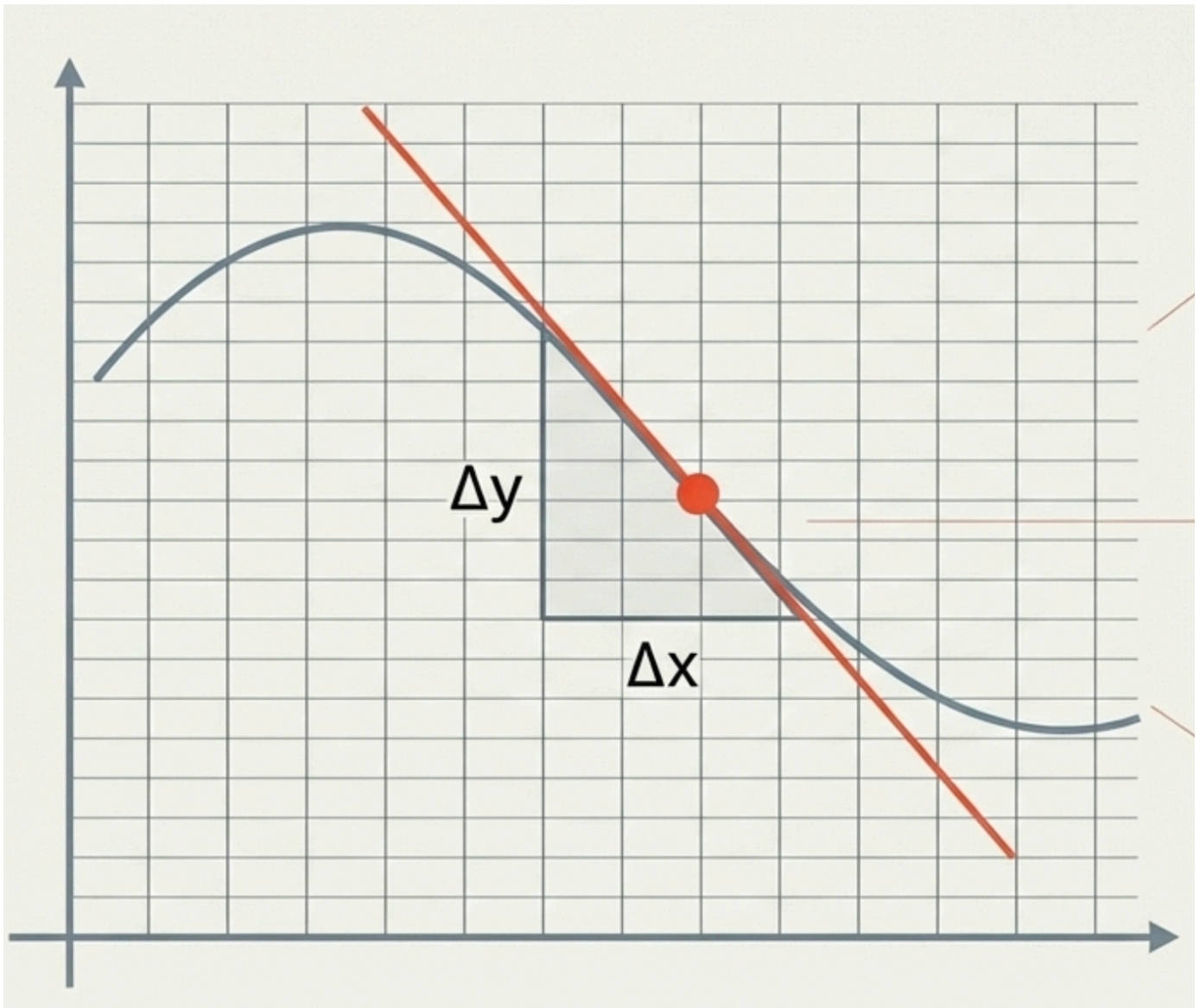
导数

导数 (Derivative)

- 是函数在某点的变化率
- 说明函数值随自变量变化的趋势

对于单变量函数 $f(x)$ 在 x_0 处的函数定义为

$$f'(x_0) = \lim_{\Delta x \rightarrow 0} \frac{f(x_0 + \Delta x) - f(x_0)}{\Delta x}$$



- 几何意义
函数曲线在该点的切线斜率
- $f'(x_0) > 0$
 $f(x)$ 在 x_0 处递增, x 增大, $f(x)$ 增大
- $f'(x_0) = 0$
 $f(x)$ 在 x_0 处达到极值
- $f'(x_0) < 0$
 $f(x)$ 在 x_0 处递减, x 增大, $f(x)$ 减小

偏导数

偏导数 (partial derivative)

- 多变量函数的变化率
- 说明每个参数对函数值的影响

损失函数通常依赖多个参数

例如损失函数 $L(w, b)$ 取決与 w 和 b

需要固定其他变量不变, 只对其中一个变量求导

对于函数 $f(x, y) = x^2 + 3xy + y^2$

函数对 x 的偏导数 $\frac{\partial f}{\partial x} = 2x + 3y$ (把 y 作常数)

函数对 y 的偏导数 $\frac{\partial f}{\partial y} = 3x + 2y$ (把 x 作常数)

核心原理

根据导数和偏导数，可以理解梯度下降。

梯度

梯度 (Gradient)

- 将所有变量的偏导数组成的 **向量**
- 指向最陡上坡的方向

对于简单线性回归 $f(x) = wx + b$ 的损失函数 $L(w, b)$ ，梯度为

$$\nabla L(w, b) = \left(\frac{\partial L}{\partial w}, \frac{\partial L}{\partial b} \right)$$

如有更多参数 $\theta_1, \theta_2, \dots, \theta_d$ 时，梯度为

$$\nabla L(\theta) = \left(\frac{\partial L}{\partial \theta_1}, \frac{\partial L}{\partial \theta_2}, \dots, \frac{\partial L}{\partial \theta_d} \right)$$

梯度的重要性质:

1. 梯度的 **方向**，是函数值增长最快的方向
2. 梯度的 **反方向**，是函数值下降最快的方向

迭代公式

基于前述，梯度下降的参数更新公式为

$$\begin{aligned}\theta_{new} &= \theta_{old} - \eta \nabla L(\theta_{old}) \\ \theta &:= \theta - \eta \nabla L(\theta)\end{aligned}$$

- θ_{old} : 当前位置
- $\nabla L(\theta_{old})$: 当前参数处的梯度
- η : 学习率 (Learning rate) 控制调参幅度
- $-\eta \cdot \nabla L$: 沿梯度反方向，按学习率调参一次
- $:=$: 计算机赋值号

为何减去梯度

因为梯度指向函数值增长的方向,所以为实现损失函数最小化，需要取梯度反方向

为何乘学习率

通过学习率控制调参的幅度，避免幅度过大错过最优，避免幅度过小浪费时间

对于线性回归，展开参数更新公式得到

$$\begin{aligned}w_{new} &= w_{old} - \eta \cdot \frac{\partial L}{\partial w} \\ b_{new} &= b_{old} - \eta \cdot \frac{\partial L}{\partial b}\end{aligned}$$

代码实现

问题设置

使用房价预测作为演示

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

np.random.seed(42) # 随机种子
n_samples = 100    # 抽样数量

# 生成随机面积
areas_m2 = np.random.uniform(90, 150, n_sample) # (100 个 90~150 平方米)

# 转换为百平米，减少两个参数的量纲差异对学习率的影响
```

```

areas = areas_m2 / 100

# 生成房价
prices = 250 * areas + 30 + np.random.randn(n_sample) * 15

print(f"房屋数量: {len(areas)} 套")
print(f"面积范围: {areas_m2.min():.1f} - {areas_m2.max():.1f} 平米 ({areas.min():.2f} - {areas.max():.2f} 百平米)")
print(f"价格范围: {prices.min():.1f} - {prices.max():.1f} 万")
print(f"真实规律: 房价 = 2.5 × 面积(平米) + 30")

```

房屋数量: 100 套
 面积范围: 90.3 - 149.2 平米 (0.90 - 1.49 百平米)
 价格范围: 248.0 - 414.8 万
 真实规律: 房价 = 2.5 × 面积(平米) + 30

推导公式

模型公式

$$\hat{y} = wx + b$$

损失函数使用 MSE

$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

将模型公式代入损失函数有

$$L(w, b) = \frac{1}{n} \sum_{i=1}^n (wx_i + b - y_i)^2$$

通过链式法则分别对 w, b 求偏导

$$\frac{\partial L}{\partial w} = \frac{1}{n} \sum_{i=1}^n 2(wx_i + b - y_i) \cdot x_i$$

$$\frac{\partial L}{\partial b} = \frac{1}{n} \sum_{i=1}^n 2(wx_i + b - y_i)$$

手动实现

手搓梯度下降

```

def gradient_descent(X, y, lr=0.1, epochs=200):
    # 初始化参数
    w, b = 0.0, 0.0
    n = len(X)

    # 初始化历史记录, 使用记录初始参数和损失
    history = {'w': [w], 'b': [b], 'loss': [np.mean((w*X + b - y)**2)]}

    # 进行迭代训练, 更新参数
    for epoch in range(epochs):
        # 计算当前预测值
        y_pred = w * X + b

        # 计算参数 w 梯度
        dw = (2/n) * np.sum((y_pred - y) * X)
        # 计算参数 b 梯度
        db = (2/n) * np.sum(y_pred - y)

        # 使用学习率更新参数
        w -= lr * dw

```

```

    b -= lr * db

    # 记录参数和损失
    history['w'].append(w)
    history['b'].append(b)
    history['loss'].append(np.mean((y_pred - y)**2))

    # 每隔 1% 训练轮数, 打印一次进度
    if epoch % (epochs/100) == 0:
        print(f"迭代 {epoch:3d}: w={w:.2f}, b={b:.2f}, loss={history['loss'][-1]:.2f}")

    return history

history = gradient_descent(areas, prices, lr=0.01, epochs=100000)
print(f"\n最终结果: 房价 = {history['w'][-1]/100:.3f} × 面积(平米) + {history['b'][-1]:.2f}")

```

```

迭代   0: w=7.85, b=6.51, loss=107933.06
...
迭代 10000: w=232.89, b=50.30, loss=182.49
...
迭代 20000: w=238.09, b=44.07, loss=181.49
...
迭代 30000: w=238.48, b=43.61, loss=181.48
...
迭代 40000: w=238.50, b=43.57, loss=181.48
...
迭代 50000: w=238.51, b=43.57, loss=181.48
...
迭代 60000: w=238.51, b=43.57, loss=181.48
...
迭代 70000: w=238.51, b=43.57, loss=181.48
...
迭代 80000: w=238.51, b=43.57, loss=181.48
...
迭代 90000: w=238.51, b=43.57, loss=181.48
...
迭代 99000: w=238.51, b=43.57, loss=181.48

最终结果: 房价 = 2.385 × 面积(平米) + 43.57

```

训练过程

1. 将训练过程可视化
2. 忽略前 100 次迭代, 避免初始值对收敛过程演示产生干扰

```

# 创建一个 1 行 3 列的子图布局
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# 绘制房屋数据散点图
axes[0].scatter(areas_m2, prices, c='steelblue', s=60, alpha=0.7, label='房屋数据')

# 绘制拟合曲线
x_line = np.linspace(85, 155, 100) # 在 (85,100) 区间, 生成 100 个等间距的点
y_line = (history['w'][-1]/100) * x_line + history['b'][-1] # 计算拟合曲线的 y 值
axes[0].plot(x_line, y_line, 'coral', lw=2,
             label=f'拟合: y={history["w"][-1]/100:.3f}x+{history["b"][-1]:.1f}')
axes[0].set_xlabel('面积 (平米)', fontsize=12)
axes[0].set_ylabel('房价 (万)', fontsize=12)
axes[0].set_title('房价预测模型', fontsize=14)
axes[0].legend(fontsize=10)
axes[0].grid(True, alpha=0.3)

# 跳过前 100 个迭代, 避免初始值对收敛过程演示的干扰

```

```

skip = 100

### 绘制参数收敛过程

# 绘制 w 目标值水平线
axes[1].axhline(250, color='steelblue', ls='--', alpha=0.5)
# 绘制 w 收敛过程
axes[1].plot(history['w'][skip:], 'steelblue', lw=2, label='w (目标: 250)')

# 绘制 b 目标值水平线
axes[1].axhline(30, color='darkorange', ls='--', alpha=0.5)
# 绘制 b 收敛过程
axes[1].plot(history['b'][skip:], 'darkorange', lw=2, label='b (目标: 30)')

axes[1].set_xlabel('迭代次数', fontsize=12)
axes[1].set_ylabel('参数值', fontsize=12)
axes[1].set_title('参数收敛过程', fontsize=14)
axes[1].legend(fontsize=10)
axes[1].grid(True, alpha=0.3)

# 绘制损失函数下降过程
axes[2].plot(history['loss'][skip:], 'orangered', lw=2)
axes[2].set_xlabel('迭代次数', fontsize=12)
axes[2].set_ylabel('损失 (MSE)', fontsize=12)
axes[2].set_title('损失函数下降', fontsize=14)
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

1. 损失值起初迅速下降，后逐渐趋于平缓
2. 参数 w 从 0 逐步逼近 238
3. 最终拟合的直线较好地拟合了大部分数据点

学习率实验

学习率 η 是梯度下降中最关键的超参数 通过实验体会不同学习率的作用

```

import numpy as np
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial Unicode MS', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False

# ===== 核心逻辑 =====
def gradient_descent(start_x, lr, n_steps=15):
    """在  $f(x)=x^2$  上做梯度下降，返回每步的  $x$  值"""
    trajectory = [start_x]
    x = start_x
    for _ in range(n_steps):
        grad = 2 * x          #  $f'(x) = 2x$ 
        x = x - lr * grad     # 参数更新
        trajectory.append(x)
        if abs(x) > 500:      # 已经爆炸，提前停
            break
    return trajectory

# ===== 四种学习率对比 =====
x0 = 4.0
configs = [
    (0.05, '太小：蜗牛爬坡', 'steelblue'),
    (0.3, '合适：稳步收敛', 'seagreen'),
    (0.9, '偏大：剧烈震荡', 'orange'),
    (1.1, '过大：梯度爆炸', 'crimson'),

```

```

]

# ----- 图1: 参数值随迭代变化 -----
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
for ax, (lr, title, color) in zip(axes.flatten(), configs):
    traj = gradient_descent(x0, lr)
    ax.plot(range(len(traj)), traj, 'o-', color=color, markersize=5)
    ax.axhline(y=0, color='gray', linestyle='--', alpha=0.5, label='最优点 x=0')
    ax.set_title(f'lr={lr} - {title}', fontsize=12)
    ax.set_xlabel('迭代次数')
    ax.set_ylabel('参数 x')
    ax.legend()
    ax.grid(True, alpha=0.3)

plt.suptitle('学习率对梯度下降的影响 (参数变化)', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# ----- 图2: 在损失曲线上可视化轨迹 -----
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
x_curve = np.linspace(-8, 8, 300)

for ax, (lr, title, color) in zip(axes.flatten(), configs):
    traj = gradient_descent(x0, lr, n_steps=10)
    # 只保留不越界的点, 方便画图
    traj_plot = [x for x in traj if abs(x) <= 10]

    ax.plot(x_curve, x_curve**2, 'k-', alpha=0.2, linewidth=2) # 损失曲线
    ax.plot(traj_plot, [x**2 for x in traj_plot], 'o', color=color, markersize=6)

    # 画箭头连接每一步
    for i in range(len(traj_plot) - 1):
        ax.annotate('', xy=(traj_plot[i+1], traj_plot[i+1]**2),
                    xytext=(traj_plot[i], traj_plot[i]**2),
                    arrowprops=dict(arrowstyle='->', color=color, lw=1.5))

    ax.set_title(f'lr={lr} - {title}', fontsize=12)
    ax.set_xlim(-9, 9)
    ax.set_ylim(-5, 70)
    ax.set_xlabel('x')
    ax.set_ylabel('f(x) = x^2')
    ax.grid(True, alpha=0.3)

plt.suptitle('学习率对梯度下降的影响 (损失曲面轨迹)', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

```

对于函数 $y = x^2$

梯度 (导数) 是 $2x$

更新公式是 $x := x - lr \times 2x = x(1 - 2lr)$

function `gradient_descent()`

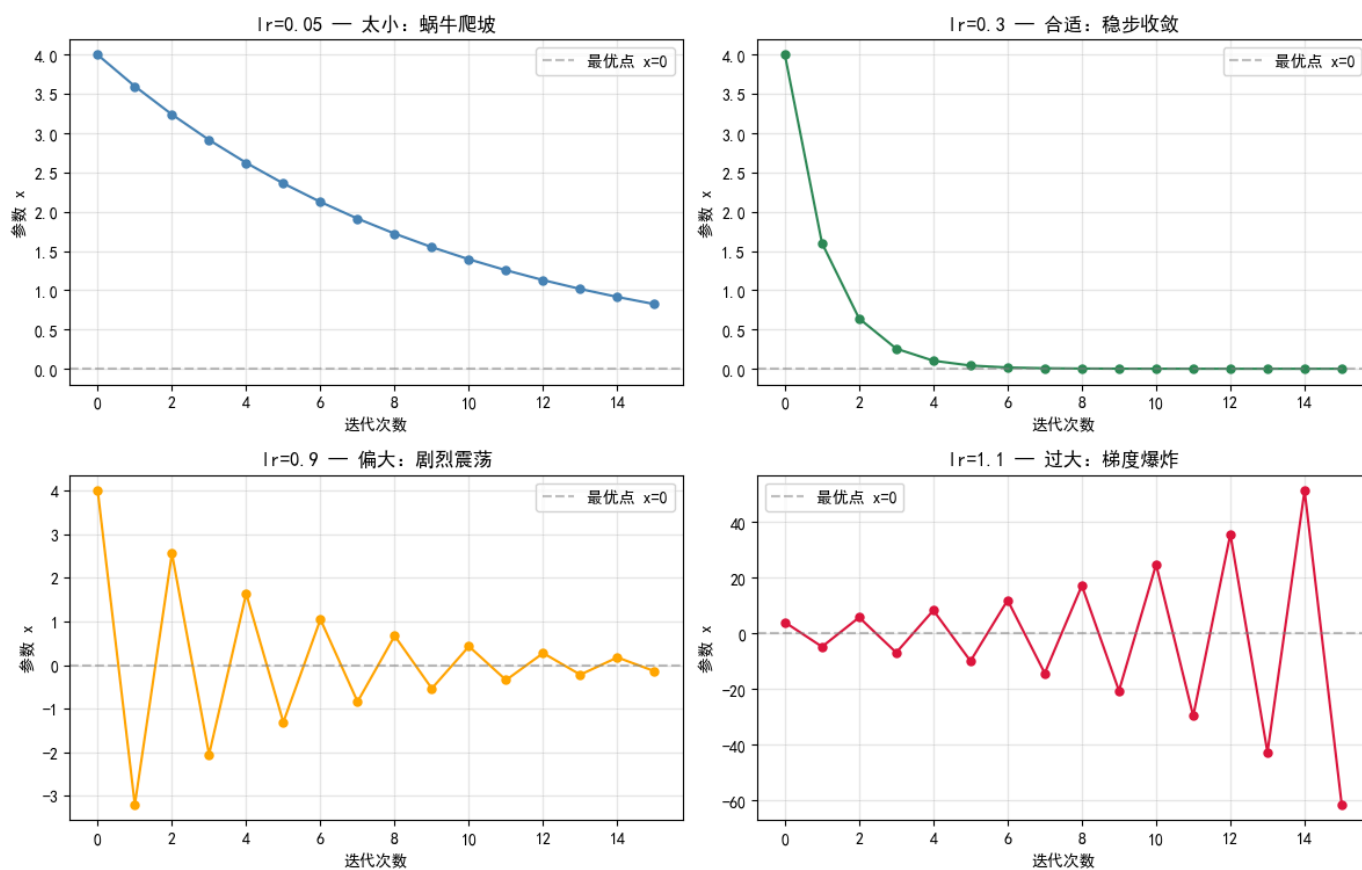
- 列表 `trajectory` 记录梯度下降过程中的 x 值
- 若 $\text{abs}(x) > 500$, 则 `break` 迭代
- 最终返回 `trajectory`

list `configs`

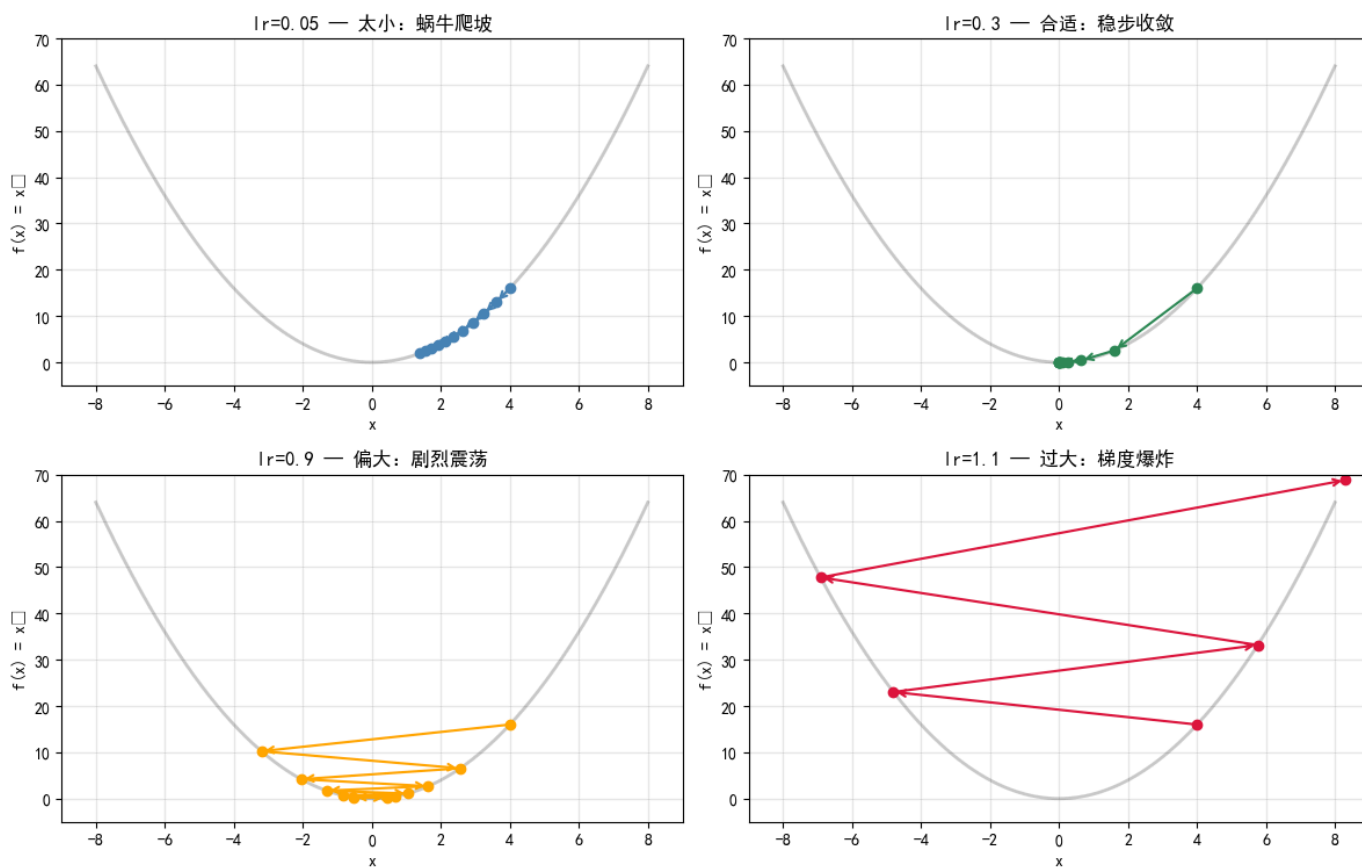
存放不同学习率的配置

1. `lr` 学习率值
2. `title` 绘图标题
3. `color` 图例颜色

学习率对梯度下降的影响（参数变化）



学习率对梯度下降的影响（损失曲面轨迹）



1. 学习率偏小 $lr = 0.05$

x 在右侧坡上慢慢下挪, 10 次迭代根本无法到达谷底 ($x = 0$)

呈现学习率太小会导致收敛极慢

2. 学习率适中 $lr = 0.3$
X 顺滑快速达到谷底 ($x = 0$)，没有多余动作
3. 学习率偏大 $lr = 0.9$
X 一步跨过谷底移至左侧，下步跨至右侧
每次跨幅逐渐变小，最终呈 Z 字形震荡收敛至底部
4. 学习率巨大 $lr = 1.1$
X 不仅在两侧反复横跳，而且越跳越高，直接飞出画幅
数字呈指数级增大，这是典型的**梯度爆炸**

[!TIP]

学习率的选择通常需要实验调整

常见的起始值有 $1e-2$, $1e-3$, $1e-4$ 等

优化算法

标准梯度下降存在局限

1. 收敛速度慢
2. 容易陷入局部最优

业界发展如下优化

1. 动量法 (Momentum)
借鉴物理学的惯性，给梯度添加“速度”，减少在狭长山谷中的震荡
2. 自适应梯度 (AdaGrad)
为每个参数自动维护独立的学习率
适合处理稀疏数据
3. 均方根传播 (RMSProp)
对 AdaGrad 的改良，关注近期的梯度变化，避免学习率降为零
适合不平稳的目标 (如 RNN)
4. 自适应矩估计 (Adam)
结合动量法的方向加速和 RMSProp 的自适应步幅
同时维护梯度的方向趋势和大小变化，既快又稳
收敛速度快，且对学习率不敏感
是目前深度学习中最广泛使用的 **默认首选优化器**

动量法 Momentum

基础梯度下降只看当前这一步的坡度，容易在狭长山谷中来回震荡，走出锯齿形路线。

动量法借鉴了物理学中惯性的概念——每次更新参数时，将历史更新方向累积为一个“速度”，连续同向则加速，来回变化则相互抵消。就像小球从山坡滚下，凭借惯性能冲过小坑洼而不是卡在里面。这样做显著减少了震荡、加速了收敛，还有助于跳出局部最优。不过惯性也是双刃剑，动量过大时可能冲过最优点。

它特别适合损失曲面存在大量狭长山谷的场景，也常用于对泛化性能要求较高的计算机视觉任务——许多经典图像模型至今仍使用 SGD + Momentum 来训练。

自适应梯度 AdaGrad

动量法解决了方向上的问题，但所有参数仍然共用同一个学习率。现实中有些参数频繁更新，有些参数很少被触及，一刀切显然不合理。

AdaGrad 的做法是为每个参数自动维护独立的学习率：它累积每个参数的历史梯度平方和，梯度大的参数自动减小步幅，梯度小的参数则保持较大学习率，让稀疏特征也有充分的学习机会。问题在于，历史梯度只增不减，到了训练后期所有参数的学习率都趋近于零，训练可能提前停滞。

它最擅长处理稀疏数据场景，比如自然语言处理中的词向量训练、推荐系统中的大规模稀疏特征学习，这类场景中不同特征的出现频率差异悬殊，AdaGrad 的自适应特性恰好能发挥优势。

均方根传播 RMSProp

RMSProp 正是针对 AdaGrad 学习率单调衰减这一问题的改进。它用指数加权移动平均替代了简单累加，让学习率具备了遗忘能力——只关注近期的梯度变化，不再被遥远的历史拖累。

这样既保留了自适应学习率的优势，又避免了后期停滞。但它只解决了步幅问题，缺少动量机制，在方向上没有加速能力。

它特别适合非平稳目标函数的优化，比如循环神经网络（RNN）的训练，这类任务中梯度的统计特性会随时间不断变化，RMSProp 的遗忘机制能够很好地适应这种动态性。

自适应矩估计 Adam

Adam 把动量的方向加速和 RMSProp 的自适应步幅结合，它同时维护两套信息——一套跟踪梯度的方向趋势（动量），一套跟踪梯度的大小变化（自适应学习率），两者结合使它既跑得快又跑得稳。它的默认超参数在绝大多数场景下直接可用，收敛快且对学习率选择不敏感。

Adam 几乎适用于所有深度学习任务，从 Transformer 语言模型到生成对抗网络，从强化学习到多模态模型，尤其在模型结构复杂、调参预算有限的情况下，它是最稳妥的选择。

对比总结

算法	一句话概括	典型使用场景
SGD	最基础的版本	简单问题，或搭配学习率调度策略
Momentum	给梯度加上惯性	损失面有很多狭长山谷
AdaGrad	给每个参数自动调学习率	稀疏数据 (如 NLP 词向量)
RMSProp	AdaGrad 的改良，学习率不会降到零	RNN 等非平稳目标
Adam	Momentum + RMSProp，全能选手	通用首选，深度学习默认优化器

手搓 Adam

Adam 迭代索引 (时间步) 起始为 1 而非 0，用于偏差校正

```
def adam_optimizer(X, y, lr=0.1, epochs=200, beta1=0.9, beta2=0.999, epsilon=1e-8):
    # 初始化参数
    w, b = 0.0, 0.0
    n = len(X)

    # ===== Adam 特有的状态变量初始化 =====
    m_w, v_w = 0.0, 0.0 # w 的一阶矩和二阶矩
    m_b, v_b = 0.0, 0.0 # b 的一阶矩和二阶矩
    # =====

    # 初始化历史记录
    history = {'w': [w], 'b': [b], 'loss': [np.mean((w*X + b - y)**2)]}

    # 进行迭代训练
    for epoch in range(epochs):
        t = epoch + 1 # Adam 的时间步 t 从 1 开始，用于偏差校正

        # 1. 计算当前预测值
        y_pred = w * X + b

        # 2. 计算当前梯度（这部分和普通梯度下降完全一样）
        dw = (2/n) * np.sum((y_pred - y) * X)
        db = (2/n) * np.sum(y_pred - y)

        # ===== Adam 核心参数更新逻辑 =====
        # 3. 更新一阶矩估计（动量：累积历史梯度方向）
        m_w = beta1 * m_w + (1 - beta1) * dw
        m_b = beta1 * m_b + (1 - beta1) * db

        # 4. 更新二阶矩估计（RMSProp：累积历史梯度平方）
        v_w = beta2 * v_w + (1 - beta2) * (dw**2)
        v_b = beta2 * v_b + (1 - beta2) * (db**2)

        # 5. 计算偏差校正后的一阶和二阶矩（解决初期向 0 偏移的问题）
        m_w_hat = m_w / (1 - beta1**t)
        m_b_hat = m_b / (1 - beta1**t)
        v_w_hat = v_w / (1 - beta2**t)
        v_b_hat = v_b / (1 - beta2**t)
```

```

# 6. 使用 Adam 公式更新参数
w -= lr * m_w_hat / (np.sqrt(v_w_hat) + epsilon)
b -= lr * m_b_hat / (np.sqrt(v_b_hat) + epsilon)
# =====

# 计算当前 loss
current_loss = np.mean((w*X + b - y)**2)

# 记录参数和损失
history['w'].append(w)
history['b'].append(b)
history['loss'].append(current_loss)

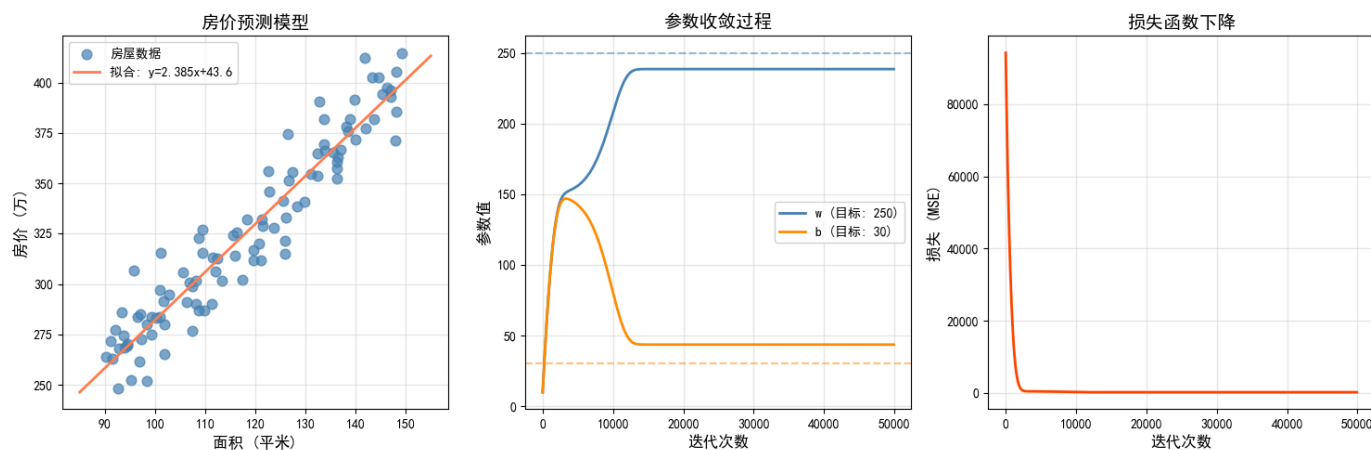
# 每隔 1% 训练轮数, 打印一次进度 (防止epochs较小报错, 加个max限制)
print_step = max(1, int(epochs / 100))
if epoch % print_step == 0:
    print(f"迭代 {epoch:5d}: w={w:.4f}, b={b:.4f}, loss={current_loss:.4f}")

return history

# ===== 运行测试 =====
history = adam_optimizer(areas, prices, lr=0.1, epochs=50000)

print(f"\n最终结果: 房价 = {history['w'][-1]/100:.3f} × 面积(平米) + {history['b'][-1]:.2f}")

```



相对 SGD 过程, Adam 过程收敛速度更快

- SDG 在 $2e+4$ 后收敛
- Adam 在 $2e+4$ 前收敛

梯度下降变体

每次更新参数需要使用多少数据计算梯度?

维度	BGD (全量)	Mini-batch	SGD (单样本)
每步计算量	大	适中	小
梯度准确度	精确	近似	噪声大
收敛稳定性	很稳定	较稳定	震荡剧烈
能否利用GPU并行	能	能且最佳	否
实际使用	小数据集	主流选择	很少单独使用

[!TIP]

Mini-batch 是深度学习训练的主流选择

[!CAUTION]

实际有时 SGD 指小批量梯度下降 (Mini64 / Mini128)

批量梯度下降 BGD

批量梯度下降 (Batch Gradient Descent, BGD) / 全批量

批量梯度下降的做法最为直接——每次用全部训练数据计算梯度，然后更新一次参数，这也正是我们前面实战中所采用的方法。

由于使用了完整数据集，它计算出的梯度最为准确，更新方向也最稳定，对于凸函数而言一定能收敛到最优解。

然而，这种面面俱到的方式代价高昂：当数据量很大时（想象有100万条数据），每走一步都需要遍历整个数据集，计算开销巨大；同时它也无法进行在线学习，必须拿到所有数据后才能开始训练。

随机梯度下降 SGD

随机梯度下降 (Stochastic Gradient Descent, SGD) / 单样本

随机梯度下降走向了另一个极端——每次只用一条随机抽取的训练数据计算梯度，然后立刻更新参数。

使得每步计算极其轻量，同时天然引入的随机性有助于模型跳出局部最优，还支持在线学习。

但仅凭单条数据估计的梯度噪声很大，更新方向不够稳定，导致收敛过程呈现剧烈震荡，整体效率反而可能受损。

小批量梯度下降 Mini-BGD

小批量梯度下降 (Mini-batch Gradient Descent, Mini-BGD)

小批量梯度下降则是对前两者的折中。它每次随机抽取一个小批量 (mini-batch) 的数据，通常为 32、64、128 或 256 条——来计算梯度并更新参数。

既保留批量梯度下降中梯度估计相对准确、方向相对稳定的优势，又兼顾了随机梯度下降计算高效、具备一定随机性的长处。

深度学习的基础

传统机器学习、深度学习、大语言模型等训练和优化，都依赖梯度下降

梯度下降是深度学习模型的底层逻辑

反向传播

深度神经网络或有百万级甚至十亿级参数

训练如此规模的网络模型，核心依赖如下步骤：

1. 前向传播 (Forward Propagation)
计算输出值和损失值
输入数据，从输入层逐层至隐藏层，最终得到输出和损失值
2. 反向传播 (Back Propagation)
利用链式法则计算梯度
从损失函数出发，利用链式法则逐层计算每个参数的梯度
3. 梯度下降 (Gradient Descent)
用算得的梯度更新所有参数

[!TIP]

深度学习训练 = 反向传播 + 梯度下降

大语言模型

自简单的线性回归，至如今的大语言模型 (LLM)

机器学习的基本范式从未改变

1. 用数据寻找规律
2. 用模型表达规律
3. 用损失函数衡量误差
4. 用梯度下降优化参数
5. 用不断迭代更新模型

相较线性回归的梯度下降，对 LLM 的训练过程基本一致

只是模型规模更大

- 模型框架复杂 (Transformer 架构)

- 参数规模庞大 (千亿级别)
- 数据规模庞大 (全互联网)
- 损失函数不同 (交叉熵)
- 优化器更先进 (AdamW)

核心思路一致

沿着梯度方向更新参数，实现最小损失函数。

04. 数据预处理

本节内容

1. 数据预处理的重要性
2. 工具与数据准备
3. 数据划分与防泄漏
4. EDA：数据探索分析
5. 数据清理：缺失值处理
6. 异常值检测与处理
7. 特征编码方法
8. 特征构造与删除
9. 特征缩放方法
10. 流程封装与总结

预处理的意义

[!CAUTION]

“Garbage In, Garbage Out.”

垃圾进，垃圾出

现实世界的的数据常有问题：

1. 缺失值或异常值
2. 非数值类型的数据，如文本、类别等
3. 量纲差异巨大

常用工具

1. Pandas
2. Matplotlib
3. Seaborn
4. Scikit-Learn (sklearn)

准备环境与数据

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 设置绘图风格
sns.set_style("whitegrid")
plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial Unicode MS', 'DejaVu Sans'] # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号

# 从网络读取泰坦尼克号数据集
url = 'https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv'
df = pd.read_csv(url)

# 查看数据概览
print("数据集形状:", df.shape)
print("\n前5行数据:")
display(df.head())
```

```
print("\n数据类型与缺失值情况:")
print(df.info())
```

数据集划分

先划分数据集，再做预处理
数据集划分为三部分

数据集	占比	用途
训练集	60%~80%	训练模型，学习数据中的规律
验证集	10%~20%	调整超参数，评估模型在陌生数据上的表现
测试集	10%~20%	模拟真实世界数据，最终评估模型泛化能力

[!CAUTION]
防止数据泄露 (Data Leakage) 原则
所有 `fit` 操作只在训练集内进行
测试集内只做 `transform` 操作

```
from sklearn.model_selection import train_test_split

# 定义特征 X 和目标标签 y (Survived)
X = df.drop('Survived', axis=1)
y = df['Survived']

# 划分训练集和测试集 (80% 训练, 20% 测试)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,      # 固定随机种子, 保证可复现
    stratify=y            # 分层抽样, 保持正负样本比例一致
)

print(f"训练集: {X_train.shape[0]} 样本") # 712
print(f"测试集: {X_test.shape[0]} 样本")  # 179
print(f"\n训练集存活率: {y_train.mean():.2%}") # ≈38.34%
print(f"测试集存活率: {y_test.mean():.2%}")   # ≈38.55%
```

`stratify` vt. 分层；成层；阶层化

设置 `stratify = y` 后, `sklearn` 会确保两者中正负样本比例与原始数据保持一致。
接下来所有探索和规则制定 (EDA、计算均值等) 必须只在 `X_train` 上进行!

数据探索分析 (EDA)

- 数据探索分析 (Exploratory Data Analysis)
类似对数据的全面体检
通过统计指标和可视化图表，直观发现数据的问题和规律，为后续预处理提供依据

基本信息

使用 `.info()` 查看数据集的数据类型和缺失情况

- 数据类型

```
# 查看训练集数据类型和缺失情况
X_train.info()
```

- 缺失比例

```
# 缺失值统计
missing = X_train.isnull().sum()
missing_pct = (missing / len(X_train) * 100).round(1)
pd.DataFrame({"缺失数": missing, "缺失率%": missing_pct}).query("缺失数 > 0")
```

数值特征统计

```
X_train.describe()
```

注意 Fare 最大 512, 最小 0, 量纲差异巨大, 后面需要缩放。

数据可视化

使用 `Matplotlib` 和 `seaborn` 对数据进行可视化, 发现更多细节

```
fig, axes = plt.subplots(2, 3, figsize=(15, 10))

# 1. 存活率分布
y_train.value_counts().plot.bar(ax=axes[0,0], color=['#e74c3c', '#2ecc71'])
axes[0,0].set_title('存活率分布')
axes[0,0].set_xticklabels(['Dead (0)', 'Survived (1)'], rotation=0)

# 2. 年龄分布
X_train['Age'].hist(bins=30, ax=axes[0,1], color='steelblue', edgecolor='black')
axes[0,1].set_title('年龄分布')

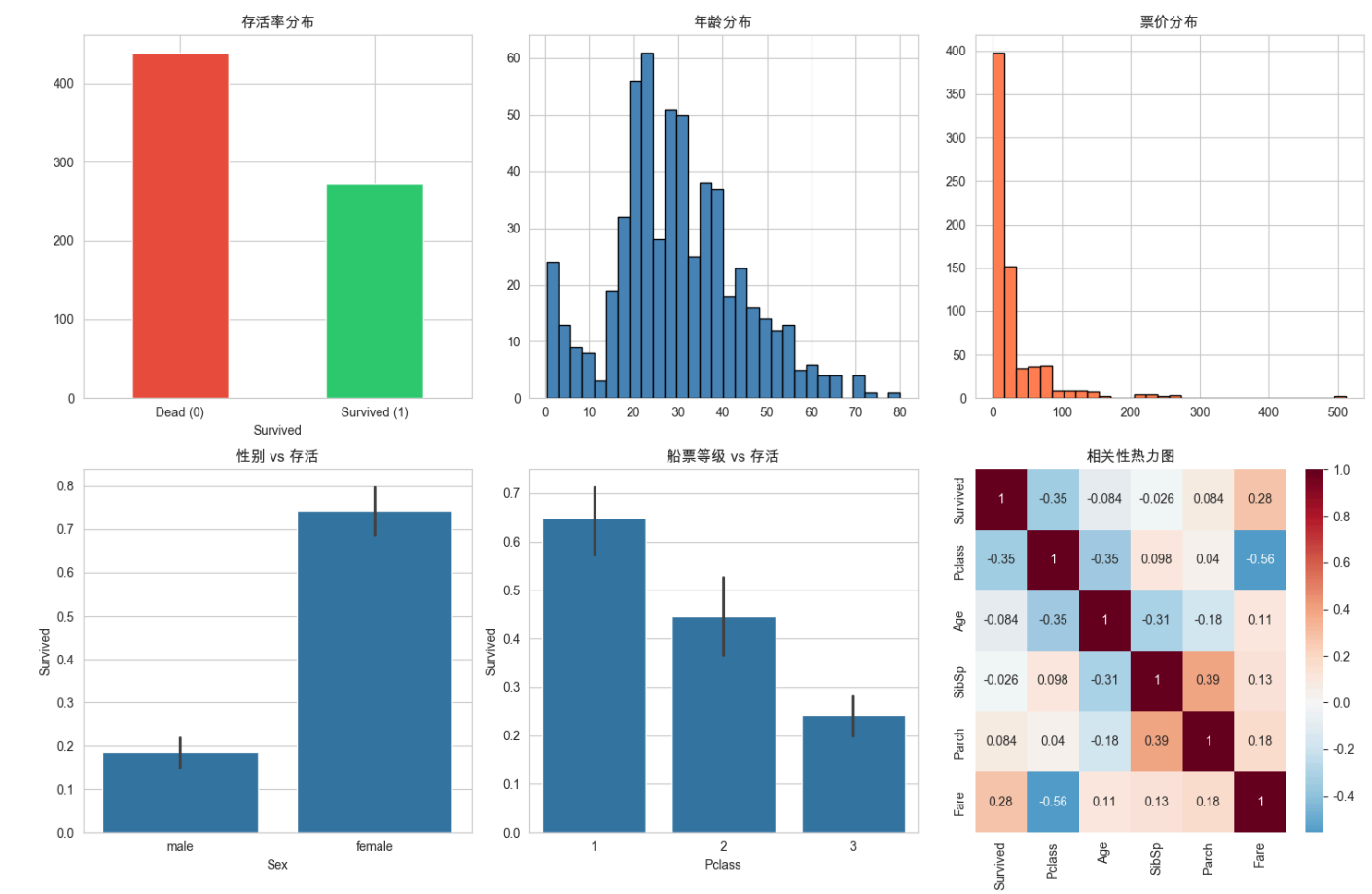
# 3. 票价分布
X_train['Fare'].hist(bins=30, ax=axes[0,2], color='coral', edgecolor='black')
axes[0,2].set_title('票价分布')

# 4. 性别 vs 存活
train_data = X_train.copy()
train_data['Survived'] = y_train
sns.barplot(x='Sex', y='Survived', data=train_data, ax=axes[1,0])
axes[1,0].set_title('性别 vs 存活')

# 5. 船票等级 vs 存活
sns.barplot(x='Pclass', y='Survived', data=train_data, ax=axes[1,1])
axes[1,1].set_title('船票等级 vs 存活')

# 6. 相关性热力图
numeric_cols = train_data[['Survived', 'Pclass', 'Age', 'SibSp', 'Parch', 'Fare']]
sns.heatmap(numeric_cols.corr(), annot=True, cmap='RdBu_r', center=0, ax=axes[1,2])
axes[1,2].set_title('相关性热力图')

plt.tight_layout()
plt.show()
```



EDA 关键发现

发现	处理
女性存活率远高于男性 (~74% vs ~19%)	Sex 是强特征，需编码为数值
一等舱存活率最高 (~63%)	Pclass 是强特征，考虑保留或独热编码
Age 缺失约 20%	需要填充，分布略偏态，选用中位数
Cabin 缺失 77%	直接填充意义不大，考虑转化为"有/无客舱信息"
Fare 严重右偏 (均值 > 中位数，长尾分布)	可能需要对数变换或截断处理
PassengerId、Name、Ticket 对预测无直接帮助	考虑删除，或从 Name 中提取有用信息

数据清洗

数据清洗的核心环节通常涵盖

- 1. 剔除重复值
- 2. 填补缺失值
- 3. 处理异常值

处理重复值

对于数据中重复的行删除即可

```
# 检查重复行
duplicates = X_train.duplicated().sum()
print(f"重复行数: {duplicates}")

# 如果有重复行
if duplicates > 0:
```

```
X_train = X_train.drop_duplicates()
y_train = y_train[X_train.index] # 保持 y 与 X 同步
```

`X_train`和`y_train`是靠索引 (Index) 对齐的

当从`X_train`删除了某一行，`y_train`也要删除对应的行。

处理缺失值

当数据集中出现缺失值时，处理策略需根据缺失比例及特征类型灵活选择：

- **按缺失比例决定删除策略**
 1. 缺失极低 (< 5%) 且为随机缺失 → 直接删除对应样本行
 2. 缺失极高 (> 70%) → 该特征信息量匮乏，通常直接删除整列
- **按特征类型选择填充方法**
 1. **数值型特征**

用均值或中位数填充。

均值适用于正态分布且无异常值的场景；

中位数对极端值不敏感，在数据偏态或存在异常值时更加稳健
 2. **类别型特征**

用众数填充，或新增 "Unknown" 类别。

众数填充适合低缺失率场景；

将缺失独立标记为 "Unknown" 则能保留"缺失本身可能蕴含的业务信息"

`sklearn` 提供 `SimpleImputer` 工具类处理缺失值，它可以扫描数据列，发现空缺后，根据设定的 `strategy`，自动计算一个值填进去

1. 初始化时设置 `strategy`
2. 在训练集上调用 `.fit()` 算出均值/中位数/众数，保存在对象内部
3. 在测试集/新数据上调用 `.transform()`：将保存的数据进行填充

```
from sklearn.impute import SimpleImputer

# — Age: 中位数填充 (因为 Age 分布略有偏态) —
age_imputer = SimpleImputer(strategy='median')
# fit 只用训练集!
age_imputer.fit(X_train[['Age']])
print(f"训练集 Age 中位数: {age_imputer.statistics_[0]}") # 28.0

X_train['Age'] = age_imputer.transform(X_train[['Age']]).ravel()
X_test['Age'] = age_imputer.transform(X_test[['Age']]).ravel() # 用训练集的中位数!

# — Embarked: 众数填充 (只缺 2 个) —
embarked_imputer = SimpleImputer(strategy='most_frequent')
embarked_imputer.fit(X_train[['Embarked']])
print(f"训练集 Embarked 众数: {embarked_imputer.statistics_[0]}") # 'S'

X_train['Embarked'] = embarked_imputer.transform(X_train[['Embarked']]).ravel()
X_test['Embarked'] = embarked_imputer.transform(X_test[['Embarked']]).ravel()

# — Cabin: 缺失 77%, 先转化为"有无客舱信息", 再删除原列 —
X_train['HasCabin'] = X_train['Cabin'].notna().astype(int)
X_test['HasCabin'] = X_test['Cabin'].notna().astype(int)

X_train.drop(columns=['Cabin'], inplace=True)
X_test.drop(columns=['Cabin'], inplace=True)

# 验证
print("\n处理后缺失值:")
print(X_train.isnull().sum())
```

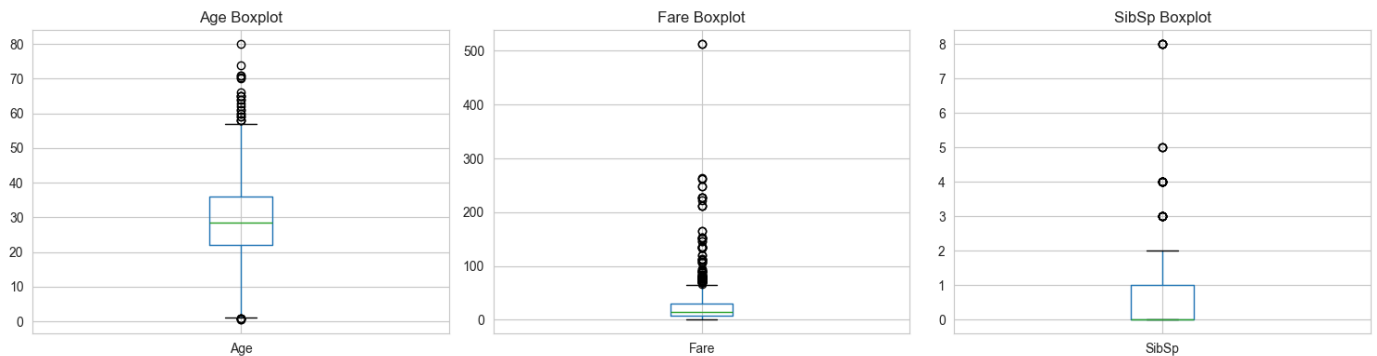
处理异常值

异常值 (Outliers) 是指数据中显著偏离其他观测值的极端值
它们可能是由于测量误差、数据录入错误或真实的极端情况产生的
可以使用**箱线图**检测异常值

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for i, col in enumerate(['Age', 'Fare', 'SibSp']):
    X_train.boxplot(column=col, ax=axes[i])
    axes[i].set_title(f'{col} Boxplot')

plt.tight_layout()
plt.show()
```



1. Age (年龄):

- 大部分乘客集中在 20-40 岁之间 (箱体部分)
- 但在 60-80 岁之间有很多离群点
这说明船上有不少高龄乘客, 虽然人数少, 但他们是真实存在的, 不是数据错误

2. Fare (票价):

- 极度右偏**
绝大多数人的票价非常低 (箱体被压扁在底部, 接近 0-50 美元)
- 但是有极少数人的票价高得离谱, 超过 500 美元! 这些可能是一等舱乘客
- 对模型的影响**
极端数值会让模型难以训练, 若不处理 (比如取对数 **log** 或分箱), 模型则可能会过度关注这几个高价票乘客, 而忽略普通大众

3. SibSp (兄弟姐妹/配偶数):

- 大部分人都是 0 或 1, 独自旅行或夫妻同行
- 但也有一些大家庭, 带着 3、4、5 甚至 8 个亲属, 这也是真实的异常值

使用统计学方法 (IQR) 识别并处理票价 (Fare) 中的极端异常值, 防止它们干扰机器学习模型的训练。
不必直接删除这些异常数据, 采用截断的策略, 将过高的票价强行拉回到合理范围内即可。

```
# 用 IQR 方法检测
def detect_outliers_iqr(data, column, factor=1.5):
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
    lower = max(0, Q1 - factor * IQR)
    upper = Q3 + factor * IQR
    outliers = data[(data[column] < lower) | (data[column] > upper)]
    print(f"{column}: Q1={Q1:.1f}, Q3={Q3:.1f}, IQR={IQR:.1f}")
    print(f"  合理范围: [{lower:.1f}, {upper:.1f}]")
    print(f"  异常值数量: {len(outliers)} ({len(outliers)/len(data):.1%})")
    return lower, upper

lower, upper = detect_outliers_iqr(X_train, 'Fare')

# 对于 Fare, 我们选择截断而非删除
# 因为高票价可能是真实的, 头等舱确实很贵
X_train['Fare'] = X_train['Fare'].clip(lower=lower, upper=upper)
X_test['Fare'] = X_test['Fare'].clip(lower=lower, upper=upper)
```

特征编码

由于大多数机器学习算法仅支持数值型输入，因此必须将数据集中文本数据通过编码技术映射为数值形式，以便模型能够进行数学运算。

针对不同场景有三种常见的特征编码方法

1. 序数编码 (Ordinal Encoding)
2. 独热编码 (One-Hot Encoding)
3. 标签编码 (Label Encoding)

这三种编码方式的核心区别在于是否保留了类别的顺序信息以及是否引入了虚假的数值大小关系。

选择哪种方式，取决于你的数据特征类型和你使用的模型算法。

展示对原始数据进行特征编码

```
=== 原始数据 ===
   survived    sex  pclass  embark_town
0          0  male        3   Southampton
1          1 female        1    Cherbourg
2          1 female        3   Southampton
3          1 female        1   Southampton
4          0  male        3   Southampton
```

序数编码 Ordinal Encoding

序数编码将类别映射为整数，且严格保留类别之间的顺序关系。

比如 低 $\rightarrow$ 0, 中 $\rightarrow$ 1, 高 $\rightarrow$ 2

适用于特征本身具有明确的等级或顺序，希望模型能够理解 “高 > 中 > 低” 这种数学关系

比如

- 教育程度
小学(1) < 初中(2) < 高中(3) < 本科(4) < 硕士(5) < 博士(6)
- 满意度评分
非常不满意(0) < 不满意(1) < 一般(2) < 满意(3) < 非常满意(4)
- 衣服尺码
XS(1) < S(2) < M(3) < L(4) < XL(5)
- 飞机座舱
经济舱(1) < 商务舱(2) < 头等舱(3)

我们对 Titannic 的 Fare 数据做下处理

1等舱 > 2等舱 > 3等舱

`sklearn` 的 `OrdinalEncoder` 工具类默认按字母顺序或出现顺序，这里我们手动指定 `categories` 以保证逻辑正确

`Pclass` 本身已经是数字 1, 2, 3，为了演示效果，我们构造一个模拟的 “等级” 列: `['Low', 'Mid', 'High']`

```
from sklearn.preprocessing import OrdinalEncoder

df_ordinal = df.copy()
df_ordinal['fare_category'] = pd.cut(df['Fare'], bins=3, labels=['Low', 'Mid', 'High'])

oe = OrdinalEncoder(categories=[['Low', 'Mid', 'High']]) # 显式指定顺序
df_ordinal['fare_cat_encoded'] = oe.fit_transform(df_ordinal[['fare_category']])

print(f"顺序定义: Low < Mid < High -> 0, 1, 2")
print(df_ordinal[['fare_category', 'fare_cat_encoded']].head())
```

独热编码 One-Hot Encoding

线性模型（如线性回归、逻辑回归、SVM、神经网络）通过计算数值的大小和距离来学习规律。

如果对无序分类特征（如颜色）直接使用序数编码，

比如把颜色标记为：红=1, 绿=2, 蓝=3，

模型会认为蓝色比红色大 3 倍，或者绿色介于红蓝之间。
对于颜色这种没有大小之分的概念，这种假设是完全错误的，会导致模型训练偏差。

- 独热编码的核心思想
 - 把每个类别取值都变成一个独立的列
 - 不再用一个大小的数字，而是用一组 0 和 1 来表示

假设我们有一个颜色特征，包含红、绿、蓝三种取值，转换效果如下：

样本	颜色 (原始文本)	错误的序号编码
A	● 红	1
B	● 绿	2
C	● 蓝	3

独热编码转换为

样本	是红色	是绿色	是蓝色
A	1	0	0
B	0	1	0
C	0	0	1

如此，每种颜色都是平等的，没有任何大小顺序之分。

独热编码虽然好，但有个缺点：有 N 种颜色，就要增加 N-1 列。
如果只有 3 种颜色，增加 2 列没问题。
如果有 1000 种城市名称，就会增加 999 列！
这会让数据变得非常稀疏，计算变慢甚至导致内存溢出。
仅当类别数量较少时使用独热编码，类别很多可以考虑 Embedding。

```
from sklearn.preprocessing import OneHotEncoder

df_ohe = df.copy()

ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
# 注意：输入必须是二维数组，所以用 [['embark_town']]
ohe_result = ohe.fit_transform(df_ohe[['embark_town']])

# 获取生成的列名（例如：embark_town_Cherrybourg）
ohe_columns = ohe.get_feature_names_out(['embark_town'])

# 转换为 DataFrame 以便合并
ohe_df = pd.DataFrame(ohe_result, columns=ohe_columns, index=df_ohe.index)

# 将独热编码列拼接回主表，并删除原始文本列
df_final = pd.concat([df_ohe.drop('embark_town', axis=1), ohe_df], axis=1)

print("\n=== One-Hot Encoding 演示 (Embark Town) ===")
print(f"生成的新列: {list(ohe_df.columns)}")
print(df_final.head())
```

标签编码 Label Encoding

标签编码将类别简单映射为整数（0, 1, 2, ...）
虽然不保证顺序，但引入了数值大小

适用场景仅限于三类情况

- 1. 二分类特征
 - 如性别，0/1等价于独热且更省空间

2. 分类任务的目标变量/标签列
大多数分类算法要求目标变量必须是整数标签
3. 树模型
如XGBoost、随机森林等，因其不依赖数值距离，可处理高维无序类别

严禁将标签编码用于线性模型和多分类无序特征，此类情况必须改用独热编码和序数编码。

```
from sklearn.preprocessing import LabelEncoder

# 性别是二分类特征，可以使用标签编码
le = LabelEncoder()
X_train_encoded = X_train.copy()
X_train_encoded['Sex'] = le.fit_transform(X_train['Sex']) # male=1, female=0
```

特征构建与删除

原始数据往往只是原材料，直接丢给模型效果通常并不理想

很多时候我们需要对现有的特征进行合并、删除等操作，才能挖掘出数据背后真正的规律

基于对泰坦尼克号背景的理解，做三件事：

1. 合并特征
将分散的亲属数量合并为“家庭规模”，并标记出“独自一人”的乘客，因为家庭结构可能直接影响逃生策略
2. 提取信息
从看似无用的姓名中提取出头衔（如 Mr, Mrs, Master），识别出社会地位和特殊群体
3. 删减噪音
果断删除那些缺失太多、格式混乱或者会导致数据泄露的列，只保留最核心的特征

```
def create_features(df):
    df = df.copy()

    # 1. 家族大小
    df['family_size'] = df['sibsp'] + df['parch'] + 1

    # 2. 是否独自一人
    df['is_alone'] = (df['family_size'] == 1).astype(int)

    # 3. 从姓名提取头衔 (Mr, Mrs, Miss, Master 等)
    df['title'] = df['name'].str.extract(r' ([A-Za-z]+\.)', expand=False)
    # 归并稀有头衔
    df['title'] = df['title'].replace(['Lady', 'Countess', 'Capt', 'Col', 'Don', 'Dr',
                                       'Major', 'Rev', 'Sir', 'Jonkheer', 'Dona'], 'Rare')
    df['title'] = df['title'].replace('Mlle', 'Miss')
    df['title'] = df['title'].replace('Ms', 'Miss')
    df['title'] = df['title'].replace('Mme', 'Mrs')

    # 4. 删除无用特征
    drop_cols = ['name', 'ticket', 'cabin', 'deck', 'alive', 'adult_male', 'who', 'class', 'embark_town']
    df = df.drop(columns=[c for c in drop_cols if c in df.columns])

    return df

X_train_fe = create_features(X_train)
X_test_fe = create_features(X_test)

print("新增特征后形状:", X_train_fe.shape)
print(X_train_fe[['family_size', 'is_alone', 'title']].head())
```

特征缩放

Titanic 数据集部分特征量纲差异极大

```
Age:      0 ~ 80      (十位数)
Fare:     0 ~ 100+    (百位数)
SibSp:    0 ~ 8       (个位数)
Embarked_C: 0 or 1    (0-1)
```

直接使用这样的数据进行训练，模型会错误地认为数值较大的特征（如 Fare）比数值较小的特征（如 Age）更重要，从而赋予其过大的权重。损失函数的等高线会呈现极度狭长的椭圆形。

导致梯度下降在更新参数时反复震荡，需要极多迭代次数才能找到最优解，甚至可能无法收敛。

为了解决这一问题，我们通常需要特征数据进行缩放，以下两种处理方式最为常见

1. 标准化 Standardization
2. 归一化 Normalization

标准化 Standardization

这是最常用的方法，尤其适用于数据分布近似正态分布的场景。

它将数据转换为均值为 0，标准差为 1 的分布。

$$z = \frac{x - \mu}{\sigma}$$

标准化不会将数据强制压缩到特定区间，因此处理后的数值可以是负数。

[!CAUTION]

因为标准化基于均值和标准差计算，所以容易受到异常值的影响。

得益于数据清洗对异常值的处理，标准化相对而言比较稳定，是线性模型、逻辑回归、神经网络和支持向量机 (SVM) 等算法的首选预处理方法。

```
from sklearn.preprocessing import StandardScaler

# 选取需要缩放的数值型列（例如 'Age' 和 'Fare'）
numeric_cols = ['Age', 'Fare']

# 标准化
scaler = StandardScaler()

# 只在训练集上 fit，获取均值和标准差
scaler.fit(X_train[numeric_cols])

# 用相同的参数转换训练集和测试集
X_train_scaled = X_train.copy()
X_test_scaled = X_test.copy()

X_train_scaled[numeric_cols] = scaler.transform(X_train[numeric_cols])
X_test_scaled[numeric_cols] = scaler.transform(X_test[numeric_cols])

print("标准化完成")
print(X_train_scaled[numeric_cols].head())

print(f"\n均值检查（应接近0）: {X_train_scaled[numeric_cols].mean().values}")
print(f"标准差检查（应接近1）: {X_train_scaled[numeric_cols].std().values}")
```

归一化 Normalization

归一化将数据线性变换到一个固定的区间，通常是

$$0, 1$$
$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

- 优点
 - 能严格限定数据范围，确保结果不会出现负数。
- 缺点
 - 受异常值影响明显
 - 如果数据中混入一个极大的离群点，会导致其他正常数值失去原有的分布特征。
- 适用
 1. 图像数据处理
 - 像素天然有 0~255 的边界
 2. 作为神经网络的输入
 - Sigmoid 等激活函数是有界的
 3. 在算法强制要求数据范围时

工程实践

前面的步骤虽然清晰，但手动操作容易出错且难以维护

`sklearn` 提供提供 `Pipeline` 整合流程，用 `ColumnTransformer` 对不同类型列做不同处理意义：

1. 防止数据泄露
 - 自动保证 `fit` 只在训练集
 - `transform` 应用到测试集
2. 代码简洁
 - 将预处理和模型封装在一起
3. 便于部署
 - 保存一个 `Pipeline` 对象即可用于生产环境预测

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

# 重新加载干净的原始划分数据（为了演示 Pipeline）
X_train, X_test, y_train, y_test = train_test_split(
    df.drop('Survived', axis=1),
    df['Survived'],
    test_size=0.2, random_state=42
)

# 定义我们要处理的列
numeric_features = ['Age', 'Fare', 'SibSp', 'Parch']
categorical_features = ['Sex', 'Embarked', 'Pclass'] # Pclass虽是数字，但其实是类别(1,2,3等舱)

# 1. 为数值列创建流水线：先填补缺失值 -> 再做标准化
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

# 2. 为类别列创建流水线：先填补缺失值 -> 再做独热编码
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(drop='first', handle_unknown='ignore')) # 忽略测试集中没见过的新类别
])

# 3. 使用 ColumnTransformer 将两条流水线组合起来
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ])

# 4. 一键处理！
# fit_transform 训练集
X_train_ready = preprocessor.fit_transform(X_train)
# transform 测试集
X_test_ready = preprocessor.transform(X_test)
```

```
print("预处理流水线搭建成功！生成的矩阵形状:", X_train_ready.shape)
```

原则总结

本节阐明数据预处理的全流程，可分四条核心原则：

1. Garbage In, Garbage Out

数据质量决定模型上限，再好的算法也无法从垃圾数据中学到有价值的规律

2. 防止数据泄露

先划分数据集，所有 `fit` 操作仅在训练集上进行，测试集只做 `transform`

3. EDA 驱动

通过 EDA 观测数据实际分布，根据业务含义选择填充策略、编码方式和特征变换

4. Pipeline 封装

将预处理、特征工程和模型训练封装为流水线，确保流程可复用、无泄露、易部署

05. 模型评估

本节内容

1. 评估模型的必要性
2. 建立基线模型
3. 回归评估指标详解
4. 基线模型评估与残差分析
5. 交叉验证诊断
6. 过拟合与欠拟合预告

基线模型 Baseline

基线模型的意义

- 基线模型 Baseline
作为深入评估的可靠起点，是后续优化的参照基准
是首个快速实现、相对简单，但具备完整流程的模型

加州房价案例

- 样本规模
20,640 条记录
- 特征维度
 1. 家庭收入
 2. 房龄
 3. 房间数量
 4. 卧室数量
 5. 街区人口
 6. 家庭人口
 7. 纬度
 8. 经度
- 预测目标
各街区房屋价格中位数，单位为10万美元
目标变量的最大值刚好是 5.00
这是数据采集时做了**截断操作**，所有超过 50 万美元的房价都被统一记录为 5.0
这个操作会在后续的残差分析中产生明显的`[#^DataTruncationEffect|截断效应]`。`^DataTruncationOperate`

数据集加载及检视

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler

# 加载数据
housing = fetch_california_housing()
X = pd.DataFrame(housing.data, columns=housing.feature_names)
y = housing.target # 房价中位数 (单位: 10万美元)

print(f"样本数: {X.shape[0]}, 特征数: {X.shape[1]}")
print(f"目标变量范围: [{y.min():.2f}, {y.max():.2f}], 均值: {y.mean():.2f}")
print(X.head())
```

数据集预处理

```
# 标准化 + 划分数据集
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

print(f"训练集: {X_train.shape[0]} 样本")
print(f"测试集: {X_test.shape[0]} 样本")

# 训练 baseline 模型
model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

检视预测情况

```
for i in range(10):
    print(f"真实值: {y_test[i]}, 预测值: {y_pred[i]}")
```

`y_pred`是模型对测试集的预测结果, `y_test`是真实值

评估要解决的问题:

1. 计算二者差异
2. 如何量化差异
3. 区分二者差异

回归模型评估指标

模型训练完成后, 需要在测试集上评估其泛化能力

回归任务有四个常用的核心指标

1. MAE
2. MSE
3. RMSE
4. R^2

不同指标对误差的敏感度不同, 需结合业务场景选择

均值绝对误差 MAE

均值绝对误差 (Mean Absolute Error, MAE)

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- 核心思想
计算预测值和真实值的偏差的绝对值，再求平均
- 优点
 1. 预测值与目标变量的单位一致，直观易懂
 2. 对**异常值相对不敏感**，不会过度放大极端错误
- 缺点
绝对值在零点不可导，作为损失函数在优化过程中不如平方项方便

[!TIP]

它的单位和目标变量一致，是所有指标中最容易解释的。

均方误差 MSE

均方误差 (Mean Squared Error, MSE)

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- 核心思想
计算预测值与真实值偏差的平方，再求平均
- 优点
平方操作会**呈指数级放大极大误差的影响**
适用于对大误差零容忍的场景，如医疗、自动驾驶等应用
- 缺点
单位在平方后会失去直观的业务含义，如“平方万元”

和 MAE 的思路类似，MSE 但把取绝对值换成了取平方，这个平方操作会放大大误差的影响：
偏差为 10 的样本，在 MSE 中贡献的惩罚是偏差为 1 的样本的 100 倍，而在 MAE 中只是 10 倍。

所以，如果你的业务场景对大偏差零容忍（比如预测偏差超过某个阈值就会造成严重后果），MSE 是更合适的优化目标。
不过由于平方操作，MSE 的单位变成了目标变量单位的平方，不太直观。

[!TIP]

MSE 也是梯度下降训练时最常用的损失函数。

均方根误差 RMSE

均方根误差 (Root Mean Squared Error, RMSE)

$$\begin{aligned} RMSE &= \sqrt{MSE} \\ &= \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \end{aligned}$$

- 核心思想
对 MSE 开根号
- 优点
 1. 单位还原，预测值的量纲与目标变量一致，容易解释业务含义
 2. 惩罚大误差，继承 MSE 对大偏差敏感的特点

把 MSE 开个根号，单位就回到和目标变量一致了。
既保留了 MSE 对大误差的敏感性，又方便直接解读

[!TIP]

在实际工程和数据科学竞赛 (Kaggle) 中，常作为回归任务的默认标准

决定系数 R^2

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

- 核心理念
 - 相对直接使用均值预测，模型的预测性能好多少
- 无量纲指标
 - 衡量模型消除了总方差中多大比例的不确定性，可在不同数据集间横比
- 指标解读
 1. $R^2 = 1$
 - 完美预测，没有误差
 2. $R^2 = 0$
 - 模型表现同均值预测等效，暂无习得有效特征
 3. $R^2 < 0$
 - 模型表现较均值预测低效，模型选择错误或者模型设计存在严重错误

基线模型评估结果

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f"MAE: {mae:.4f}    (平均偏差 {mae*10:.2f} 万美元) ")
print(f"MSE: {mse:.4f}")
print(f"RMSE: {rmse:.4f}    (平均偏差 {rmse*10:.2f} 万美元) ")
print(f"R²: {r2:.4f}    (解释了 {r2*100:.1f}% 的方差) ")
```

评估结果解读

- MAE = 0.5332
 - 意味着模型预测值平均偏离真实房价约 5,332 美元
- RMSE = 0.7456
 1. RMSE 大于 MAE，说明存在一些偏差较大的样本把 RMSE 拉高了
 2. RMSE 和 MAE 的差距越大，说明误差分布越不均匀，存在较大的离群误差
- $R^2 = 0.5757$
 1. 模型解释了约 57.6% 的房价方差
 2. 剩余 42.4% 的方差来自模型未捕获的因素——可能是特征不够、也可能是线性假设太强
 3. 对于只用 8 个特征的线性模型，这个结果是一个合理的起点，但还有明确的提升空间

指标选择原则

场景	推荐指标	原因
对大误差零容忍（如金融风控、安全领域）	MSE / RMSE	平方惩罚放大大误差，优化时会优先减少极端偏差
数据中存在离群点，需要稳健的度量	MAE	不受个别极端值的过度影响
不同量纲/不同数据集的模型横向对比	R^2	无量纲，归一化到 0~1 范围
向非技术背景的业务方汇报	RMSE / MAE	单位和目标变量一致，容易解释

残差分析

残差 Residual

- 误差指标给出全局性诊断的数字结果
- 残差分析能反映模型的具体优劣表现

$$e_i = y_i - \hat{y}_i$$

残差值 = 真实值 - 预测值

理想中完美模型所有的残值值均为零
现实中实际模型的残差值分布能反映数据的结构信息

```

residuals = y_test - y_pred

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# 图1: 预测值 vs 真实值
axes[0].scatter(y_test, y_pred, alpha=0.3, s=10, color='steelblue')
axes[0].plot([0, 5], [0, 5], 'r--', linewidth=2, label='Perfect prediction')
axes[0].set_xlabel('Actual', fontsize=12)
axes[0].set_ylabel('Predicted', fontsize=12)
axes[0].set_title('预测值与真实值对比图', fontsize=13)
axes[0].legend()

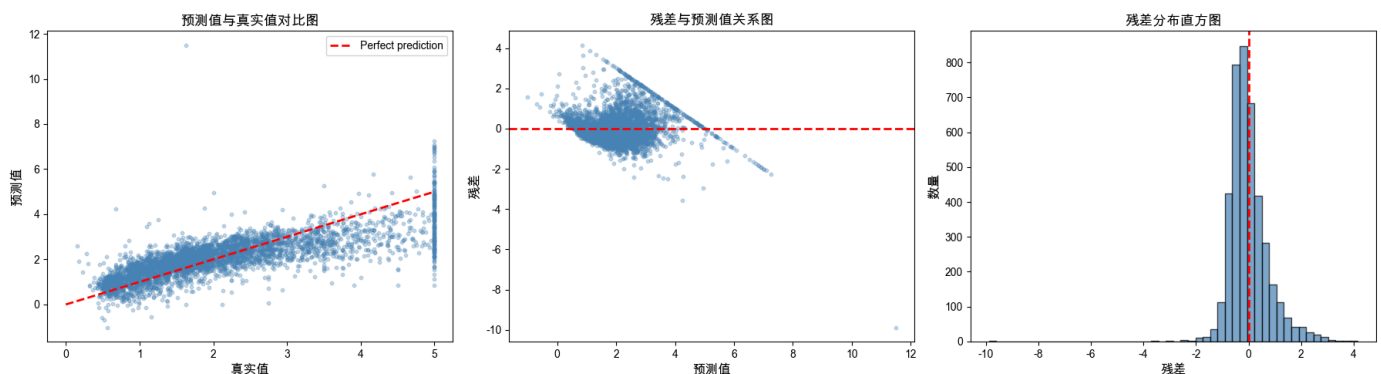
# 图2: 残差 vs 预测值
axes[1].scatter(y_pred, residuals, alpha=0.3, s=10, color='steelblue')
axes[1].axhline(y=0, color='r', linestyle='--', linewidth=2)
axes[1].set_xlabel('Predicted', fontsize=12)
axes[1].set_ylabel('Residuals', fontsize=12)
axes[1].set_title('残差与预测值关系图', fontsize=13)

# 图3: 残差分布
axes[2].hist(residuals, bins=50, edgecolor='black', color='steelblue', alpha=0.7)
axes[2].axvline(x=0, color='r', linestyle='--', linewidth=2)
axes[2].set_xlabel('Residual', fontsize=12)
axes[2].set_ylabel('Count', fontsize=12)
axes[2].set_title('残差分布直方图', fontsize=13)

plt.tight_layout()
plt.show()

# 残差统计
print(f"残差均值: {residuals.mean():.4f}")
print(f"残差标准差: {residuals.std():.4f}")
print(f"残差偏度: {pd.Series(residuals).skew():.4f}")

```



预测值与真实值对比图

- 理想情况
 - 所有点都应落在红色对角线上
- 实际情况
 - 图示数据点大致沿对角线分布，模型整体具备预测能力，但存在明显的系统性偏差：
 - 真实值较高时，点落在红线下方，即预测值普遍偏低，说明 **模型倾向于低估高价房产**
 - 右侧出现的**垂直聚集现象**，即大量真实值为 5.0 的样本，正是前面提到的`[[#^DataTruncationOperate|截断操作]]`产生的效应
^DataTruncationEffect

残差值与预测值关系图

- 理想情况
 - 如果模型没有系统性问题，散点应该均匀、随机地分布在红色零线两侧
- 实际情况
 - 随着预测值增大，残差的发散程度也在变大，呈现出漏斗的形状
 - 这说明 **模型预测高房价时非常不稳定**，误差忽大忽小。

残差值分布直方图

- 理想情况
理想的残差应近似正态分布、均值为 0
- 实际情况
图中残差的峰值确实集中在零附近，但整体呈现 **右偏且带有长尾**
再次印证 **模型对部分高房价样本预测不准、严重低估** 的问题。

交叉验证

剔除随机性带来的评估偏差

单次 `train/test split` 评估有依赖运气的隐患

若换一个 `random_state`， R^2 可能从 0.57 变成 0.60 或 0.55。

如何判断 0.57 是否反映模型的真实水平，而非单次划分运气所致？

为获取稳定、统计显著的评估结果，采用 K-Fold 交叉验证

K-Fold

- 将数据均分为 K 份
- 每轮取 1 份作测试集，剩余 $K-1$ 份作训练集
- 重复 K 轮后，最终报告 K 次评估结果的均值和标准差

单指标 RMSE

```
from sklearn.model_selection import cross_val_score

# 注意: cross_val_score 会在内部完成 fit 和 predict, 不需要手动划分
scores = cross_val_score(
    LinearRegression(), X_scaled, y,
    cv=5, # 5 折交叉验证
    scoring='neg_mean_squared_error' # sklearn 约定: scoring 越大越好, MSE 前面加负号
)

# 将负 MSE 转换回 RMSE
rmse_scores = np.sqrt(-scores)

print("每折 RMSE:")
for i, score in enumerate(rmse_scores):
    print(f"  Fold {i+1}: {score:.4f}")
print(f"\n平均 RMSE: {rmse_scores.mean():.4f} ± {rmse_scores.std():.4f}")
print(f"\n标准差 {rmse_scores.std():.4f} 相对于 平均值 {rmse_scores.mean():.4f} 约 {rmse_scores.std() / rmse_scores.mean() * 100 :.2f}%")
```

平均 RMSE 0.7459 ± 0.0437

- 平均值 0.7459
- 标准差 0.0437
- $\frac{\text{std}}{\text{mean}} = 5.86\%$

相对平均值，标准差极小（约5.9%）说明线性回归模型在不同数据划分下表现稳定

多指标 RMSE+MAE+R2

一次计算多个指标，可用 `cross_validation`

```
from sklearn.model_selection import cross_validate

cv_results = cross_validate(
    LinearRegression(), X_scaled, y,
    cv=5,
    scoring={
        'rmse': 'neg_root_mean_squared_error',
```

```
        'mae': 'neg_mean_absolute_error',
        'r2': 'r2'
    },
    return_train_score=True # 同时返回训练集上的得分（用于诊断过拟合）
)

print(f"{'指标':<12} {'训练集':>12} {'测试集':>12} {'差距':>10}")
print("-" * 48)
for metric in ['rmse', 'mae', 'r2']:
    train_key = f'train_{metric}'
    test_key = f'test_{metric}'

    if metric == 'r2':
        train_mean = cv_results[train_key].mean()
        test_mean = cv_results[test_key].mean()
    else:
        train_mean = -cv_results[train_key].mean() # 去掉负号
        test_mean = -cv_results[test_key].mean()

    gap = abs(train_mean - test_mean)
    print(f"{'metric.upper()':<12} {'train_mean':>12.4f} {'test_mean':>12.4f} {'gap':>10.4f}")
```

结果分析

指标	训练集	测试集	差距
RMSE	0.7218	0.7459	0.0241
MAE	0.5297	0.5475	0.0178
R2	0.6071	0.5530	0.0541

1. 训练集和测试集的指标非常接近
这种模式说明模型没有过拟合
过拟合的特征是训练集上表现很好、测试集上表现很差
2. 训练集上的表现本身也不算出色
 - R^2 只有 0.61
 - 模型可能 没有充分挖掘数据中的信息，即欠拟合

06. 拟合诊断

本节内容

1. 目标与泛化能力
2. 欠拟合
3. 过拟合
4. 诊断工具：学习曲线
5. 解决欠拟合
6. 解决过拟合
7. 核心方法：正则化
8. 正则化效果对比
9. 总结

泛化能力

1. 机器学习目标
训练得到能够捕捉数据背后潜在规律的模型
2. 泛化能力
模型不仅能拟合训练数据，还能对陌生的新数据做出准确预测

模型并不总能具备良好的泛化能力

- 学习太少会欠拟合
- 学习太多会过拟合

拟合问题

欠拟合 Underfitting

模型过于简单
模型未能充分学习训练数据中的规律
在训练集和测试集上的表现都很差

过拟合 Overfitting

模型过于复杂
模型不仅学习了数据中的真实规律，也将噪音一并记住
在训练集上表现优异，但在测试集上表现退步

二者对比

	恰好拟合	欠拟合	过拟合
训练集表现	✅ 好	❌ 差	✅ 很好
测试集表现	✅ 好	❌ 差	❌ 差
核心问题	充分习得数据规律	未充分学习到规律	将噪音作规律学习

学习曲线

学习曲线

- 是诊断拟合问题的直观工具
- 展示了随着训练样本数量增加，训练误差和验证误差的变化趋势

模拟数据

假设数据的真实规律是 $y = x^2$
在此基础上添加随机噪声，生成训练数据点
现用不同复杂度的多项式来拟合数据

- **degree=1:**
线性模型，无法捕捉二次关系，导致欠拟合
- **degree=2:**
二次多项式，与真实数据生成过程匹配，良好拟合
- **degree=15:**
高次多项式，过度拟合训练数据中的噪声，导致过拟合

准备数据

1. 首先引入必要的工具，并准备一下数据
2. 然后使用PolynomialFeatures给原始特征添加平方项、交叉项

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import learning_curve
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline

# 设置中文字体，解决matplotlib中文显示问题
plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial Unicode MS', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题

# ----- 数据准备 -----
np.random.seed(42) # 设置随机种子，保证结果可复现
n_samples = 100 # 样本数量

# 在[-3, 3]区间均匀采样100个点
X = np.sort(np.random.uniform(-3, 3, n_samples)).reshape(-1, 1)
```

```
# 真实关系:  $y = x^2 + \text{高斯噪声}$ 
y = X.ravel() ** 2 + np.random.normal(0, 1, n_samples)

# ----- 定义三个不同复杂度的模型 -----
models = {
    "欠拟合 (degree=1)": make_pipeline(PolynomialFeatures(1), LinearRegression()),
    "良好拟合 (degree=2)": make_pipeline(PolynomialFeatures(2), LinearRegression()),
    "过拟合 (degree=15)": make_pipeline(PolynomialFeatures(15), LinearRegression()),
}
```

直观对比

```
# 创建1行3列的子图，展示模型的拟合效果
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# 用于绘制平滑预测曲线的测试点
X_test = np.linspace(-3, 3, 200).reshape(-1, 1)

for ax, (title, model) in zip(axes, models.items()):
    # 训练模型
    model.fit(X, y)

    # 预测
    y_pred = model.predict(X_test)

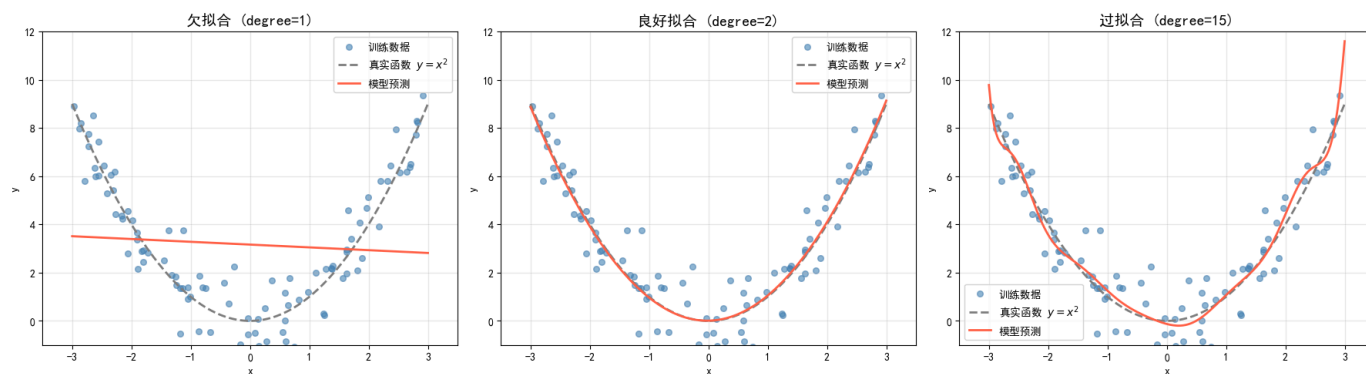
    # 绘制真实数据点 (带噪声的样本)
    ax.scatter(X, y, color='steelblue', s=30, alpha=0.6, label='训练数据')

    # 绘制真实函数曲线  $y = x^2$ 
    ax.plot(X_test, X_test.ravel() ** 2, '--', color='gray', linewidth=2, label='真实函数  $y=x^2$ ')

    # 绘制模型预测曲线
    ax.plot(X_test, y_pred, '-', color='tomato', linewidth=2, label='模型预测')

    ax.set_title(title, fontsize=14)
    ax.set_xlabel('x')
    ax.set_ylabel('y')
    ax.legend()
    ax.grid(True, alpha=0.3)
    ax.set_xlim(-3.5, 3.5)
    ax.set_ylim(-1, 12)

plt.tight_layout()
plt.savefig('fitting_comparison.png', dpi=150)
plt.show()
```



- 图 1 欠拟合
使用 $y = wx + b$
显然直线无法拟合抛物线
无论怎么调整斜率和截距，直线都无法捕捉数据的弯曲趋势

- 图 2 良好拟合
使用 $y = w_1x^2 + w_2x + b$
二次多项式与真实规律的形式完全匹配
能够精准地捕捉数据的抛物线趋势，同时不会被噪声带偏
- 图 3 过拟合
使用 $y = w_1x^{15} + w_2x^{14} + \dots + w_{15}x + b$
15 次多项式去拟合这些数据点，曲线会精确地穿过每一个训练样本
包括那些受噪声干扰偏离真实规律的点，曲线变得扭曲而怪异

学习曲线

`learning_curve` 会自动用不同数量的训练数据去训练模型，然后分别计算

1. 训练集得分（模型学自己见过的数据有多好）
2. 验证集得分（模型学没见过的新数据有多好）

最后画出两条曲线，一眼看出模型是过拟合、欠拟合还是刚刚好

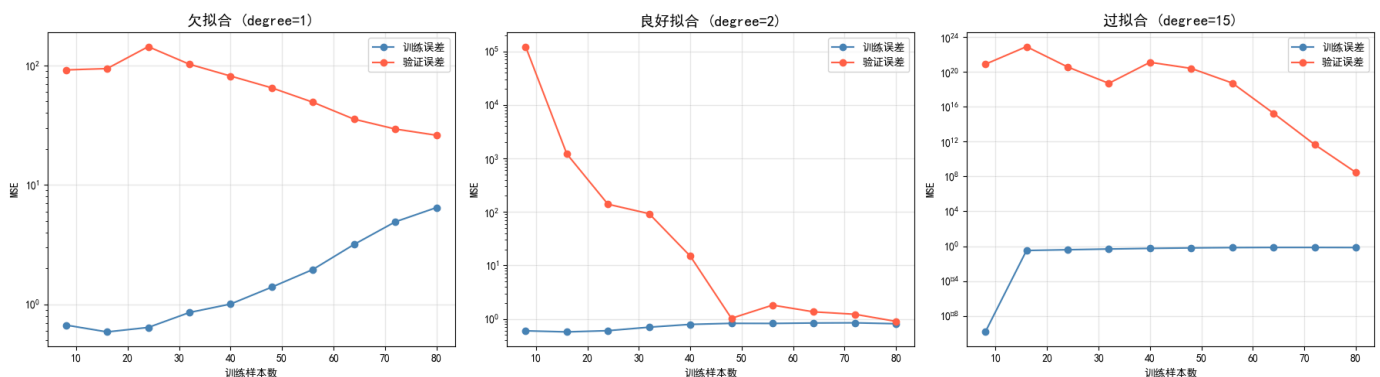
```
# 创建1行3列的子图
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# 遍历每个模型，绘制学习曲线
for ax, (title, model) in zip(axes, models.items()):
    # learning_curve 返回:
    # - train_sizes: 训练集大小数组
    # - train_scores: 各训练集大小下的训练得分 (shape: [n_sizes, n_cv_folds])
    # - val_scores: 各训练集大小下的验证得分
    train_sizes, train_scores, val_scores = learning_curve(
        model, X, y,
        train_sizes=np.linspace(0.1, 1.0, 10), # 10个训练集大小: 10%, 20%, ..., 100%
        cv=5, # 5-Fold 交叉验证
        scoring="neg_mean_squared_error", # 使用负均方误差作为评分指标
    )

    # 将负MSE 转换为正MSE，并对交叉验证结果取平均
    train_mse = -train_scores.mean(axis=1)
    val_mse = -val_scores.mean(axis=1)

    # 绘制训练误差曲线
    ax.plot(train_sizes, train_mse, "o-", color="steelblue", label="训练误差")
    # 绘制验证误差曲线
    ax.plot(train_sizes, val_mse, "o-", color="tomato", label="验证误差")
    ax.set_title(title, fontsize=14)
    ax.set_xlabel("训练样本数")
    ax.set_ylabel("MSE")
    ax.legend()
    ax.set_yscale("log") # 使用对数刻度，便于展示差异较大的误差值
    ax.grid(True, alpha=0.3)

plt.tight_layout() # 自动调整子图间距
plt.show()
```



- **图 1**
验证误差始终和训练误差非常接近，且都维持在高位 (约 20~100)
模型太简单既学不会训练数据，也泛化不好，属于典型的欠拟合
- **图 2**
初始验证误差很高，随样本数增加快速下降，最后和训练误差几乎重合，均维持低位 (5~10)
模型复杂度刚好，既拟合了训练数据，又能很好泛化到新数据
- **图 3**
训练误差几乎为 0，验证误差始终远高于训练误差，且数值巨大 (10^8)
模型太复杂 (15 次多项式)，过度学习了训练数据中的噪声和细节，完全无法泛化，属于典型的过拟合

模式	训练误差	验证误差	两者关系
欠拟合	高	高	都高，且差距不大，加数据也没用
良好拟合	低	低	都低，逐渐收敛
过拟合	很低	高	差距大，加数据可能有帮助

解决欠拟合

欠拟合意味着模型太简单，无法捕捉数据中的真实模式
核心思路是让模型更有表达力

特征工程

- 特征工程
是解决欠拟合最常用、最有效的方法之一
通过构造更有信息量的特征，即使用简单模型也能获得好效果

常见的特征工程方法有

方法	说明	示例
多项式特征	添加特征的高次项和交互项	$x_1, x_2 \rightarrow x_1, x_2, x_1^2, x_2^2, x_1x_2$
特征交叉	组合两个或多个特征	面积 = 长 $\times$ 宽
数学变换	对数、平方根、指数等	$\log(income)$
领域特征	结合业务知识构造	从日期中提取“是否周末”
分箱	将连续值离散化	年龄 $\rightarrow$ 青年/中年/老年

增加训练轮数

如果模型还没有训练到收敛就提前停止了，自然表现不佳
这时候可以

- 检查训练过程中损失是否仍在下降
- 适当增加 epochs 数量，给模型更多的学习时间，直到损失不再下降

增加模型复杂度

从简单模型 (如线性回归)，切换到复杂模型 (如随机森林、决策树)
当特征工程不够时，可以直接使用更复杂的模型。比如在模型评估中演示的加州房价 R^2
可以训练到 0.57 左右，但如果引入多项式、决策树、随机森林等复杂模型会显著提升其 R^2

```
import numpy as np
import pandas as pd
from sklearn.datasets import fetch_california_housing
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
```

```
from sklearn.metrics import r2_score, mean_squared_error

# 加载数据集
housing = fetch_california_housing()
X = pd.DataFrame(housing.data, columns = housing.feature_names)
y = housing.target

# 预处理数据
X_scaled = StandardScaler().fit_transform(X)

# 花粉数据集
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

# 定义模型
models = {
    "线性回归": make_pipeline(StandardScaler(), LinearRegression()),
    "决策树 (max_depth=5)": DecisionTreeRegressor(max_depth=5, random_state=42),
    "随机森林 (100棵树)": RandomForestRegressor(
        n_estimators=100, max_depth=10, random_state=42, n_jobs=-1
    ),
}

# 声明结果列表
results = []

# 遍历模型进行训练并评估
for name, model in models.items():
    model.fit(X_train, y_train)
    train_r2 = r2_score(y_train, model.predict(X_train))
    test_r2 = r2_score(y_test, model.predict(X_test))
    train_mse = mean_squared_error(y_train, model.predict(X_train))
    test_mse = mean_squared_error(y_test, model.predict(X_test))

    results.append({
        "模型": name,
        "训练 R²": train_r2,
        "测试 R²": test_r2,
        "训练 MSE": train_mse,
        "测试 MSE": test_mse,
        "差距 (R²)": train_r2 - test_r2,
    })

df_results = pd.DataFrame(results)
print("\n" + "=" * 80)
print("各模型对比: ")
print("=" * 80)
print(df_results.to_string(index=False, float_format="%.4f"))
```

解决过拟合

过拟合意味着模型太复杂，模型记住了训练数据的噪声

核心思路是让模型别太极端

技术	核心思想	适用场景
增加训练数据	更多数据让噪声被稀释	数据获取成本低时
正则化	给模型复杂度加惩罚	最通用的方法
降低模型复杂度	用更简单的模型	模型明显过于复杂时
特征选择	去掉无关或冗余特征	特征多而样本少时
交叉验证	更可靠地评估泛化能力	所有场景
早停	训练到验证误差开始上升时停下	迭代式模型（神经网络/GBDT）

技术	核心思想	适用场景
Dropout	随机丢弃神经元	深度学习
数据增强	通过变换扩充训练集	以图像/文本/音频为对象的深度学习

正则化

- 正则化 Regularization
本意是使之变得**规则、整齐、正常**
在机器学习中没正则化时，模型会疯狂拟合变得很不正常
使用加正则化，让模型别太极端、别太复杂，回归平滑正常的简单形态

正则化技术会惩罚不相关的特征，让模型自动降低复杂度

在多项式的演示中，用15次多项式去拟合 $y = x^2$

模型为穿过每个带有噪音的训练点，部分系数 w_j 发展为极端值

$$\hat{y} = w_0 + w_1x + w_2x^2 + \dots + w_{15}x^{15}$$

此时为过拟合

- 其中出现 $w_5 = 387.2, w_7 = -1042.8$ 这样的极端系数
- 曲线剧烈震荡

普通线性回归的损失函数

$$J(w) = \frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

问题在于函数仅关注拟合的准确度，完全不顾系数大小

- 正则化的核心思想
在损失函数中额外加一个惩罚项 系数越大惩罚越重，迫使模型尽量用小的系数拟合
- 正则化的两种方式
 - L2 正则化 Ridge
 - L1 正则化 Lasso

L2 正则化 Ridge/岭回归

山脊 Ridge

惩罚所有权重的平方和

$$J_{Ridge}(w) = MSE + \alpha \sum w_j^2$$

将所有权重均匀压缩到接近 0 的小值，但**不会恰好等于 0**

当所有特征都对预测有一定贡献，或者特征之间存在多重共线性时，L2 表现最为稳定

L1 正则化 LASSO

最小绝对值收缩与选择算子 (Least Absolute Shrinkage and Selection Operator, LASSO)

套索 Lasso

惩罚所有权重的**绝对值之和**

$$J_{LASSO}(w) = MSE + \alpha \sum |w_j|$$

将不重要特征的权重直接**压缩到0**，相当于自动进行**特征选择**

当数据集有大量特征，但真正起作用的只是少数几个时，L1 非常有价值

使用正则化解决过拟合

[!CAUTION]

在正则化之前，务必先进行标准化

由于高次项的数值通常极大，若不缩放，则导致系数的惩罚不公平

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline

# 设置中文字体
plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial Unicode MS']
plt.rcParams['axes.unicode_minus'] = False

# =====
# 1. 生成带噪声的  $y = x^2$  数据
# =====
np.random.seed(42)
n_samples = 30
X_train = np.sort(np.random.uniform(-3, 3, n_samples)).reshape(-1, 1)
y_train = X_train.ravel()**2 + np.random.normal(0, 1.5, n_samples)

# 用于绘制平滑曲线的测试点
X_plot = np.linspace(-3.5, 3.5, 200).reshape(-1, 1)

# =====
# 2. 定义模型 (15次多项式)
# =====
degree = 15

# 注意: 使用正则化时, 务必先进行标准化 (StandardScaler)!
# 因为高次项 (如  $x^{15}$ ) 的数值会极其庞大, 如果不缩放, 正则化惩罚会极不公平。

models = {
    "普通线性回归\n(无正则化, 过拟合)": make_pipeline(
        PolynomialFeatures(degree), StandardScaler(), LinearRegression()
    ),
    "Ridge (L2 正则化)\n(所有权重变小)": make_pipeline(
        PolynomialFeatures(degree), StandardScaler(), Ridge(alpha=5.0)
    ),
    "Lasso (L1 正则化)\n(自带特征选择)": make_pipeline(
        PolynomialFeatures(degree), StandardScaler(), Lasso(alpha=0.2, max_iter=10000)
    )
}

# =====
# 3. 训练模型并可视化结果
# =====
fig, axes = plt.subplots(2, 3, figsize=(16, 10))

for i, (name, model) in enumerate(models.items()):
    # 训练模型
    model.fit(X_train, y_train)

    # 获取最后一步 (回归器) 的系数  $w_j$ 
    coefs = model.named_steps[model.steps[-1][0]].coef_

    # 统计绝对值大于极小值的非零系数个数
    non_zero_count = np.sum(np.abs(coefs) > 1e-5)

    # --- 上半部分: 绘制拟合曲线 ---
    ax_curve = axes[0, i]
    ax_curve.scatter(X_train, y_train, color='black', s=30, label='训练数据')
    ax_curve.plot(X_plot, X_plot.ravel()**2, '--', color='gray', label='真实关系  $y=x^2$ ')
    ax_curve.plot(X_plot, model.predict(X_plot), color='red', linewidth=2, label='模型预测')

    ax_curve.set_title(name, fontsize=14)
    ax_curve.set_ylim(-5, 15)
    ax_curve.legend()
    ax_curve.grid(True, alpha=0.3)

    # --- 下半部分: 绘制模型系数 ( $w_0 \sim w_{15}$ ) ---

```

```

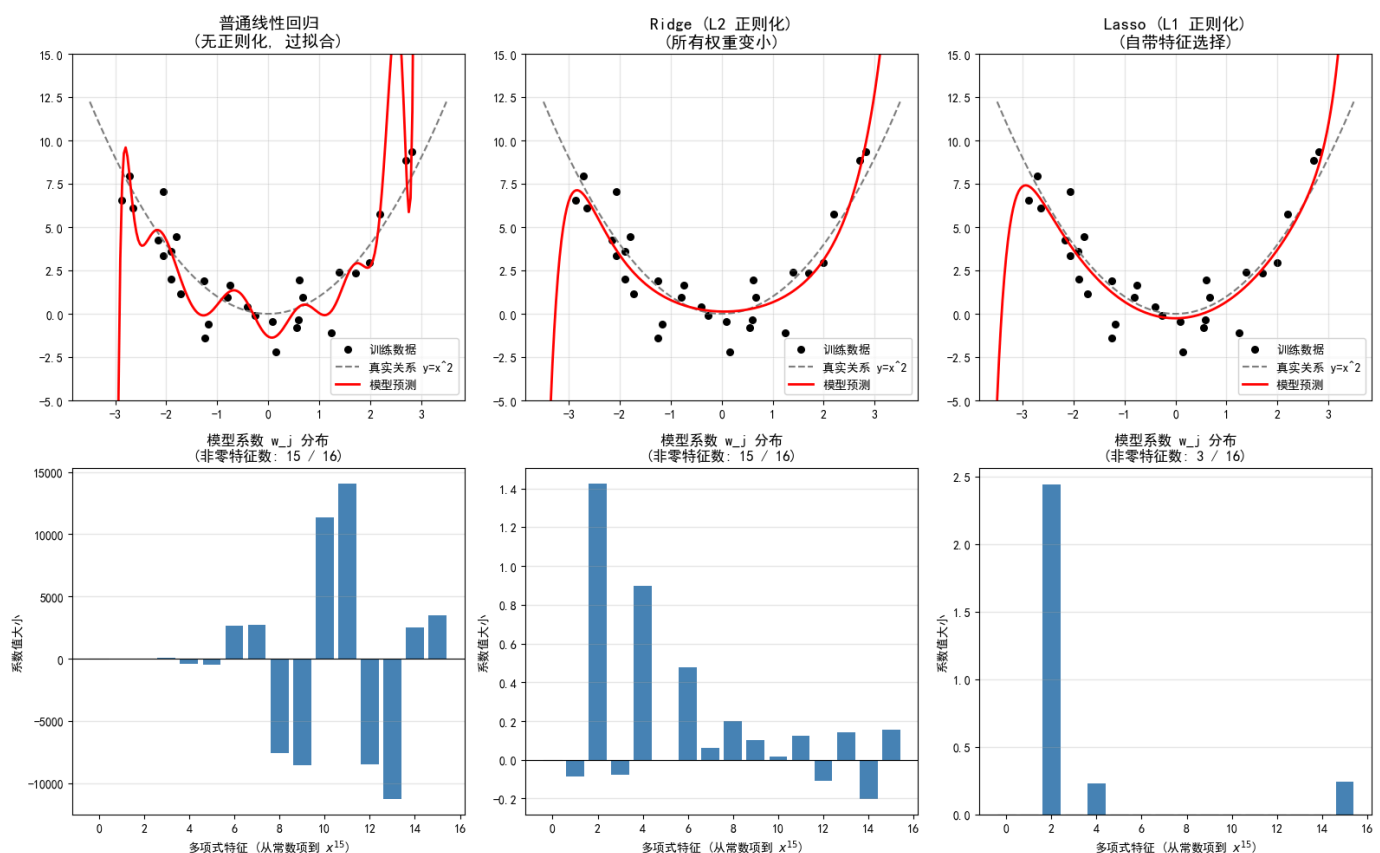
ax_coef = axes[1, i]
# 使用柱状图展示系数大小
bars = ax_coef.bar(range(len(coefs)), coefs, color='steelblue')

# 把等于 0 的系数标为灰色
for bar, coef in zip(bars, coefs):
    if abs(coef) < 1e-5:
        bar.set_color('lightgray')

ax_coef.set_title(f"模型系数 w_j 分布\n(非零特征数: {non_zero_count} / {len(coefs)})", fontsize=12)
ax_coef.set_xlabel("多项式特征 (从常数项到 $x^{15}$)")
ax_coef.set_ylabel("系数值大小")
ax_coef.axhline(0, color='black', linewidth=0.8)
ax_coef.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()

```



1. 拟合曲线表现

- 普通线性回归
曲线像过山车一样剧烈震荡, 试图穿过每一个带有噪声的训练点, 典型过拟合
- Ridge (L2)
曲线变得非常平滑, 不再去迎合个别噪声点, 基本还原了抛物线的形状
- Lasso (L1)
曲线同样变得平滑, 并且形状与 Ridge 略有不同, 非常接近真实的 $y = x^2$

2. 底层系数 w_j 表现

- 普通线性回归
为了制造出剧烈的震荡, 模型把某些系数推到了极端值
- Ridge (L2)
把所有极端的峰值都压扁, 系数大多被限制在 $[-3, 3]$ 之间, 16 个系数全都在起作用
- Lasso (L1)
大部分参数值为 0, 模型最终只保留了少数几个关键特征 (比如 x^2 项), 其他高次项直接被剔除

总结

在机器学习中，我们追求的目标始终是泛化能力——模型在未见过的新数据上依然表现良好。
欠拟合和过拟合是阻碍泛化的两大核心问题：

诊断结果	主要表现	核心问题	解决方案
欠拟合 Underfitting	训练集得分：差 测试集得分：差	模型表达能力不足	+ 做加法： 1. 增加特征（特征工程/多项式特征） 2. 增加训练轮数 3. 换用更复杂的模型
过拟合 Overfitting	训练集得分：极好 测试集得分：差	模型过于敏感、死记硬背	- 做减法： 1. 引入正则化 2. 提供更多的训练数据 3. 简化模型结构（如剪枝、减少特征维度） 4. 提前停止训练（Early Stopping）

到这里，我们已经掌握了线性回归的大部分核心知识。线性回归是整个机器学习乃至深度学习的 **基本原子**

- **逻辑回归** 不过是在线性回归的输出上套了一个激活函数；
- **深度学习** 的本质是将多个线性回归通过非线性函数层层相连；
- 即便是 **大语言模型**，其核心运算依然可以理解为多元线性回归的大规模批量处理。

07. 逻辑回归

本节内容

- 1. 分类问题与线性回归的局限
- 2. 逻辑回归的核心：Sigmoid函数
- 3. 损失函数：交叉熵推导
- 4. 梯度计算与训练
- 5. 案例：乳腺癌预测
- 6. 模型评估与ROC
- 7. 总结：与神经网络的联系

分类问题

线性回归预测**连续数值**，解决**是多少**的问题
逻辑回归预测**离散标签**，解决**是哪类**的问题

- 1. 推断垃圾邮件
- 2. 推断肿瘤良恶
- 3. 推断广告效用
- 分类 Classification
 预测离散标签
 输出有限个数的互斥类别

常见 **二分类**，也可扩展 **多分类**

逻辑回归的作用

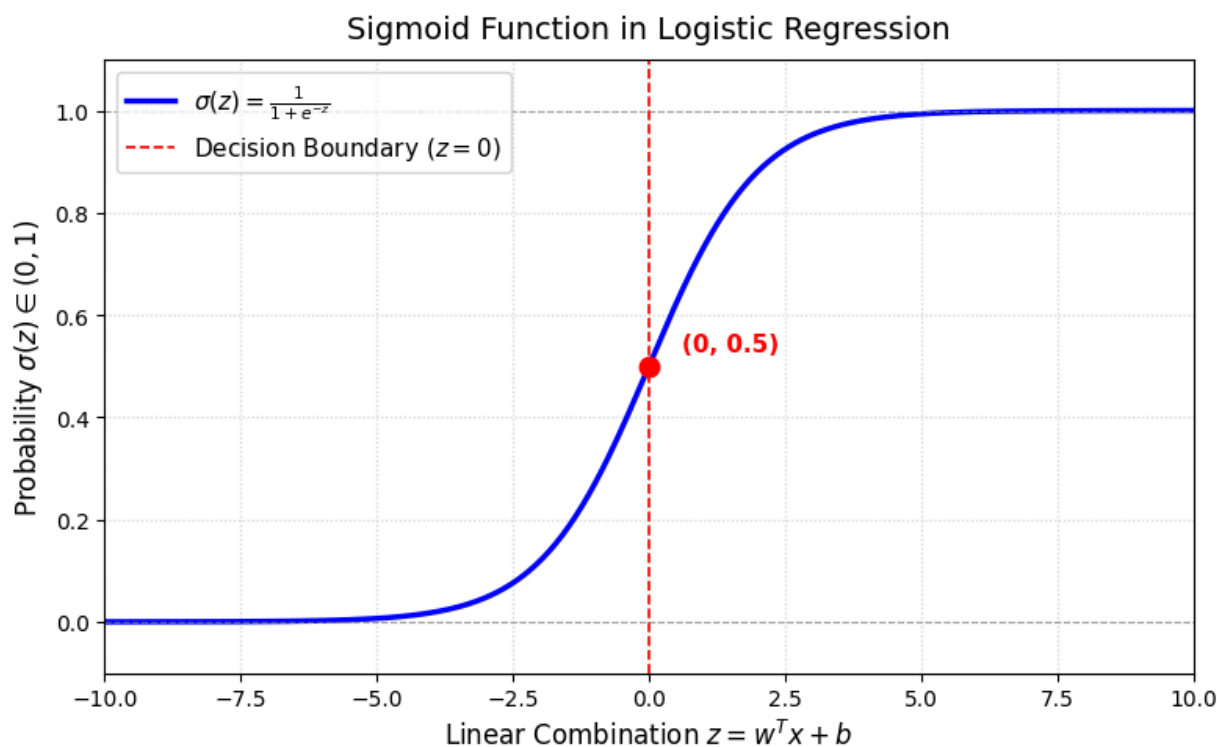
- 规避线性回归处理分类任务的缺陷
 - 1. 输出范围失控 线性回归输出值域 $(-\infty, +\infty)$
 分类问题需要输出的概率在 $(0, 1)$
 - 2. 受异常值影响
 数据集内异常样本的极端值，会严重拉偏整条回归线
 导致原本预测准确的普通样本被错误分类
- 逻辑回归的解决思路是
 为线性回归叠加变形器

Sigmoid函数

Sigmoid 函数

将线性回归的输出映射为 $(0, 1)$ 的概率值

$$z = wx + b$$
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$
$$\hat{y} = \frac{1}{1 + e^{-(wx+b)}}$$

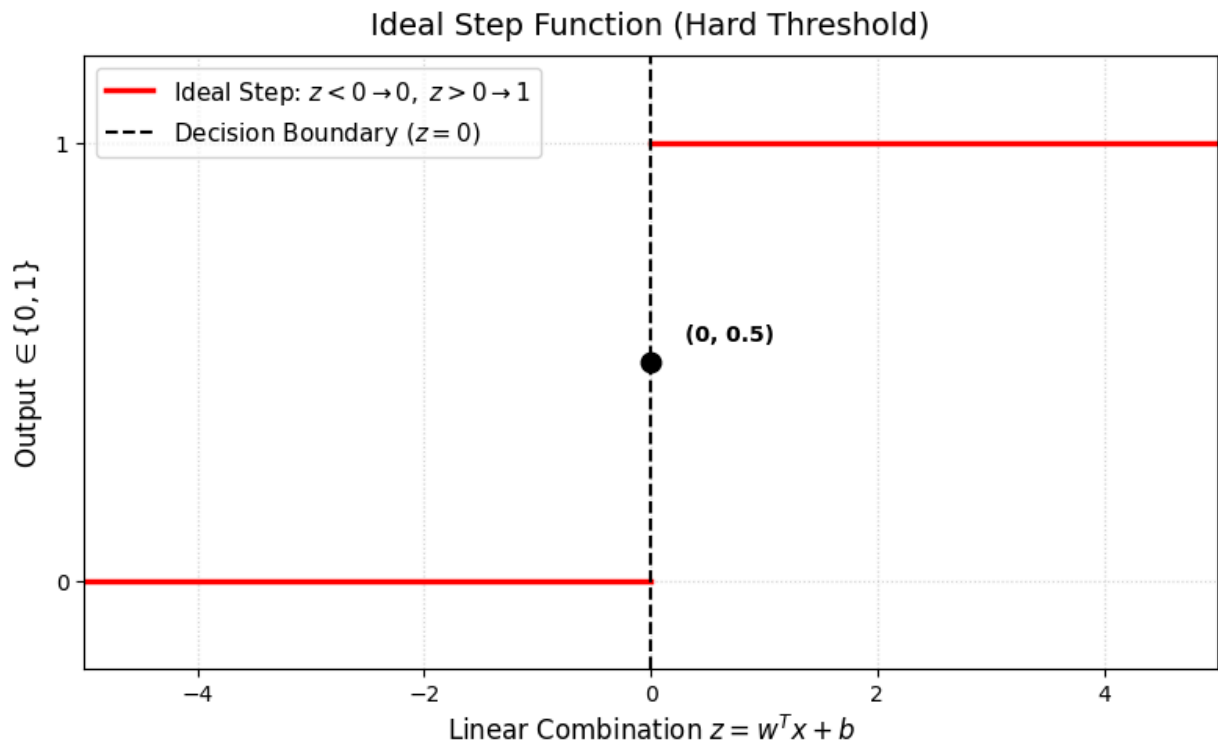


[!TIP]

逻辑回归 = 线性回归 + Sigmoid

Sigmoid 函数的优势

1. 输出天然具有概率含义
值域严格限制在 $(0, 1)$ ，0.8 代表模型有 80% 的把握认作正类
2. 自带决策分界点
 $\sigma(0) = 0.5$
 $w x + b > 0 \rightarrow$ 概率大于 0.5 $\rightarrow$ 判定作为正类
 $w x + b < 0 \rightarrow$ 概率小于 0.5 $\rightarrow$ 判定作为负类
 $w x + b = 0$ 作为决策边界
3. 处处可导 导数极简
阶跃函数在跳变点无法求导，阻碍梯度下降
相比阶跃函数，Sigmoid 连续平滑
导数公式简洁 $\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$ ，极大提升梯度下降效率

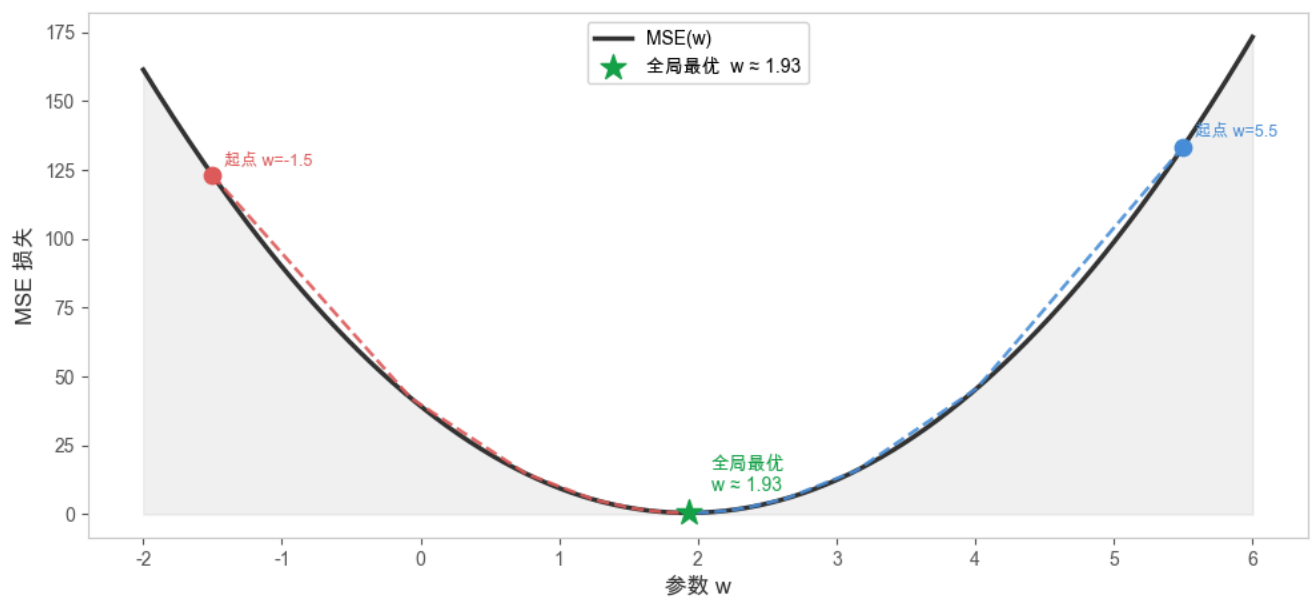


逻辑回归损失函数

为何不用 MSE

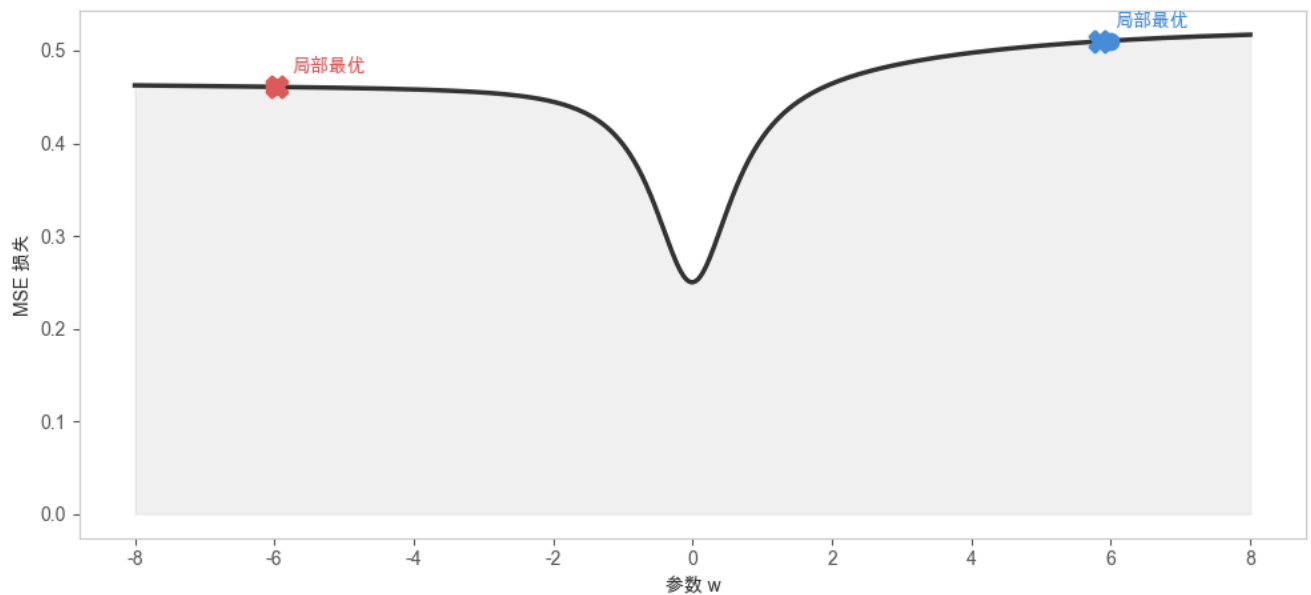
- 线性回归 + MSE = 凸函数
不论初始参数如何，通过梯度下降都能找到全局最优解

MSE 是凸函数（抛物线）：无论从哪里出发，梯度下降都能到达全局最优



- 逻辑回归 + MSE = 非凸函数
引入非线性 Sigmoid 后，损失函数不再单调，会陷入局部最优，且存在梯度消失问题

逻辑回归 + MSE (非凸函数) : 不同起点陷入不同局部最优



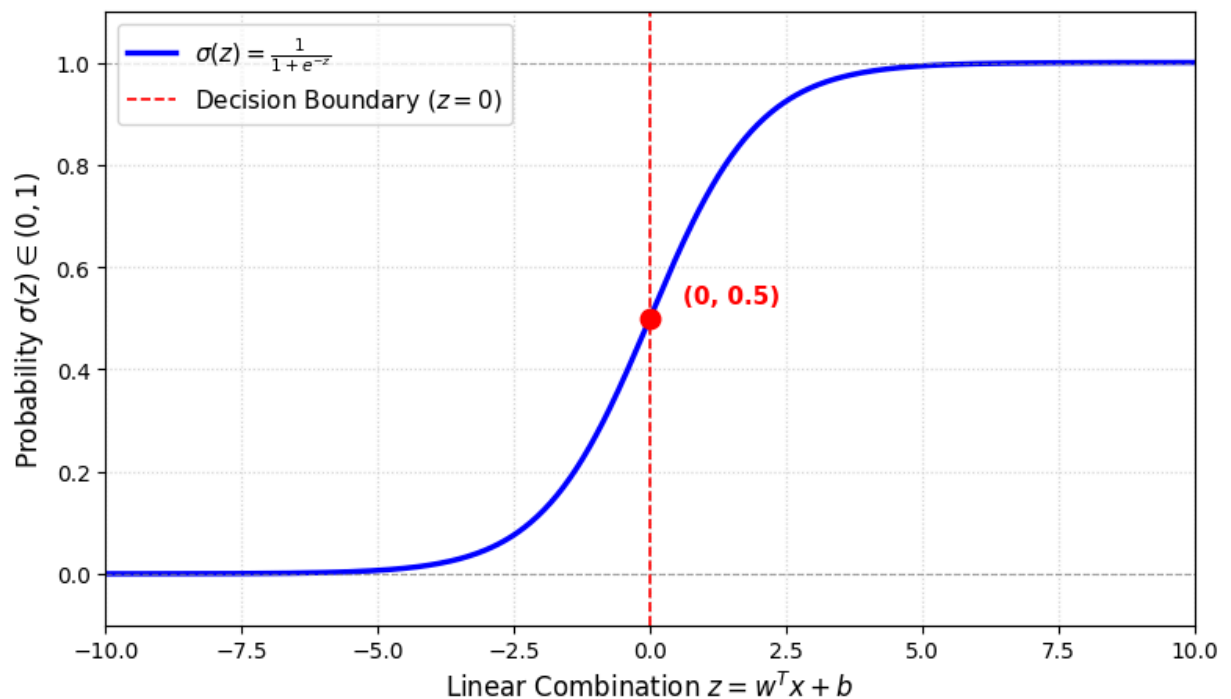
1. 局部最优

红蓝二点分别代表左右出发遇到的局部最小值或平缓区域
梯度下降过程中，会在红蓝二点陷入 **局部最优**

2. 梯度消失

Sigmoid 函数的输入值极大或极小时，输出值会逼近 1 或 0

Sigmoid Function in Logistic Regression



如果预测值接近 1，那么梯度 $\hat{y} \cdot (1 - \hat{y})$ 就会接近 0

此时即使预测错误，梯度也接近于 0，导致参数不会更新，模型停止学习，发生 **梯度消失**

分类问题损失的特征

对于二分类任务，模型输出的是一个概率 p ，这个概率有很明确的含义：

- $p \approx 1$
模型非常相信样本属于正类
- $p \approx 0$
模型非常相信样本属于负类

- $p \approx 0.5$

模型最不确定，给出的结果相当于随机

因此，一个合理的损失函数应该满足：

- 如果真实标签 $y=1$ ，那么 p 越接近 1 越好，越接近 0 惩罚越大
- 如果真实标签 $y=0$ ，那么 p 越接近 0 越好，越接近 1 惩罚越大

这里要特别注意一个常见误区： $p=0.5$ 并不是损失最大，而是模型最不确定。

真正损失最大的时候，是模型非常自信但预测错了

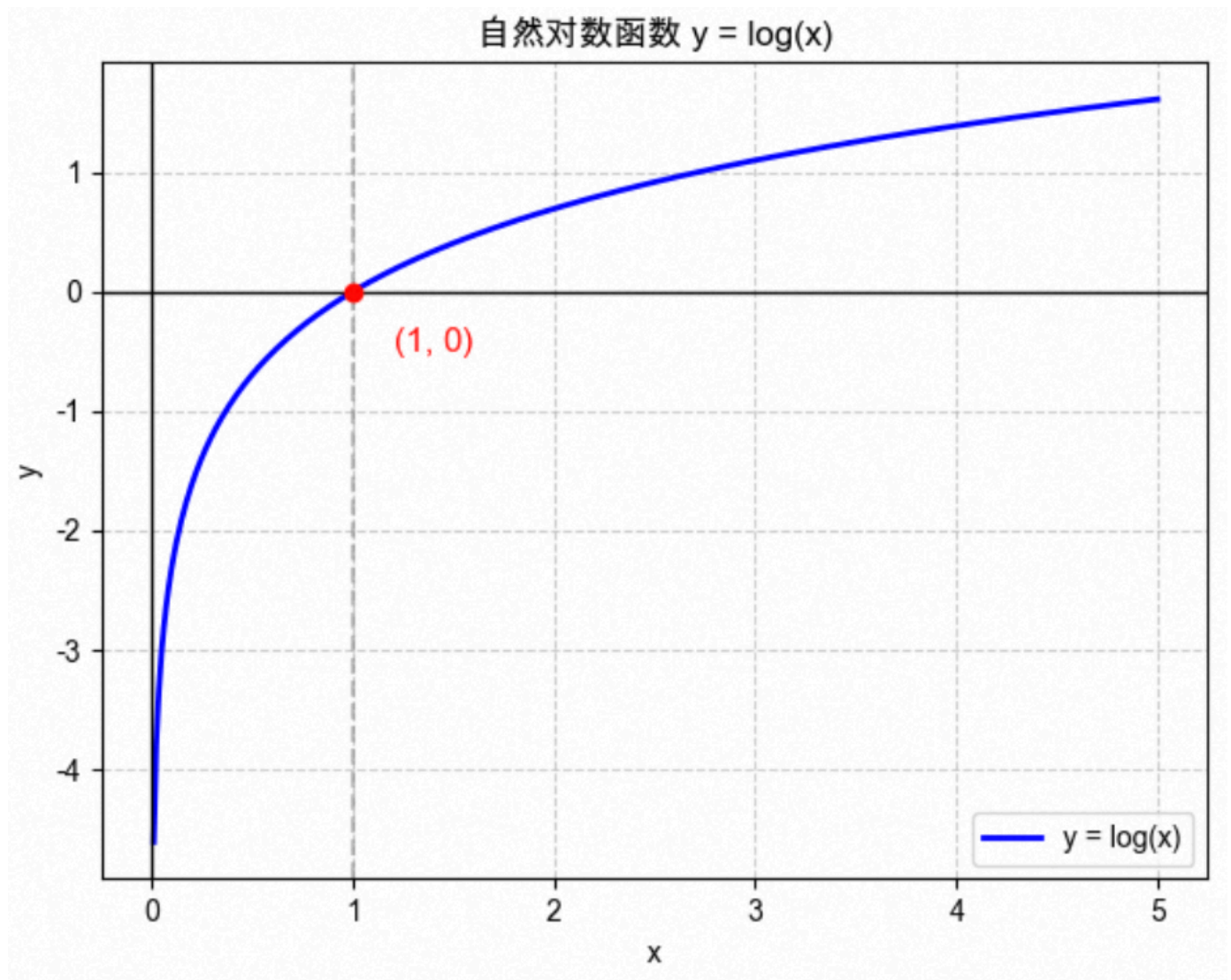
例如

- 真实 $y=1$ ，却预测 $p \approx 0$
- 真实 $y=0$ ，却预测 $p \approx 1$

基础数学

log 函数特征

- 对数函数 Logarithm
是指数函数的逆运算
在机器学习中，常用自然对数
 1. 以自然常数 $e \approx 2.718$ 为底
 2. 通常记作 $\ln(x)$ 或者写作 $\log(x)$



1. 单调性一致

对数函数是单调递增的

最大化某个概率，完全等价于最大化其对数

该特性使优化过程变得简单

2. 防止数值下溢出

多个介于 0 和 1 之间的概率相乘，结果会迅速趋近于 0，导致计算机无法精确表示
取对数后，**乘法变作加法** $\log(a \times b) = \log(a) + \log(b)$

只看 $x \in (0, 1)$ 区间，映射为模型预测的概率 p ，对 y 轴 $\log(p)$ 取负

- 当预测概率 $p \rightarrow 1$ 时， $y = -\log(p) \rightarrow 0$
- 当预测概率 $p \rightarrow 0$ 时， $y = -\log(p) \rightarrow \infty$

正好符合对损失函数的期待：

- 概率越接近真实标签，损失越小
- 概率越偏离真实标签，损失越大
- 如果模型非常自信但是发生错误，损失会被显著放大
因此 $-\log(p)$ 非常适合拿来构造分类损失

伯努利分布

[!TIP]

伯努利分布

描述在单次实验中，某个事件发生或不发生的概率。

便利数学的定义

- 成功
指定的事件发生， $Y = 1$
- 失败
指定的事件发生， $Y = 0$

二分类问题的本质，是只有两种结果的随机试验

计标签为 $y \in \{0, 1\}$ ，把该标签视作服从伯努利分布

假设结果为 1 的概率为 p ，结果为 0 的概率为 $1 - p$

$$\begin{cases} P(y = 1) = p \\ P(y = 0) = 1 - p \end{cases}$$

1. 统一概率公式

方便计算，将两个公式合并成统一的公式

$$P(Y = y) = p^y(1 - p)^{1-y}$$

2. 结合逻辑回归

在逻辑回归中， y 是样本的真实标签， $\hat{y}$ 是模型预测样本属于正类 ($y = 1$) 的概率

$$p = \hat{y} = \frac{1}{1 + e^{-(wx+b)}}$$

3. 代入伯努利分布

将 $\hat{y}$ 代入伯努利分布，得到单个样本预测正确的概率表示为

$$P(y|x) = \hat{y}^y(1 - \hat{y})^{1-y}$$

最大似然估计

- 概率 Probability
是在 **已知模型参数** 的情况下，预测 **未知数据** 出现的可能性
- 似然 Likelihood
是在 **已知观测数据** 的情况下，评估 **未知模型参数** 的合理性
- 最大似然估计 Maximum Likelihood Estimation
既然 **事件已经发生**，那么 **能最大概率导致事件发生** 的参数 θ ，就是最好的参数
- 似然函数
所有样本预测正确的概率乘积
$$L(\theta) = \prod_{i=1}^m \hat{y}_i^{y_i}(1 - \hat{y}_i)^{1-y_i}$$
- 对似然函数取对数
将连乘变为连加，方便求导

$$\log L(\theta) = \sum_{i=1}^m [y_i \log(\hat{y}) + (1 - y_i) \log(1 - \hat{y})]$$

假设抛硬币为正面的概率为 p

1. 观测数据

抛硬币 10 次，正面 8 次，反面 2 次

2. 构建似然函数

$$L(p) = p^8 \cdot (1 - p)^2$$

3. 最大化似然值

寻找 p 使得 $L(p)$ 最大

将 $L(p)$ 对 p 求导发现

当 $p \in (0, \frac{4}{5})$ 时, $L(p)$ 单调递增

当 $p \in (\frac{4}{5}, 1)$ 时, $L(p)$ 单调递减

$$\max L(p) = L(\frac{4}{5}) = \frac{4^8}{5^{10}}$$

$$\begin{aligned} L'(p) &= 8p^7 \cdot (1 - p)^2 - 2p^8(1 - p) \\ &= 10p^9 - 18p^8 + 8p^7 \\ &= 10p^7(p - \frac{4}{5})(p - 1) \end{aligned}$$

$$\begin{cases} L'(p) < 0, & p \in (\frac{4}{5}, 1) \\ L'(p) = 0, & p = \frac{4}{5} \\ L'(p) > 0, & p \in (0, \frac{4}{5}) \end{cases}$$

4. 形成结论

根据最大似然估计, 抛硬币为正面的概率 p 最可能是 0.8

假设有 m 个独立同分布的样本, 模型的预测概率为 $\hat{y}$

那么这 m 个样本同时按照真实标签 $\hat{y}_i$ 发生的概率 (似然函数 L) 是

$$L(\theta) = \prod_{i=1}^m \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i}$$

对两边取对数后, 将连乘变为连加, 方便求导

$$\log L(\theta) = \sum_{i=1}^m [y_i \log(\hat{y}) + (1 - y_i) \log(1 - \hat{y})]$$

交叉熵损失

交叉熵损失 (Cross-Entropy Loss, CEL)

在统计学中对似然函数 **最大化**

在机器学习对损失函数 **最小化**

对似然函数取负后, 再除以样本量 m 求平均, 形成逻辑回归的最终损失函数 **交叉熵损失**

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y_i \log(\hat{y}) + (1 - y_i) \log(1 - \hat{y})]$$

对于任何一个样本, 真实标签 y 要么是 0 要么是 1, 公式中永远只有一项在起作用, 另一项会被自动乘以 0 消掉。

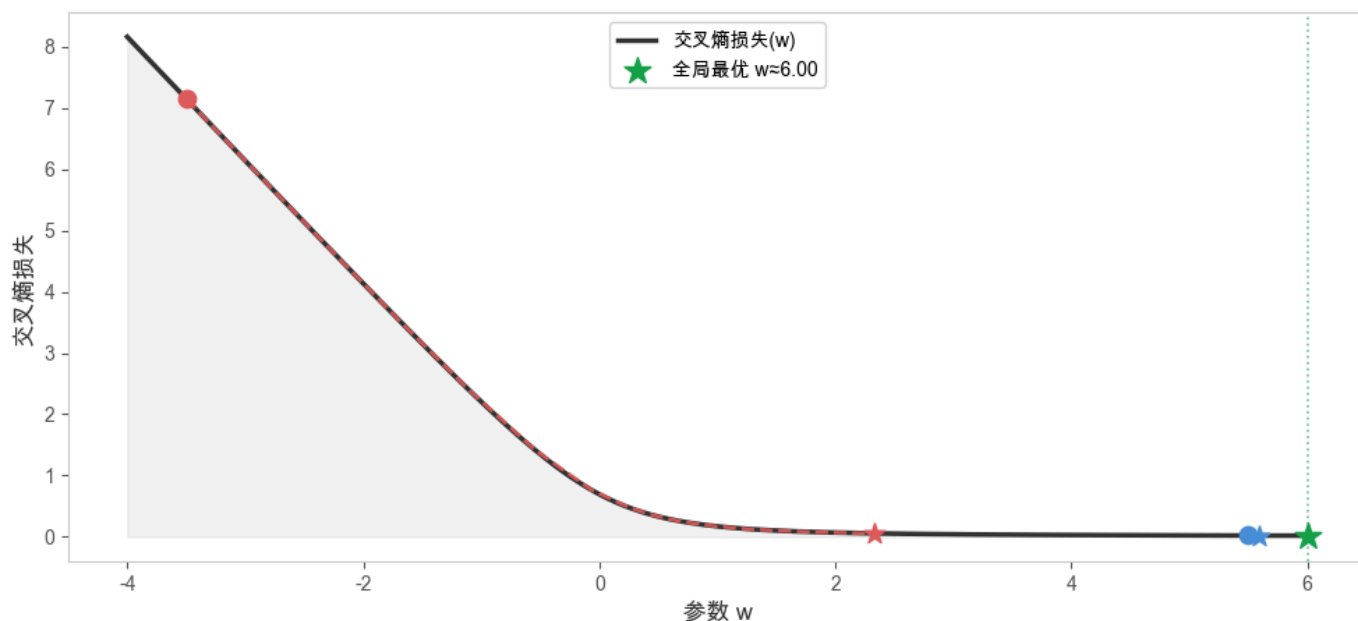
真实标签	损失项	含义
$y = 1$	$-\log(\hat{y})$	预测概率越接近 1, 损失越小; 接近 0 时损失 $\rightarrow +\infty$
$y = 0$	$-\log(1 - \hat{y})$	预测概率越接近 0, 损失越小; 接近 1 时损失 $\rightarrow +\infty$

使用交叉熵作为损失函数时，它是一个严格的凸函数

无论初始权重在哪里，梯度都不会消失，

在远离最优解的地方，梯度反而很大，能够引导模型快速且稳定地收敛到全局最优。

交叉熵损失是凸函数：无论从哪里出发，梯度下降都收敛到全局最优



梯度下降

由于 $\hat{y} = \frac{1}{1 + e^{-(wx+b)}}$ 包含 w, b ，因此不论对权重 w 或偏置 b 求导，过程都绕不开对 $\hat{y}$ 求导

令 $z = wx + b$ ，逻辑回归损失函数 L 对 z 求导

$$\begin{aligned}
 L &= -[y \ln(\hat{y}) + (1 - y) \ln(1 - \hat{y})] \\
 \frac{\partial L}{\partial \hat{y}} &= -\frac{y}{\hat{y}} + \frac{1 - y}{1 - \hat{y}} \\
 \text{令 } z &= wx + b, \quad \hat{y} = \frac{1}{1 + e^{-(wx+b)}} = \frac{1}{1 + e^{-z}} \\
 \frac{\partial \hat{y}}{\partial z} &= \hat{y}(1 - \hat{y}) \\
 \frac{\partial L}{\partial z} &= \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} = \hat{y} - y \\
 \frac{\partial z}{\partial w} &= x, \quad \frac{\partial z}{\partial b} = 1 \\
 \text{代入 } J &= \frac{1}{m} \sum L \\
 \frac{\partial J}{\partial w} &= \frac{1}{m} \sum (\hat{y} - y) \cdot x_j \\
 \frac{\partial J}{\partial b} &= \frac{1}{m} \sum (\hat{y} - y)
 \end{aligned}$$

	线性回归	逻辑回归
权重梯度	$\frac{1}{m} \sum (\hat{y} - y) \cdot x_j$	$\frac{1}{m} \sum (\hat{y} - y) \cdot x_j$
偏置梯度	$\frac{1}{m} \sum (\hat{y} - y)$	$\frac{1}{m} \sum (\hat{y} - y)$
$\hat{y}$ 含义	$w x + b$	$\text{sigmoid}(wx + b) = \frac{1}{1 + e^{-(wx+b)}}$

乳腺癌分类案例

使用 `scikit-learn` 内置的乳腺癌数据集 Breast Cancer Wisconsin

目标:

二分类 {恶性0, 良性1}

加载数据

```
from sklearn.datasets import load_breast_cancer
import pandas as pd

# — 加载数据 —————
data = load_breast_cancer()
X, y = data.data, data.target

print(f'样本数: {X.shape[0]}, 特征数: {X.shape[1]}')
print(f'类别: {data.target_names} → 0={data.target_names[0]}, 1={data.target_names[1]}')
print(f'各类别数量: 恶性={sum(y==0)}, 良性={sum(y==1)}')

# 查看特征
df = pd.DataFrame(X, columns=data.feature_names)
df['label'] = y
df.describe()
```

数据分布

- 样本数量
596
- 特征数量
30
- 类别分布
恶性(0) 212个
良性(1) 357个

特征值的量纲差异很大,需要标准化后再训练

预处理数据与训练模型

1. 划分数据集
 - 80% 训练集 / 20% 测试集
 - `stratify=y` 保证两个子集类别比例一致
2. 预处理特征
 - 用训练集的均值和标准差做标准化
 - 将前者相同的变换应用到测试集
3. 训练模型
 - 交叉熵损失
 - L2正则化

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# — 划分训练集 / 测试集 —————
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# — 特征标准化 (逻辑回归对量纲敏感) —————
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# — 训练逻辑回归模型 —————
model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train_scaled, y_train)
```

```
print('训练完成 ✅')
print(f'训练集准确率: {model.score(X_train_scaled, y_train):.4f}')
print(f'测试集准确率: {model.score(X_test_scaled, y_test):.4f}')
```

- 结果

1. 训练集准确率 0.9890
2. 测试集准确率 0.9825

- 分析

1. 训练集和测试集准确率均在 **97%~98%** 左右, 说明模型拟合效果好且没有明显过拟合
2. 训练集与测试集准确率相差不大, 模型具备良好的**泛化能力**
3. 逻辑回归作为线性模型, 在该数据集上表现出色, 说明乳腺癌特征与标签之间存在较强的线性可分性

预测概率分布可视化

逻辑回归不仅输出类别预测, 还会给出每个样本属于某类别的概率值 $\hat{p} \in [0, 1]$

通过直方图展示两类样本的预测概率分布, 可以直观看出模型的区分能力

```
import numpy as np
import matplotlib.pyplot as plt

# plt.rcParams['font.family'] = 'Arial Unicode MS'
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# — 获取测试集预测概率 —————
proba = model.predict_proba(X_test_scaled)[: , 1] # 良性 (1) 的概率
preds = model.predict(X_test_scaled)

# — 按真实标签分组 —————
proba_malignant = proba[y_test == 0] # 真实恶性样本的预测概率
proba_benign = proba[y_test == 1] # 真实良性样本的预测概率

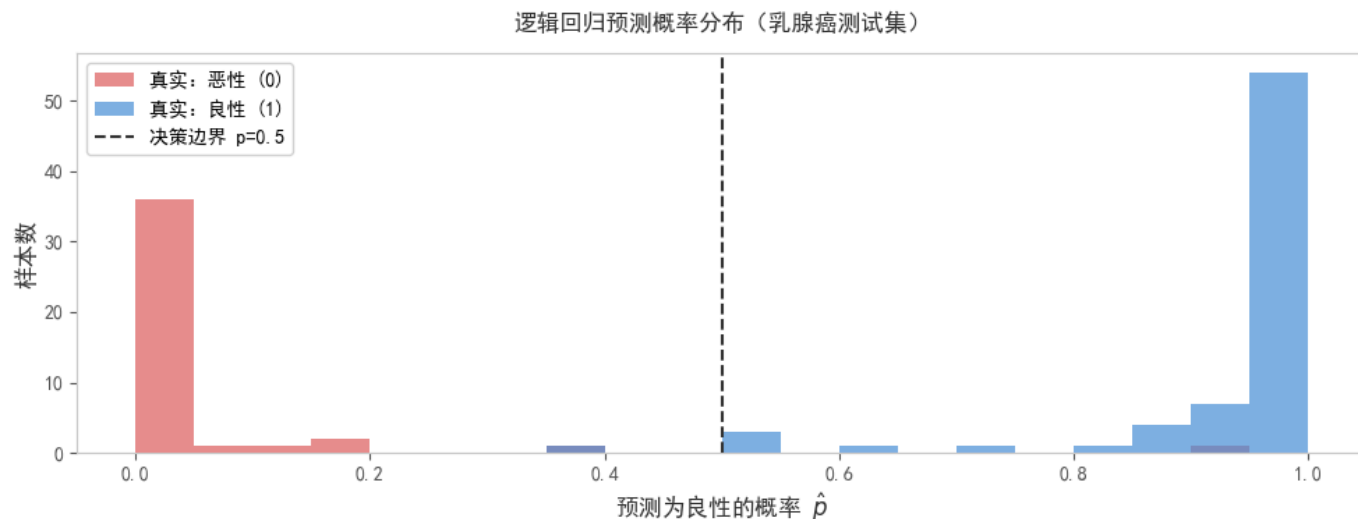
fig, ax = plt.subplots(figsize=(10, 4))
fig.patch.set_facecolor('white')
ax.set_facecolor('white')

ax.hist(proba_malignant, bins=20, color='#e05c5c', alpha=0.7, label='真实: 恶性 (0)', range=(0, 1))
ax.hist(proba_benign, bins=20, color='#4a90d9', alpha=0.7, label='真实: 良性 (1)', range=(0, 1))
ax.axvline(0.5, color='#333333', linewidth=1.5, linestyle='--', label='决策边界 p=0.5')

ax.set_xlabel(r'预测为良性的概率  $\hat{p}$ ', color='#333333', fontsize=12)
ax.set_ylabel('样本数', color='#333333', fontsize=12)
ax.set_title('逻辑回归预测概率分布 (乳腺癌测试集)', color='#333333', fontsize=12, pad=12)
ax.tick_params(colors='#555555')
for spine in ax.spines.values():
    spine.set_edgecolor('#cccccc')
ax.legend(fontsize=10, facecolor='white', edgecolor='#cccccc')

plt.tight_layout()
plt.show()

print('\n 🌸 红色分布集中在 0 附近 (模型认为这些是恶性), 蓝色分布集中在 1 附近 (模型认为是良性)')
print(' 两组分布分离越明显, 说明模型区分能力越强。')
```



- **红色（恶性）**：概率值大多集中在 0 附近，说明模型正确地将大部分恶性样本预测为低良性概率
- **蓝色（良性）**：概率值大多集中在 1 附近，说明模型正确地将大部分良性样本预测为高良性概率
- 两组分布 **几乎不重叠**，中间只有极少数样本落在决策边界 ($p=0.5$) 附近，说明模型区分能力很强
- 少量红色样本出现在右侧（高概率区）、少量蓝色样本出现在左侧（低概率区），这些就是**分类错误的样本**

模型评估指标

线性回归模型的评估指标以连续值的误差衡量预测值与真实值的误差

1. MSE
2. RMSE
3. R^2

分类任务模型的输出是离散的类别标签

- 准确率 Accuracy
预测正确的样本占总样本的比例
- 混淆矩阵 Confusion Matrix
是分类评估的核心工具
- 精确率
- 召回率
- F1
- ROC
- AUC

混淆矩阵

把所有预测结果按 [真是类别 × 预测类别] 排列成一张表格

	预测为正	预测为负
实际为正	TP 真正例 ✓	FN 假负例 ✗ 漏报
实际为负	FP 假正例 ✗ 误报	TN 真负例 ✓

- TP (True Positive)
实际是正类，预测也是正类 ✓
- TN (True Negative)
实际是负类，预测也是负类 ✓
- FP (False Positive)
实际是负类，预测是正类 ✗ (误报)
- FN (False Negative)
实际是正类，预测是负类 ✗ (漏报)

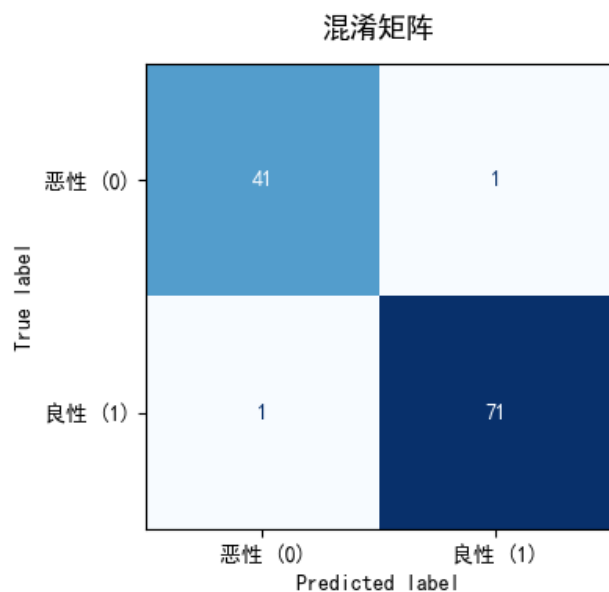
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# plt.rcParams['font.family'] = 'Arial Unicode MS'
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

y_pred = model.predict(X_test_scaled)
cm = confusion_matrix(y_test, y_pred)

fig, ax = plt.subplots(figsize=(5, 4))
fig.patch.set_facecolor('white')
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=['恶性 (0)', '良性 (1)']
)
disp.plot(ax=ax, colorbar=False, cmap='Blues')
ax.set_title('混淆矩阵', fontsize=13, pad=12)
plt.tight_layout()
plt.show()

tn, fp, fn, tp = cm.ravel()
print(f'TP={tp}  FP={fp}  FN={fn}  TN={tn}')
```



结果分析

- 对角线上的数字越大越好，代表预测正确的样本数
- **FN（漏报恶性）** 在医疗场景中危害最大——真实是恶性却被预测为良性，患者可能因此延误治疗
- **FP（误报恶性）** 的代价相对较低，只是增加不必要的复查
- 混淆矩阵直观展示了模型在每个类别上的具体表现，是准确率之外最重要的参考

精确率/召回率/F1

1. 精确率 Precision

预测为正类的样本中，真正是正类的比例
反映预测质量

$$Precision = \frac{TP}{TP + FP}$$

2. 召回率 Recall

所有真正的正类中，被成功预测出来的比例

反映覆盖能力

$$Recall = \frac{TP}{TP + FN}$$

3. F1

精确率和召回率的调和平均，综合衡量两者

Precision 和 Recall 都高，F1 才高

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

- 精确率召回率二者存在权衡
降低决策阈值可以提高召回率（漏报更少），但会降低精确率（误报更多）。根据业务场景决定侧重哪个。

```
from sklearn.metrics import classification_report, accuracy_score

print('准确率 (Accuracy) :', accuracy_score(y_test, y_pred))
print()
print(classification_report(y_test, y_pred, target_names=['恶性 (0)', '良性 (1)']))
```

结果分析

- **macro avg**: 对每个类别单独计算后取平均，不考虑类别样本数量差异
- **weighted avg**: 按各类别样本量加权平均，更能反映整体表现
- 在乳腺癌这类场景中，**恶性 (0) 的召回率**是最重要的指标——宁可误报，不能漏报
- 若召回率不足，可以通过调低决策阈值（默认 0.5）来提升，以牺牲部分精确率为代

ROC 与 AUC

1. ROC

精确率/召回率依赖于决策阈值（默认 0.5），换一个阈值结果就不同。

ROC 曲线通过遍历所有可能的阈值

- 横轴

$$FPR = \frac{FP}{FP + TN}$$

假正率，即负类被误判为正类的比例

- 纵轴

$$TPR = \frac{TP}{TP + FN}$$

真正率，即正类被正确识别的比例

2. AUC (Area Under Curve)

ROC 曲线下方面积，取值 [0, 1]

- AUC = 1.0 完美模型
- AUC = 0.5 随机猜测
- AUC 越大，模型区分正负类的能力越强，且 **与阈值无关**

```
from sklearn.metrics import roc_curve, auc

proba = model.predict_proba(X_test_scaled)[: , 1]
fpr, tpr, thresholds = roc_curve(y_test, proba)
roc_auc = auc(fpr, tpr)

fig, ax = plt.subplots(figsize=(6, 5))
fig.patch.set_facecolor('white')
ax.set_facecolor('white')

ax.plot(fpr, tpr, color='#4a90d9', lw=2, label=f'ROC 曲线 (AUC = {roc_auc:.4f}) ')
ax.fill_between(fpr, tpr, alpha=0.08, color='#4a90d9')
ax.plot([0, 1], [0, 1], color='#999999', lw=1.2, linestyle='--', label='随机猜测 (AUC = 0.5) ')

idx = (np.abs(thresholds - 0.2)).argmin() # 找到最接近0.2的阈值索引
plt.scatter(fpr[idx], tpr[idx], c='yellow', s=50, label=f'阈值=0.2 (TPR={tpr[idx]:.2f}, FPR=
```

```
{fpr[idx]:.2f}}')

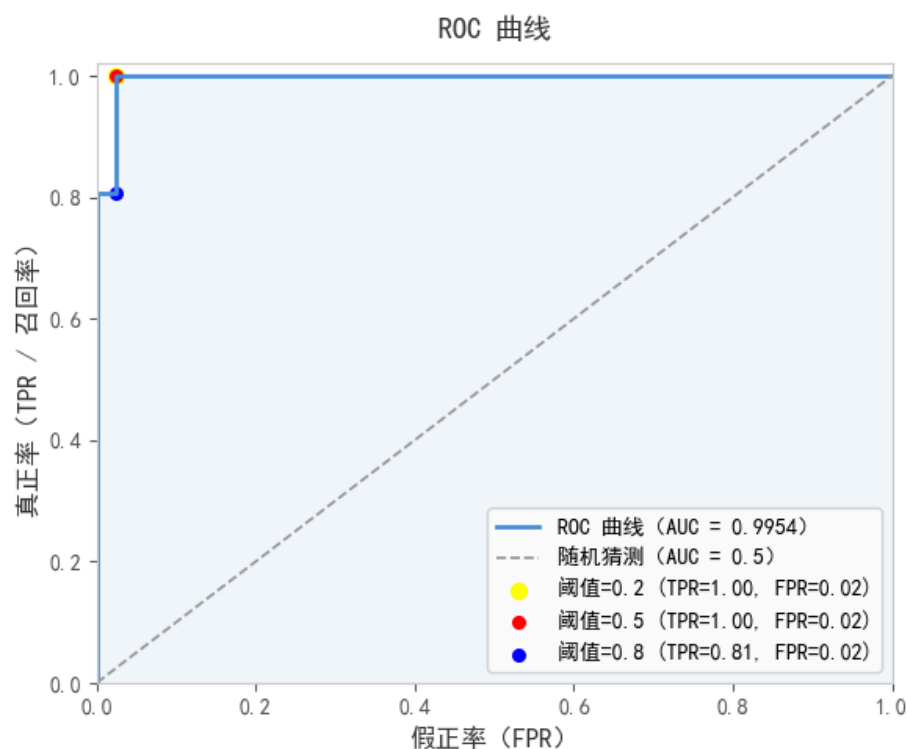
idx = (np.abs(thresholds - 0.5)).argmin() # 找到最接近0.5的阈值索引
plt.scatter(fpr[idx], tpr[idx], c='red', s=30, label=f'阈值=0.5 (TPR={tpr[idx]:.2f}, FPR={fpr[idx]:.2f})')

idx = (np.abs(thresholds - 0.8)).argmin() # 找到最接近0.8的阈值索引
plt.scatter(fpr[idx], tpr[idx], c='blue', s=30, label=f'阈值=0.8 (TPR={tpr[idx]:.2f}, FPR={fpr[idx]:.2f})')

ax.set_xlabel('假正率 (FPR)', color='#333333', fontsize=12)
ax.set_ylabel('真正率 (TPR / 召回率)', color='#333333', fontsize=12)
ax.set_title('ROC 曲线', color='#333333', fontsize=13, pad=12)
ax.tick_params(colors='#555555')
for spine in ax.spines.values():
    spine.set_edgecolor('#cccccc')
ax.legend(fontsize=10, facecolor='white', edgecolor='#cccccc')
ax.set_xlim([0, 1])
ax.set_ylim([0, 1.02])

plt.tight_layout()
plt.show()

print(f'AUC = {roc_auc:.4f}')
```



结果分析

- AUC 接近 **1.0**，ROC 曲线紧贴左上角，说明模型在任意阈值下都能较好地区分恶性和良性
- 曲线越靠近左上角（高 TPR + 低 FPR），模型性能越好；对角虚线代表随机猜测的基准线
- AUC 与具体阈值无关，是比单一准确率更稳健的综合评价指标，适合用于**模型之间的横向对比**

评估指标总结

指标	公式	适用场景
准确率	$(TP + TN)/n$	类别均衡时的快速评估
精确率	$TP/(TP + FP)$	误报代价高（如垃圾邮件过滤）
召回率	$TP/(TP + FN)$	漏报代价高（如疾病检测）
F1	精确率与召回率的调和平均	两者都重要时的综合指标

指标	公式	适用场景
AUC	ROC 曲线下面积	与阈值无关的模型整体能力评估

多分类问题

逻辑回归天生是二分类器，但可以扩展到多分类。

从二分类扩展到多分类时，需要 Softmax 回归 (也叫多项逻辑回归，Multinomial Logistic Regression)

Softmax 回归能一次性输出所有类别的概率。

[!TIP]

Softmax 回归

每个分类都有线性打分模型计算**独立得分**

通过 Softmax 函数，所有得分转化为**概率分布**

所有类别的**概率总和严格为1**

多分类逻辑回归中，每个类别都有一个线性打分函数，模型先为每个类别计算分数，再通过 Softmax 把这些分数转成一组概率，最后选择概率最大的类别作为预测结果。

$$Softmax(Z_i) = \frac{e^{Z_i}}{\sum_j e^{Z_j}}$$
$$P(y = k) = \frac{e^{Z_k}}{\sum_{j=1}^K e^{Z_j}}$$

如果只有 2 个类别，Softmax 就退化成了 Sigmoid，所以 Sigmoid 是 Softmax 的特例。

通向深度学习

逻辑回归不仅是传统的机器学习算法，还与深度学习有深远的联系

逻辑回归可视为一个没有隐藏层的单层神经网络

- 输入层
特征 $x_1, x_2, \dots$
- 权重与偏置
神经网络的连接权重 w 和偏置 b
- 激活函数
Sigmoid / Softmax
- 输出层
逻辑回归 + Softmax 是现代神经网络常用的输出层结构

总结

维度	线性回归	逻辑回归
解决什么问题	回归（预测数值）	分类（预测类别）
输出	任意实数	0~1 的概率
模型	$\hat{y} = wx + b$	$\hat{y} = Sigmoid(wx + b)$
损失函数	MSE	交叉熵
梯度公式	$(\hat{y} - y) \cdot x$	$(\hat{y} - y) \cdot x$
评估指标	MSE / RMSE / R ²	Accuracy / F1 / AUC
正则化	Ridge(L2) / Lasso(L1)	同样适用，参数叫 C=1/λ

- 逻辑回归本质上是在线性回归的基础上加了一个 **Sigmoid**，把输出压缩到 0~1
- 损失函数必须换成交叉熵，否则优化会出问题
- 梯度公式和线性回归形式完全一样，只是 $\hat{y}$ 的计算方式不同
- 分类问题不能只看准确率，要根据业务场景选择 Precision / Recall / F1
- 逻辑回归虽然简单，但**可解释性强、训练快、不容易过拟合**，工业界依然大量使用

08. 决策树

本节内容

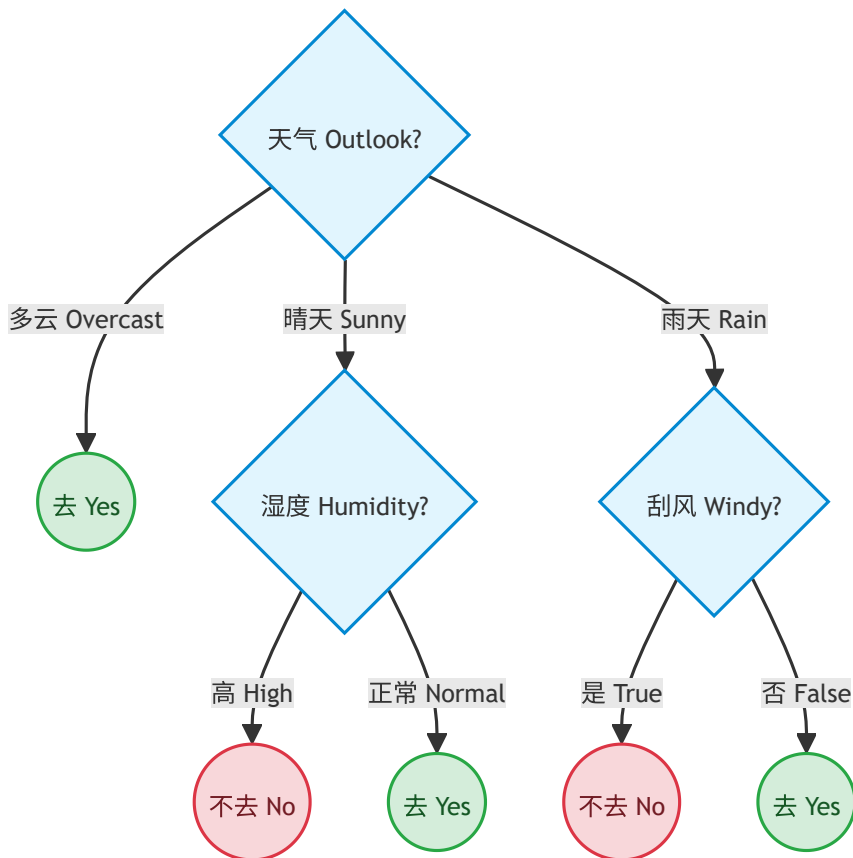
- 1. 直观分类算法
- 2. 决策树结构与术语
- 3. 构建决策树：纯度概念
- 4. 分裂准则：熵与信息增益
- 5. 基尼系数与CART算法
- 6. 递归构建决策树过程
- 7. 鸢尾花分类案例
- 8. 过拟合与剪枝策略
- 9. 决策树优缺点总结
- 10. 集成学习

天气决策打球

- 特征
 - 1. 天气 晴天/多云/下雨
 - 2. 温度 炎热/温和/凉爽
 - 3. 湿度 高度/正常
 - 4. 刮风 是/否

Outlook	Temperature	Humidity	Windy	Play
Sunny	Hot	High	False	No
Sunny	Mild	High	False	No
Overcast	Hot	High	True	Yes
Overcast	Mild	Normal	False	Yes
Rain	Mild	High	False	Yes
Rain	Cool	Normal	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Mild	High	True	No
Rain	Mild	High	True	No
Overcast	Cool	Normal	True	Yes

决策树的核心思想
通过一系列“如果...那么...”的判断，逐步得出结论的过程



决策树

- 决策树 Decision Tree
 - 是一种模仿人类决策时的层层提问方式的机器学习算法
 - 将数据按照不同特征进行递归分裂，最终形成倒立的树状决策结构
 - 1. 内部节点
 - 代表对某个特征的判断
 - 2. 树枝分支
 - 代表判断后的不同大难
 - 3. 叶节点
 - 代表最终的决策结果
- 是经典监督学习算法，既可以解决分类问题，也可以用于回归任务
- 决策路径透明，是机器学习最具可解释性的模型之一

构建决策树

节点纯度

- 纯度 Purity
 - 构建过程的关键问题是如何从众多特征中挑选进行分裂的特性

构建决策树的终极目标，就是每一次特征分类后，都能让分出来的子集纯度尽可能地高，直到所有的叶节点都变成 100% 纯净树就建好了。

- 按“温度”分组，纯度低
 - 在 Hot/Mild/Cool 各组中，有是有否结果混杂

Temp	Play
Hot	No, Yes
Mild	Yes, Yes, No, No, Yes
Cool	Yes, Yes, Yes

- 按“天气”分组，纯度高
 - 发现多云分组所有标签都是 Yes，这个节点的纯度极高。

Outlook	Play
Sunny	No, No, Yes, No
Overcast	Yes, Yes, Yes
Rain	Yes, Yes, No

纯度度量

熵 Entropy

概念来自物理和信息论，描述系统的混乱程度
在决策树中，用来衡量一个节点中各类别分布的混乱程度

- 如果一个节点里一半去打球、一半不去（50% vs 50%），悬念最大，熵值最高，最混乱。
- 如果一个节点里全部都去打球或全部都不去（100% vs 0%），毫无悬念，熵值为 0，最纯净。
熵的值越高，说明这个节点越不纯，我们就越希望通过特征分裂来降低它的混乱程度。

信息增益 Information Gain

仅仅知道一个节点的混乱程度还不够，我们更关心用某个特征分裂之后，混乱程度下降了多少。

信息增益的计算公式

信息增益 = 分裂前的熵 – 分裂后各子节点的加权平均熵

简单来说，就是用这个特征提问后，我们消除了多少不确定性。
信息增益越大，说明这个特征区分能力越强。
在小张打网球的例子中，天气通常会获得最高的信息增益，因此被选为根节点。

基尼系数 Gini

基尼系数
从当前节点中随机抽取两个样本，它们属于不同类别的概率

亦称**基尼不纯度 (Gini Impurity)**，和熵的物理意义类似，都能衡量数据的混乱程度
但对熵进行计算需要对数运算，对算力硬件压力大，
基尼系数计算只需乘法和加法，业界倾向使用基尼不纯度计算

- 基尼系数越小，说明节点越纯（系数为0时完全纯净），大多数样本属于同一类别；
- 基尼系数越大，说明节点越乱，各类别分布越均匀。

$$Gini(D) = 1 - \sum_{k=1}^K p_k^2$$
$$Gini(D) = \sum_{k=1}^K p_k(1 - p_k)$$

- D : 当前节点的数据集
- K : 类别总数
- p_k : 第 k 类样本在 D 中的概率

在 `scikit-learn` 等主流库中，`CART`算法决策树 `DecisionTreeClassifier` 默认使用基尼系数

决策树的生长

构建决策树的过程就变成了一个递归的流程，具体步骤如下

1. 计算指标
- 计算当前节点使用各项特征进行划分后子节点的基尼系数；
2. 选择最优
- 选择划分后基尼系数最小（即数据纯度提升最大）的特征，作为当前节点的分裂特征；
3. 分裂数据
- 按照该特征的不同取值，将当前数据分裂成若干子节点；
4. 递归生长
- 对每个子节点重复上述过程，让决策树继续向下生长；

5. 生成叶节点

当满足停止条件时停止分裂，将当前节点设为叶节点，并确定最终的预测结果。

如此反复递归，直到所有分支都变为纯净的叶节点，或者已经没有可用的特征可以继续分裂，一棵完整的决策树就构建完成了。

鸢尾花案例

加载数据及翻译

使用 `scikit-learn` 自带的鸢尾花(Iris) 数据集

- 样本150个
- 特征4个
- 标签3个

```
import pandas as pd
from sklearn.datasets import load_iris

# 1. 加载数据集
iris = load_iris()

# 2. 为了方便阅读，我们将英文特征名和类别名翻译成中文
feature_names_cn = ['花萼长度(cm)', '花萼宽度(cm)', '花瓣长度(cm)', '花瓣宽度(cm)']
target_names_cn = ['山鸢尾', '变色鸢尾', '维吉尼亚鸢尾']

# 3. 转换成表格(DataFrame)形式展示
df = pd.DataFrame(iris.data, columns=feature_names_cn)
df['最终分类结果'] = [target_names_cn[i] for i in iris.target]

print("===== 1. 数据集前5行预览 =====")
print(df.head())

print("\n===== 2. 数据集包含的分类及数量 =====")
print(df['最终分类结果'].value_counts())
```

基尼建树绘图

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn import tree
import matplotlib.pyplot as plt

# --- 中文支持 ---
# plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial Unicode MS', 'Songti SC']
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

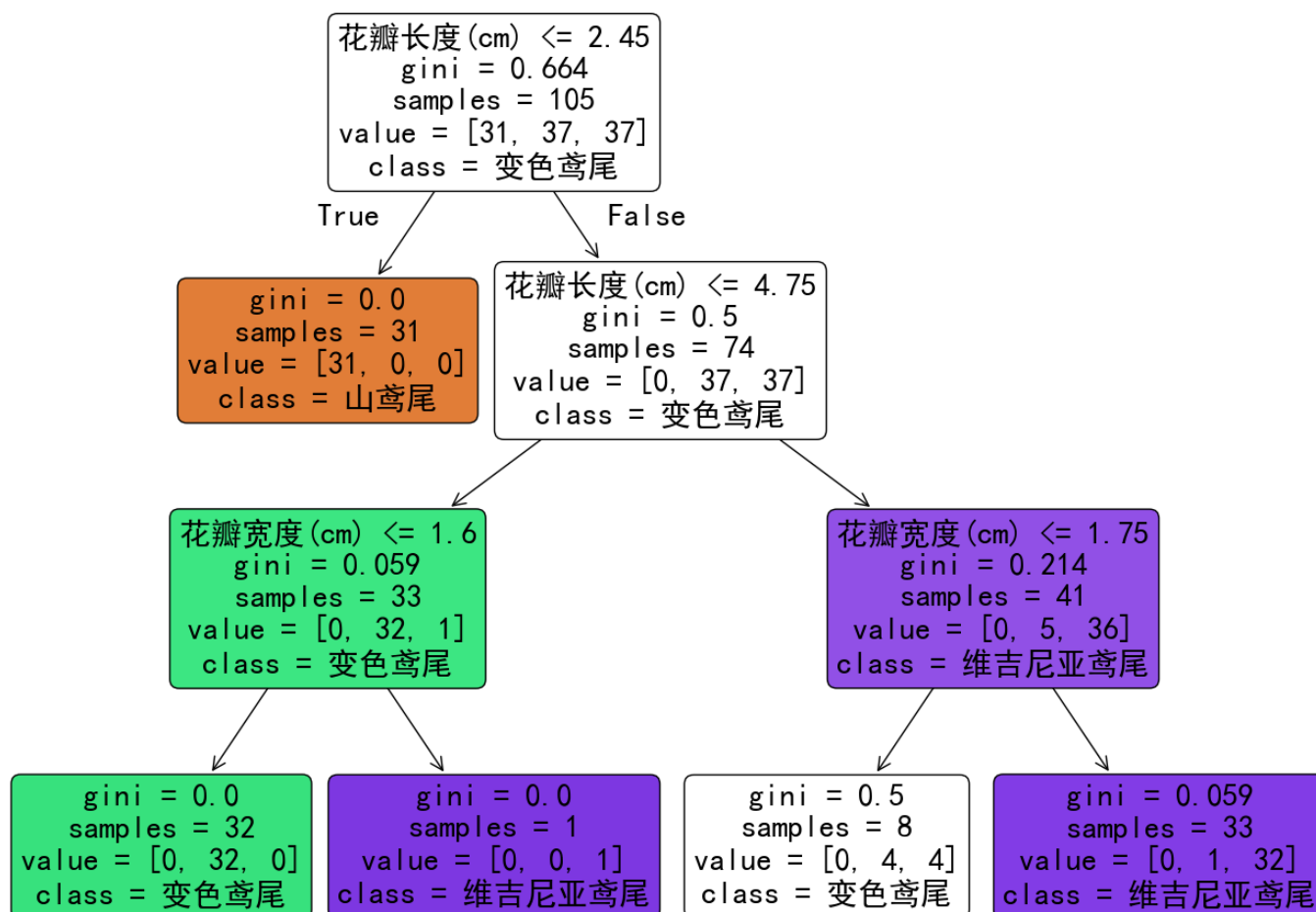
# ===== 1. 划分数据 =====
X = iris.data      # 4个特征
y = iris.target    # 答案标签
# 留出30%的数据作为测试集，70%用来训练
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# ===== 2. 构建与训练模型 =====
# criterion='gini' : 使用基尼系数
# max_depth=3      : 树最多往下长3层（停止条件）
clf = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=42)

# 开始建树（算基尼系数 -> 选特征 -> 分裂 -> 递归）
clf.fit(X_train, y_train)

# ===== 3. 预测成绩 =====
y_pred = clf.predict(X_test)
print(f"决策树模型的预测准确率为: {accuracy_score(y_test, y_pred) * 100:.2f}%\n")
```

```
# ===== 4. 画出这棵树 =====  
plt.figure(figsize=(16, 12))  
tree.plot_tree(clf,  
               feature_names=feature_names_cn,  
               class_names=target_names_cn,  
               filled=True, # 涂色: 基尼系数越小, 颜色越深  
               rounded=True)  
  
plt.show()
```



节点信息解读

- **samples**: 该节点包含的总样本数
- **value**: 各类别的样本分布
- **gini**: 基尼不纯度, 衡量该节点的混乱程度
- **class**: 该节点的默认类别, 这个节点里数量最多的类别
- 节点的着色越深, 代表分类越准确

可视化模型通过 **花瓣长度** 和 **花瓣宽度** 逐步区分三种鸢尾花

鸢尾花类别	关键阈值	节点纯度
山鸢尾	花瓣长度 ≤ 2.45	完全纯净 (gini = 0)
变色鸢尾	$2.45 < \text{花瓣长度} \leq 4.75$ and 宽度 < 1.6	完全纯净 (gini = 0)
	花瓣长度 > 4.75 and $1.6 < \text{宽度} \leq 1.75$	混合较多 (gini = 0.5)
维吉尼亚鸢尾	$2.45 < \text{花瓣长度} \leq 4.75$ and 宽度 > 1.6	完全纯净 (gini = 0)
	花瓣长度 > 4.75 and 宽度 > 1.75	整体纯净 (gini ≈ 0.06 , 含少量杂质)

数字化得到结论: **花瓣长度和宽度决定了种类**

- 花瓣越短 → 越可能是山鸢尾
- 花瓣中等 → 多为变色鸢尾
- 花瓣最长 → 多为维吉尼亚鸢尾
- 花瓣长度 >4.75 且 $1.6 < \text{宽度} < 1.75$ 时变色鸢尾与维吉尼亚有重叠

剪枝防止过拟合

任由决策树生长会为异常值建立分支, 最终导致过拟合

因此需要剪除多余分支, 保留核心主干

剪枝 Pruning

1. 预剪枝 Pre-pruning

提前设定规则, 决策树在生长中触发规则时, 强制停止分裂

- 限制最大深度

`max_depth = 3` 当树长至第 3 层时必须停下, 即使基尼系数还未降至 0

- 限制叶节点最少样本数

如果至子节点只剩 2 个样本, 那么继续分支将失去普遍代表性, 不如此时停止

2. 后剪枝 Post-pruning

先让树毫无保留地生长到最大, 让所有叶节点都绝对纯净, 然后再从下往上检查。

如果剪掉某个底部分支并不会导致准确率明显下降, 甚至能提高在测试集上的表现, 那就应该把它剪掉。

在实际工程中, 预剪枝因实现简单、计算量小而被广泛使用;

而后剪枝理论上效果更好, 但在大数据场景下计算成本较高。

决策树优劣

• 优点

1. 直观易解释, 可视化效果好
2. 能处理非线性关系, 既能回归也能分类
3. 对量纲不敏感, 预处理要求低
自主挑选分裂特征, 自主计算分裂特征的阈值

• 缺点

1. 单棵决策树的稳定性差
2. 对噪声敏感, 容易过拟合

集成学习

集成学习 Ensemble Learning

由多棵决策树构成的森林, 能弥补单棵树稳定性差的缺点

以决策树为基础, 繁衍出工业界和各类算法竞赛中最主流的顶级算法:

• 随机森林 Random Forest

构建很多相互独立的决策树。

每次建树时, 只随机抽取部分数据和部分特征。

面对未知数据时, 让所有的树独立预测, 然后投票表决, 少数服从多数

极大地降低了单棵树由于过拟合带来的误判风险

• 梯度提升树家族

由多棵决策树组合。

但树与树之间不再是独立的, 而是按序完成投票。

若第一棵树的预测产生误差, 则第二棵树专门针对第一棵树的错误进行纠正

通过这种迭代, 不断提升模型的精度。

1. GBDT
2. XGBoost
3. LightGBM

09. 随机森林

本节内容

1. 决策树的优劣
2. 随机森林原理
3. 双随机性详解
4. OOB样本价值
5. 构建步骤
6. 代码实现
7. 特征重要性分析
8. 总结

决策树的优劣

- 优势
极佳的可解释性
- 劣势
天然的脆弱性，对噪音敏感容易过拟合

Leo Breiman 于 2001 年提出**随机森林**，以处理决策树不稳定的风险

集成学习

- **集成学习** Ensemble Learning
将多个弱学习器(分类器)组合成一个强学习器(分类器)，以取得较单一模型更好的泛化性能
三个臭皮匠顶个诸葛亮

随机森林属于集成学习

- 专家会诊模式
不再依赖单棵树进行决策，而是通过**随机采样**建立多棵不同的决策树
预测时让所有树独立判断，最终汇总结果 (分类投票 / 回归求平均值)
有效中和单棵树的偏见与错误，使得模型**既准确又稳健**

随机森林

- 森林 Forest
由多棵决策树组成
决策树是基础模型，通过系列的 "if-then-" 规则对数据逐层切分
- 随机 Random
算法的灵魂
为了森林的多样性，引入**双重随机性**，对数据和特征随机抽样

1. 样本随机 Bootstrap
采用**有放回抽样** **resample**
2. 特征随机 Random Subspace
决策树在每个节点进行分裂时，并不考虑全部特征
而是随机抽取其中一个子集 (通常取 $\sqrt{d}$ 个特征， d 为总特征数)，
仅在这些候选特征中寻找最优分裂点

如果对特征的抽取过程不随机，那么所有树都会优先选择最强的特征作为根节点，导致树的结构高度相似
加入特征随机，部分树被迫挑选其他特征进行分裂，形成不同视角

个体不必完美，多样性构建稳定

袋外估计

袋外样本 (Out of Bag, OOB) 可以提供免费的验证集

每次 Bootstrap 采样，某个样本在 N 次抽取中始终未被选中的概率为

$$P(\text{未被选中}) = (1 - \frac{1}{N})^N \xrightarrow{N \rightarrow \infty} \frac{1}{e} \approx 36.8\text{pct}$$

每棵树约有 1/3 的样本未参与训练，称为 **袋外样本** (Out of Bag, OOB)

零浪费机制，训练与验证的无缝融合

天然的验证集，省去额外划分验证集和交叉验证的繁琐
所有数据都能直接投入训练并同步支持超参数调优，在小数据集场景下尤为珍贵

构建步骤

1. 构建专家团

设定森林中树的数量 `n_estimators`，常为100, ,500

2. 逐棵生成决策树

对于第 t 棵树 ($t = 1, 2, \dots, T$)

1. 通过 Bootstrap 有放回抽取 N 个样本作为训练集
2. 在每个节点分裂时，随机选取 m 个候选特征，在其中找到最佳切分点
3. 让树充分生长，通常不剪枝，或设置较大的最大深度 `max_depth`

3. 集成输出

所有树的训练完成后，对新样本 x 进行预测

- 分类任务采用 **多数表决** Majority Voting

$$\hat{y} = \arg \max_c \sum_{t=1}^T \mathbb{I}[h_t(x) = c]$$

$\hat{y}$: 每棵树输出一个标签

$\arg \max_c$: 所有可能的类别中，使后续表达式最大化的类别 c

$\sum_{t=1}^T$: 从 $t = 1$ 到 T 求和

$h_t(x)$: 第 t 个基础学习器(分类器) 对输入 x 的预测输出

$\mathbb{I}[\text{condition}]$: 指示函数 Indicator Function

- 若condition为真则 $\mathbb{I}[\text{condition}] = 1$
- 若condition为假则 $\mathbb{I}[\text{condition}] = 0$

- 回归任务采用 **简单平均** Averaging

$$\hat{y} = \frac{1}{T} \sum_{t=1}^T h_t(x)$$

$\hat{y}$: 每棵树输出一个数值

代码实现

使用 100 棵决策树组成随机森林，对鸢尾花数据集做训练、验证

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# 加载数据
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# 构建随机森林
rf = RandomForestClassifier(
    n_estimators=100,      # 树的数量
    max_features='sqrt',  # 每次分裂时随机选取的特征数
    oob_score=True,       # 启用 OOB 估计
    random_state=42,
    n_jobs=-1             # 并行训练
)

rf.fit(X_train, y_train)

# 评估
print(f"OOB 准确率: {rf.oob_score_: .4f}")
print(f"测试集准确率: {accuracy_score(y_test, rf.predict(X_test)): .4f}")
```

- OOB 准确率 0.9429
 - 测试集准确率 1.0000
- 模型仅仅通过利用训练过程中产生的袋外数据，就打出了 94% 的准确率
使用测试集预测时，因为鸢尾花数据集量比较小，比较幸运的做到了成功率 100%

特征重要性

业界最常用的能力：理解哪些特征在起作用，哪些特征可以舍弃

两种计量特征重要性的方法

1. 基于不纯度减少的重要性 MDI
统计每个特征在所有树的所有节点上带来的不纯度（如基尼系数）下降量之和，再求平均值
是 `sklearn` 默认的方法，计算速度快。在训练时即可得到。
会对 **高基数特征** 有偏好，如 ID 列对预测毫无意义，但会得到 MDI 的高评价
2. 基于排列的重要性 Permutation Importance
将特征值随机打乱，观察模型性能下降程度。
若下降越多，则重要性越强。
更可靠，不受基数影响

```
import matplotlib.pyplot as plt
from sklearn.inspection import permutation_importance

# 基于不纯度的重要性
feat_names = load_iris().feature_names
importances = rf.feature_importances_

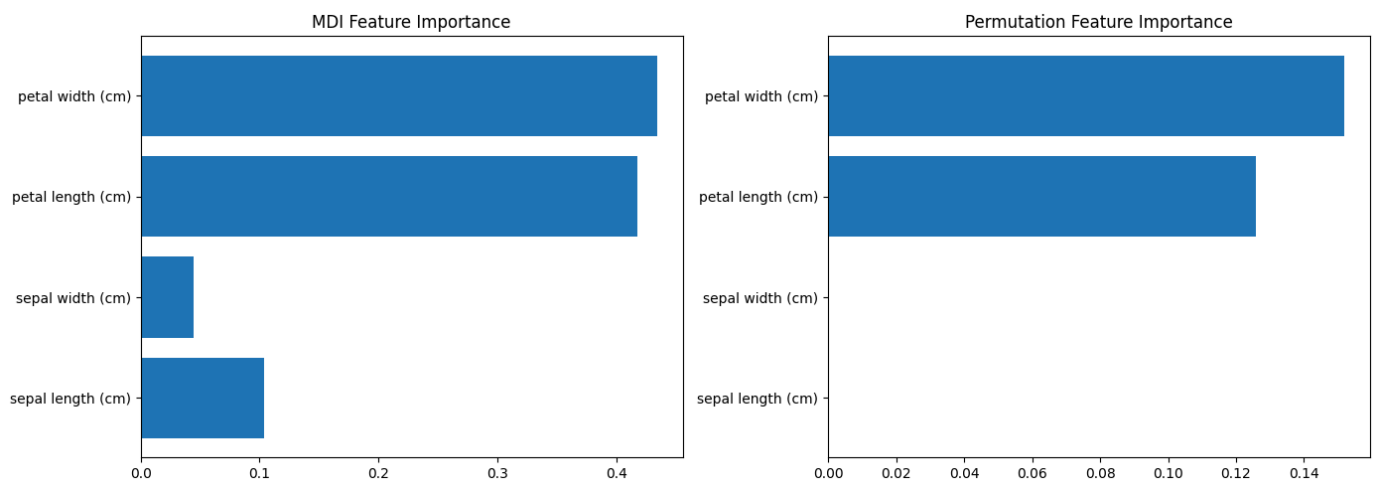
# 基于排列的重要性
perm_imp = permutation_importance(rf, X_test, y_test, n_repeats=30, random_state=42)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].barh(feat_names, importances)
axes[0].set_title('MDI Feature Importance')

axes[1].barh(feat_names, perm_imp.importances_mean)
axes[1].set_title('Permutation Feature Importance')

plt.tight_layout()
plt.show()
```



总结优劣

- 随机森林的优点
引入“**样本随机 + 特征随机**”双重随机性制造多样性
再用 **集体投票** 消除个体偏差
未必是精度最高的模型，但是值得首先尝试的模型

1. 抗过拟合能力强
极大降低模型的方差
 2. 极简的数据预处理
无需归一化，能处理缺失值，对异常值不敏感
 3. 对高维数据友好
从容应对大量特征，无需显式降维
 4. 附带特种重要性
自然输出特征贡献度，利于业务归因分析
 5. 支持并行计算
树的生成完全独立，完美利用多核 CPU 加速
- 随机森林缺点
 1. 黑盒模型
单棵树可解释性强，但多棵树双随机后，就失去直观性难以解释
 2. 计算与内存开销大
保存大量的树结构需要较多的内存空间；
预测时每条数据经过所有树，预测速度相对慢，不适合低延迟场景
 3. 对极端噪音敏感
在噪音极大的分类任务重，仍可能会出现过拟合

梯度提升树家族

梯度提升树好比 **站在巨人的肩膀上**

1. XGBoost
2. LightGBM
3. CatBoost

10. Boosting模型

本节内容

1. Boosting 模型与集成学习
2. Boosting 串行训练原理
3. Bagging 与 Boosting 对比
4. 梯度提升决策树GBDT
5. GBDT 代码实现演示
6. GBDT 优劣分析
7. 主流 Boosting 模型严谨
8. Boosting 实战与对比
9. 总结

Bagging vs Boosting

集成学习有两大主流思想：

1. Bagging
2. Boosting

他们目标相同，都要把多个弱模型组成一个强模型，但是实现路径不同

Bagging 少数服从多数

Bagging 思想指导随机森林过程，又称 **自举聚集** (Bootstrap Aggregating)

- 有放回采样 (resample)
对每棵树引入双重随机性，随机训练数据、随机特征，形成多样性
- 最终投票/平均
综合所有树的意见给出结果
- 代表算法
随机森林

Boosting 接力纠错

- 串行训练
 - 每棵树依次训练
- 专注错误
 - 后树专注纠正前树的错误
- 累加预测
 - 最终累加所有树的预测
- 代表算法
 - 1. XGBoost
 - 2. LightGBM
 - 3. CatBoost



Boosting 的精髓

每个新模型都在拟合前面模型的残差（错误），接力合作，逐步逼近真实答案。

对比

维度	Bagging（随机森林）	Boosting（梯度提升树）
训练方式	多棵树并行独立训练	多棵树串行依次训练
每棵树的目标	学习原始目标	学习前面模型的残差

GBDT 核心原理

梯度提升决策树 (Gradient Boosting Decision Tree, GBDT)

$$F(x) = h_1(x) + h_2(x) + h_3(x) + \dots$$

与线性回归有固定维度的参数 θ 不同，GBDT 的优化方式是每轮添加一整棵新树 $h_m(x)$

GBDT 作者 Friedman 将梯度下降类比为如下场景

	经典梯度下降	GBDT
优化对象	参数 θ 一组数字	函数 $F(x)$ 一个模型
当前状态	当前参数 θ_{m-1}	当前模型 $F_{m-1}(x)$
下降方向	负梯度 $-\frac{\partial L}{\partial \theta}$	负梯度 $-\frac{\partial L}{\partial F(x)}$

	经典梯度下降	GBDT
更新方式	$\theta_m = \theta_{m-1} - \eta \cdot \frac{\partial L}{\partial \theta}$	$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$
每步操作	调整参数数值	添加新树 $h_m(x)$

$$F_m(x) = F_0(x) + \eta \sum_{k=1}^m h_k(x)$$

- GBDT 循环操作
 - 计算每个样本上的负梯度，作为当前模型最该修正的方向
 - 训练一棵决策树以拟合这些负梯度
 - 将该树乘以学习率，再添加到现有模型上，作为沿着下坡方向的一小步
- 当损失函数是均方误差时
此时负梯度恰是残差

$$L = \frac{1}{2}(y - F(x))^2$$

L 对 $F(x)$ 求导并取负号，负梯度 = $-\frac{\partial L}{\partial F(x)} = y - F(x)$ = 残差(residual)

推广其他损失函数，负梯度不再是简单的残差

损失函数	负梯度（每棵新树拟合的目标）
均方误差（MSE）	$y - F(x)$ ，是残差
绝对误差（MAE）	$\text{sign}(y - F(x))$ ，只有方向 ± 1 ，对异常值更稳健
对数损失（分类）	$y - p$ ，真实标签减去预测概率，这也是为什么 GBDT 可以处理分类问题

对于能求导的损失函数，GBDT 都能计算出负梯度，进而训练新树拟合

- Gradient 梯度
指优化方法，用负梯度指引新树的学习方向
- Boosting 提升
指最终效果，叠加多棵弱决策树提升模型表现

合并二者即为 **梯度提升决策树** Gradient Boosting Decision Tree

GBDT 代码实现

手搓简版

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeRegressor

# 生成带噪声的正弦曲线数据
np.random.seed(42)
X = np.sort(np.random.rand(200, 1) * 10, axis = 0)
y = np.sin(X).ravel() + np.random.randn(200) * 0.2

# 初始化参数
n_trees = 9 # 树的总量
learning_rate = 0.3 # 学习率
trees = [] # 树的容器
F = np.full_like(y, y.mean()) # 步骤1: 用均值初始化

# 准备绘图
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
fig, axes = plt.subplots(3, 3, figsize = (16,9))

# 建模训练
```

```

for i in range(n_trees):
    # 计算残差 (MSE下的负梯度)
    residual = y - F

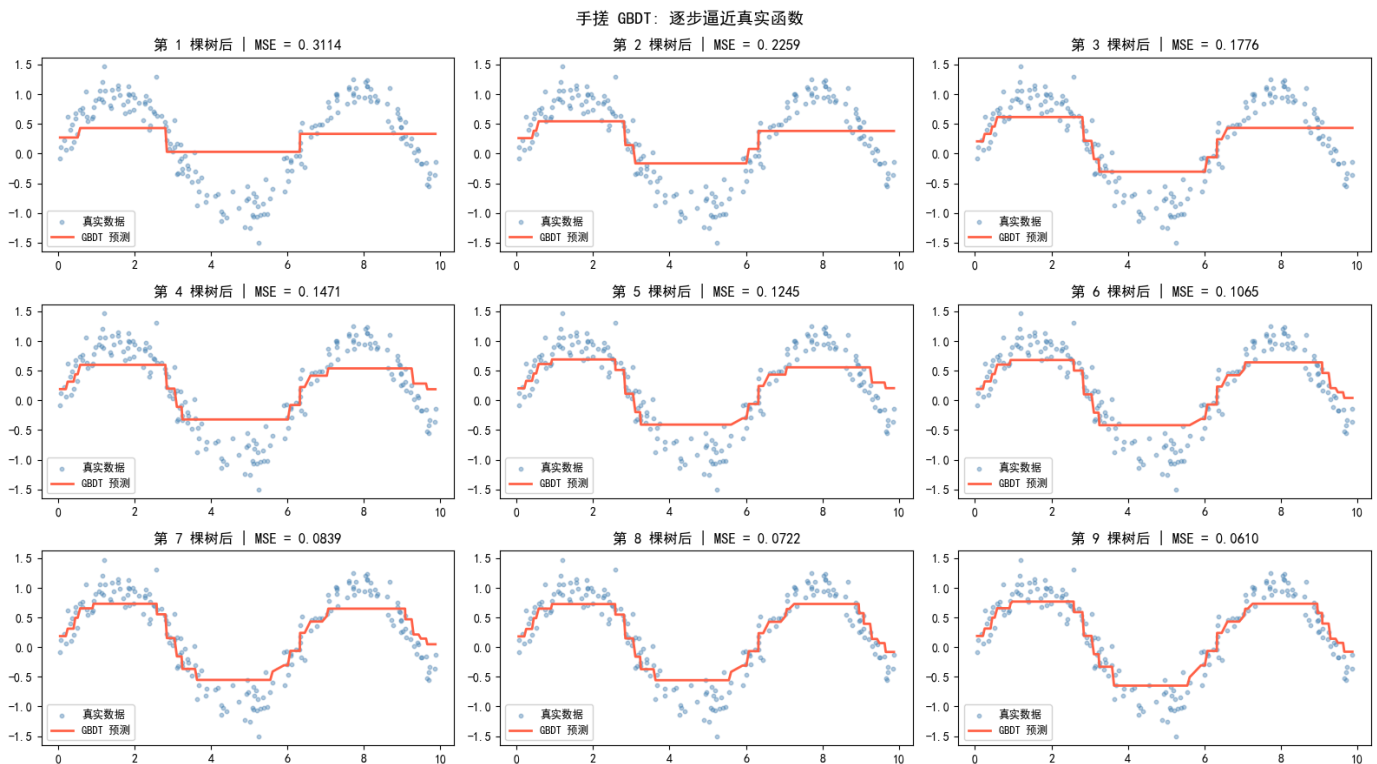
    # 用一棵浅树拟合残差
    tree = DecisionTreeRegressor(max_depth = 2)
    tree.fit(X, residual)
    pred = tree.predict(X)
    trees.append(tree)

    # 更新模型
    F += learning_rate * pred

    # 可视化
    mse = np.mean((y - F) ** 2)
    ax = axes[i // 3][i % 3] # 子图行列索引
    ax.scatter(X, y, s = 10, alpha = 0.4, color = 'steelblue', label = '真实数据')
    ax.plot(X, F, color = 'tomato', linewidth = 2, label = f'GBDT 预测')
    ax.set_title(f'第 {i + 1} 棵树后 | MSE = {mse:.4f}', fontsize = 12)
    ax.legend(fontsize = 9)

plt.suptitle('手搓 GBDT: 逐步逼近真实函数', fontsize = 14, fontweight = 'bold')
plt.tight_layout()
plt.show()

```



每棵新树都在查漏补缺，模型一轮比一轮精准

- 第 1~3 棵树
红色曲线只是一个粗略的阶梯形，MSE 较大
- 第 4~6 棵树
曲线开始有了正弦的轮廓，MSE 明显下降
- 第 7~9 棵树
曲线越来越贴合蓝色散点，MSE 持续缩小

观察残差

```

fig, axes = plt.subplots(3, 3, figsize = (16,9))

F = np.full_like(y, y.mean())

```

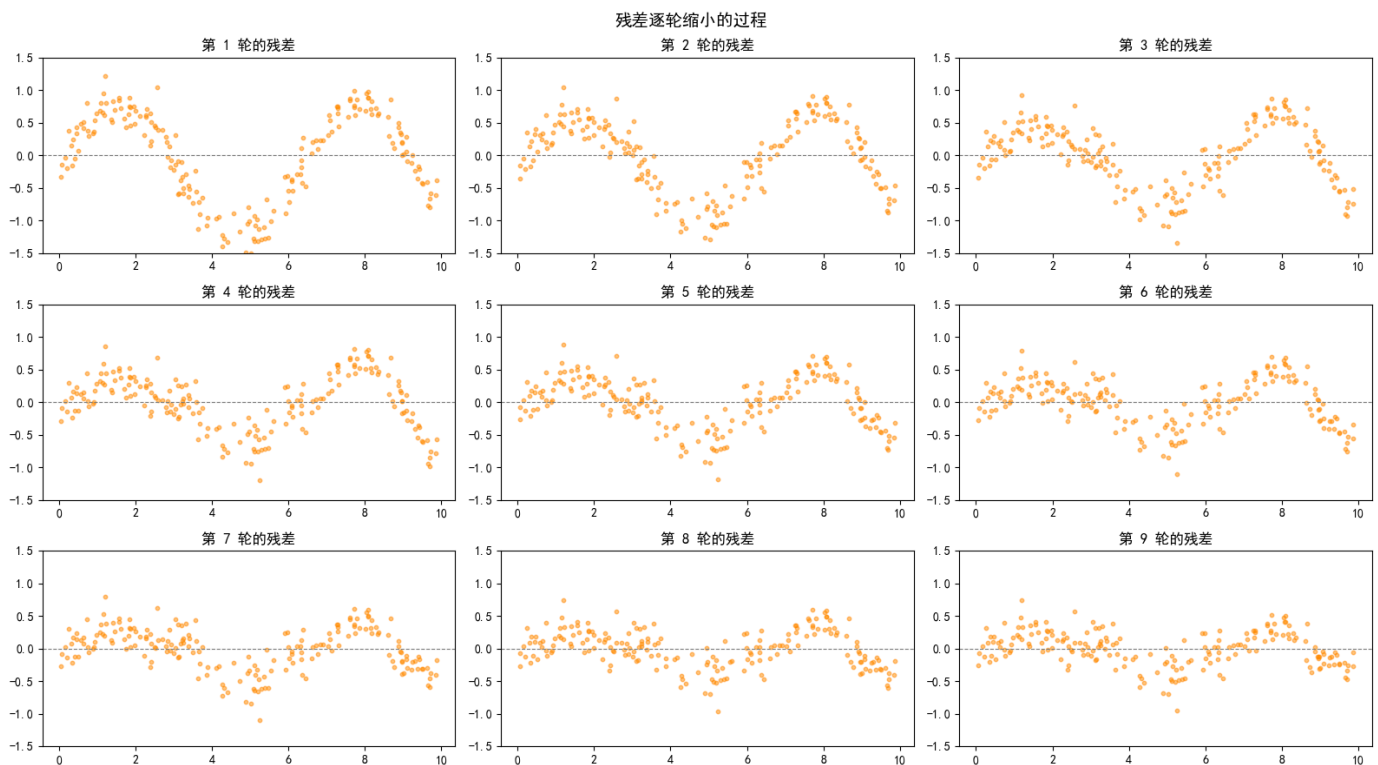
```

for i in range(n_trees):
    residual = y - F
    tree = DecisionTreeRegressor(max_depth = 2)
    tree.fit(X, residual)
    F += learning_rate * tree.predict(X)

    ax = axes[i // 3][i % 3]
    ax.scatter(X, residual, s = 10, alpha = 0.5, color = 'darkorange')
    ax.axhline(y = 0, color = 'gray', linestyle = '--', linewidth = 0.8)
    ax.set_title(f'第 {i + 1} 轮的残差', fontsize = 12)
    ax.set_ylim(-1.5, 1.5)

plt.suptitle('残差逐轮缩小的过程', fontsize = 14, fontweight = 'bold')
plt.tight_layout()
plt.show()

```



Boosting 逐步纠错的过程:

残差（橙色散点）逐轮向零线收敛

GBDT优劣

优点

1. 预测精度高
在结构化/表格数据上表现良好
2. 处理混合特征
对数值型、类别型 (需要编码) 的数据均适用
3. 自带特征重要性
帮助解释特征影响
4. 对特征缩放不敏感
不需要标准化或归一化
5. 损失函数灵活
适配多种业务需求

缺点

1. 训练速度慢
串行训练，树之间无法并行

- 2. 容易过拟合
 - 尤其是数据量少、噪声大时，需要仔细调参
- 3. 对超参数敏感
 - 需要认真调节学习率、树的数量、节点深度等
- 4. 不擅长超高维稀疏数据
 - 如文本的 one-hot 编码
- 5. 原生无法直接处理类别特征
 - 经典 GBDT 需要对类别型特征手动编码

业界演进

XGBoost

2014年 陈天奇 开发 XGBoost (eXtreme Gradient Boosting)

核心改进：

- 正则化
 - 在目标函数中显式加入树复杂度的惩罚项（叶节点数量 + 叶节点权重的 L2 正则），有效防止过拟合
- 二阶梯度信息
 - 经典 GBDT 只用一阶梯度，XGBoost 同时利用二阶梯度，使得分裂点选择更精准
- 工程优化
 - 支持列采样、缓存感知访问、核外计算等，使得单机处理上亿样本成为可能
- 内置正则化和早停 (Early Stopping)
 - 训练过程更可控

LightGBM

2017年 微软 推出 LightGBM，大数据集训练相较 XGBoost 快 5~10 倍

核心改进

- 直方图算法
 - XGBoost 默认遍历所有数据点寻找分裂点；
 - LightGBM 将连续的浮点特征分段 (分桶) 成离散的直方图，寻找分裂点时只需遍历这些分桶，计算量呈指数下降，并且内存占用极小
- 带深度限制的 **Leaf-wise** 生长策略
 - 传统决策树是逐层整齐生长的 (level-wise)
 - LightGBM 顺沿大残差值节点往下生长 (leaf-wise)，得到非对称结构树，但在同等叶子节点下能达到更高精度
- 引入新技术加快训练速度
 1. 单边梯度采样 (Gradient-based One Side Sampling, GOSS)
 2. 互斥特征捆绑 (Exclusive Feature Bundling, EFB)

CatBoost

2017年 Yandex 针对类别特征的处理推出 CatBoost (Category Boosting)

在处理具有大量类别特征的业务场景，CatBoost 表现突出

核心改进：

- 原生支持类别特征
 - XGBoost 需要开发者手动对类别特征进行 One-hot 编码
 - CatBoost 内置 Target Statistics 方法自动处理类别特征
- 排序提升
 - GBDT 家族具有预测偏移的隐患，即当前训练数据的目标值会影响当前模型的构建
 - CatBoost 通过严谨的排序策略，确保对每个样本训练时，仅使用排序靠前的样本，极大提升泛化能力
- 对称树结构
 - CatBoost 强制生成完全对称的决策树，使得预测速度 (推理速度) 比 XGBoost 和 LightGBM 显著更快

三者对比

场景	推荐框架
需要极致精度，如Kaggle 竞赛	XGBoost 或 LightGBM

场景	推荐框架
需要快速训练，大数据集 (>100万样本)	LightGBM
需要处理大量类别特征	CatBoost

XGBoost 演示

准备数据

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split

# 加载加州房价数据集
data = fetch_california_housing()
X, y = data.data, data.target
feature_names = data.feature_names

# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f"训练集大小: {X_train.shape}")
print(f"测试集大小: {X_test.shape}")
print(f"特征: {feature_names}")
```

训练模型

1. Random Forest

```
import time
from sklearn.ensemble import RandomForestRegressor

# ---随机森林---
rf_start = time.time()
rf = RandomForestRegressor(
    n_estimators= 200,      # 200棵树
    max_depth= 10,         # 最深10层
    random_state= 42,      # 随机种子
    n_jobs= -1             # CPU 并行加速
)

rf.fit(X_train, y_train)
rf_time = time.time() - start
rf_pred = rf.predict(X_test)
```

2. XGBoost

```
import time
from xgboost import XGBRegressor

# ---XGBoost---
xgb_start = time.time()
xgb = XGBRegressor(
    n_estimators = 200,      # 200棵树
    max_depth = 5,          # 最深5层
    learning_rate = 0.1,    # 学习率
    subsample = 0.8,        # 每次训练抽选样本比例
    colsample_bytree = 0.8, # 每次训练抽选特征比例
    random_state = 42,      # 随机种子
    n_jobs = -1,           # CPU 并行加速
)
```

```
xgb.fit(X_train, y_train)
xgb_time = time.time() - xgb_start
xgb_pred = xgb.predict(X_test)
```

对比结果

```
from sklearn.metrics import mean_squared_error, r2_score

# ---对比结果---
print("=" * 50)
print(f"{'指标':<15}{'随机森林':>12}{'XGBoost':>12}")
print("=" * 50)
print(f"{'MSE':<15} {mean_squared_error(y_test, rf_pred):>12.4f} "
      f"{'mean_squared_error(y_test, xgb_pred):>12.4f}")
print(f"{'R² Score':<15} {r2_score(y_test, rf_pred):>12.4f} "
      f"{'r2_score(y_test, xgb_pred):>12.4f}")
print(f"{'训练时间(秒)':<15} {rf_time:>12.2f} {xgb_time:>12.2f}")
```

指标	随机森林	XGBoost
MSE	0.2951	0.2183
R2 Score	0.7748	0.8334
训练时间(秒)	0.83	0.35

早停防过拟合

XGBoost 的一个实用功能是 **Early Stopping**

当验证集上的性能不再提升时，自动停止训练

```
from xgboost import XGBRegressor

xgb_es = XGBRegressor(
    n_estimators=10000,      # 设一个很大的上限
    max_depth=5,
    learning_rate=0.05,      # 用更小的学习率
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    early_stopping_rounds=20 # 20轮不提升就停止
)

xgb_es.fit(
    X_train, y_train,
    eval_set=[(X_test, y_test)],
    verbose=50              # 每50轮打印一次
)

print(f"\n最佳迭代轮数: {xgb_es.best_iteration}")
print(f"最佳MSE: {mean_squared_error(y_test, xgb_es.predict(X_test)):.4f}")
```

- 最佳迭代轮数：862
- 最佳MSE：0.1963

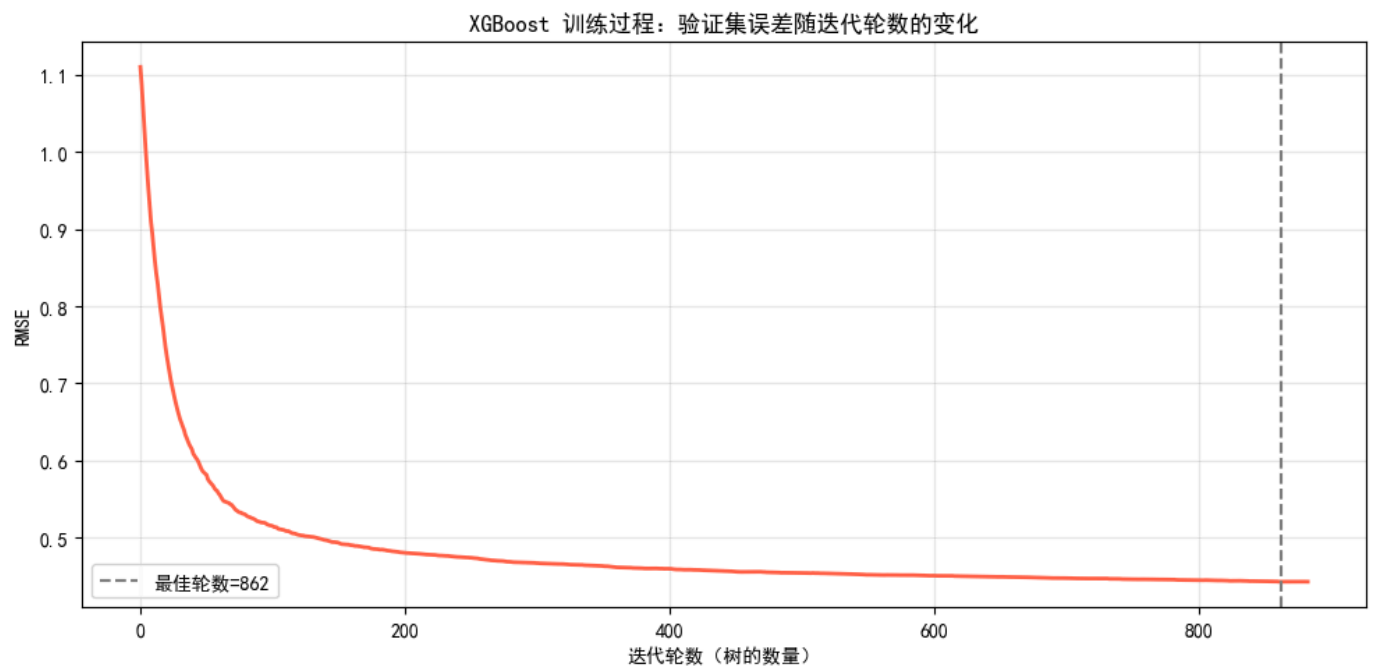
Early Stopping 省略预估树的总量

预设一个足够大的上限，让算法自己找到最佳停止点

可视化训练过程

```
# 获取训练过程中的损失变化
results = xgb_es.evals_result()

plt.figure(figsize=(10, 5))
plt.plot(results['validation_0']['rmse'], color='tomato', linewidth=2)
plt.xlabel('迭代轮数 (树的数量)')
plt.ylabel('RMSE')
plt.title('XGBoost 训练过程：验证集误差随迭代轮数的变化')
plt.axvline(x=xgb_es.best_iteration, color='gray',
            linestyle='--', label=f'最佳轮数={xgb_es.best_iteration}')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



会看到验证集的 RMSE 先快速下降，然后趋于平缓，最终可能略微上升（过拟合的信号），Early Stopping 恰好最低点附近停下来。

总结

对决策树以及使用决策树的两大集成学习分支随机森和GBDT 进行总结

	决策树	随机森林	GBDT / XGBoost
模型数量	1棵	多棵（并行）	多棵（串行）
集成策略	无	Bagging：投票/平均	Boosting：逐步累加
单棵树特点	可深可浅	深树（低偏差高方差）	浅树（高偏差低方差）
核心目标	基线模型	降低方差	降低偏差
过拟合风险	高	低	中等
训练速度	快	较快（可并行）	较慢（串行）
预测精度	一般	好	通常最好

- **决策树**是基础模型——简单符合直觉，但容易过拟合
- **随机森林**用 Bagging 思想，让多棵树并行投票，降低方差
- **GBDT**用 Boosting 思想，让多棵树串行纠错，降低偏差
- **XGBoost / LightGBM / CatBoost**是 GBDT 的工业级进化，更快、更准、更易用

在实际工作中，面对结构化的表格数据，GBDT 家族（尤其是 XGBoost 和 LightGBM）几乎是默认的首选方案，它们在无数 Kaggle 竞赛和工业场景中证明了自己的实力。

11. K-Means 聚类

无监督学习让数据物以类聚

本节内容

1. 无监督聚类问题
2. K-Means 核心思想
3. 质心与簇内相似度
4. K-Means 迭代流程
5. 完整流程演示
6. K-Means 优化
7. 聚类结构常见应用
8. 总结

无监督聚类问题

没有标签的数据内含结构或规律，需要通过算法揭示

- 聚类问题
在没有标签的前提下，根据样本之间的相似性，将数据自动划分为若干组，使得**同组内样本尽可能相似，不同组之间尽可能有差异**
 1. 用户画像与分群
根据购买频次、消费金额等，将用户划分为高价值、潜力或流失风险用户
 2. 图像压缩
从像素中找出典型色彩，减少存储空间
 3. 异常检测
识别偏离常规的数据点（如异常访问、恶意攻击），只通过数据分布特征识别

K-Means 核心思想

K-Means

- 是经典聚类算法，将一批无标签数据自动划分为K个簇
 - K：预先指定的簇数
 - Means：均值，即每个簇的中心位置由该簇内所有样本的均值来表示
- 目标是找到K个具有代表性的中心点，使每个样本归属于离自己最近的中心，并尽量减小样本到中心的距离

质心与簇内相似度

质心 Centroid

是一个簇内的中心位置，通常取该簇所有点在各个维度上的**均值**

$$\text{假设三点 } A(x_a, y_a), B(x_b, y_b), C(x_c, y_c), \\ \text{则质心为 } (\frac{x_a + x_b + x_c}{3}, \frac{y_a + y_b + y_c}{3})$$

距离度量

最常用的距离度量是 **欧氏距离** (直线距离)

二位平面中A、B两点的距离

$$d(A, B) = \sqrt{(A_x - B_x)^2 + (A_y - B_y)^2}$$

推广 n 维空间

$$d(A, B) = \sqrt{\sum_{i=1}^n (A_i - B_i)^2}$$

其基本思路并没有变化，只是在每一个维度上分别求差、平方、求和，最后再开平方。

簇内相似度

- K-Means 目标
同个簇的样本尽量彼此接近，并且尽量靠近该簇质心

簇内相似度的度量使用 **簇内平方和** (Sum of Squared Errors, SSE)

$$SSE = \sum_{i=1}^K \sum_{x \in C_i} \|x - \mu_i\|^2$$

- C_i : 表示第 i 个簇
- μ_i : 表示第 i 个簇的质心
- $\|x - \mu_i\|^2$: 表示样本点到质心的距离平方

SSE 越小，说明簇内样本越紧凑，聚类效果越好

K-Means 的本质就是通过不断调整样本划分和质心位置，使簇内平方和持续减小，直到聚类结果趋于稳定

K-Means 迭代流程

选中心 → 分簇 → 更新中心 → 再分簇 → 直到稳定

1. 初始化质心
预先指定聚类的簇数 K
然后从数据集中随机选取 K 个初始中心作为质心
2. 分配样本
对于数据集中的每一个样本，分别计算它到各个质心的距离，
并将其划分到距离最近的那个质心所在的簇中。
经过这一步，所有样本都会被分配到某一个簇里。
3. 更新质心
当所有样本完成划分后，每个簇内部就包含了一组成员。
再需要根据当前簇中的所有样本，重新计算该簇的中心点，即求这些样本在各个维度上的均值。
更新后的质心会更准确地反映该簇样本的整体位置。
4. 重复迭代
由于质心位置发生了变化，原有的样本划分结果可能不再是最优的。
因此，需要重新计算每个样本到新质心的距离，并再次将样本分配到最近的簇中。
随后再依据新的簇划分结果更新质心。
如此反复，算法便进入持续迭代的过程。
5. 收敛停止
当满足以下任一条件时，算法可以停止迭代：
 - 质心的位置几乎不再发生变化；
 - 样本的簇归属不再变化；
 - 目标函数（如簇内平方和）的下降幅度已经非常小；
 - 达到预先设定的最大迭代次数

综上，K-Means 的核心就是在**样本分配**和**质心更新**之间不断循环，
通过迭代方式逐步减小簇内误差，直到聚类结果趋于稳定。

代码实现

生成数据

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

plt.rcParams['font.sans-serif'] = ['SimHei'] # 中文显示
plt.rcParams['axes.unicode_minus'] = False # 负号显示

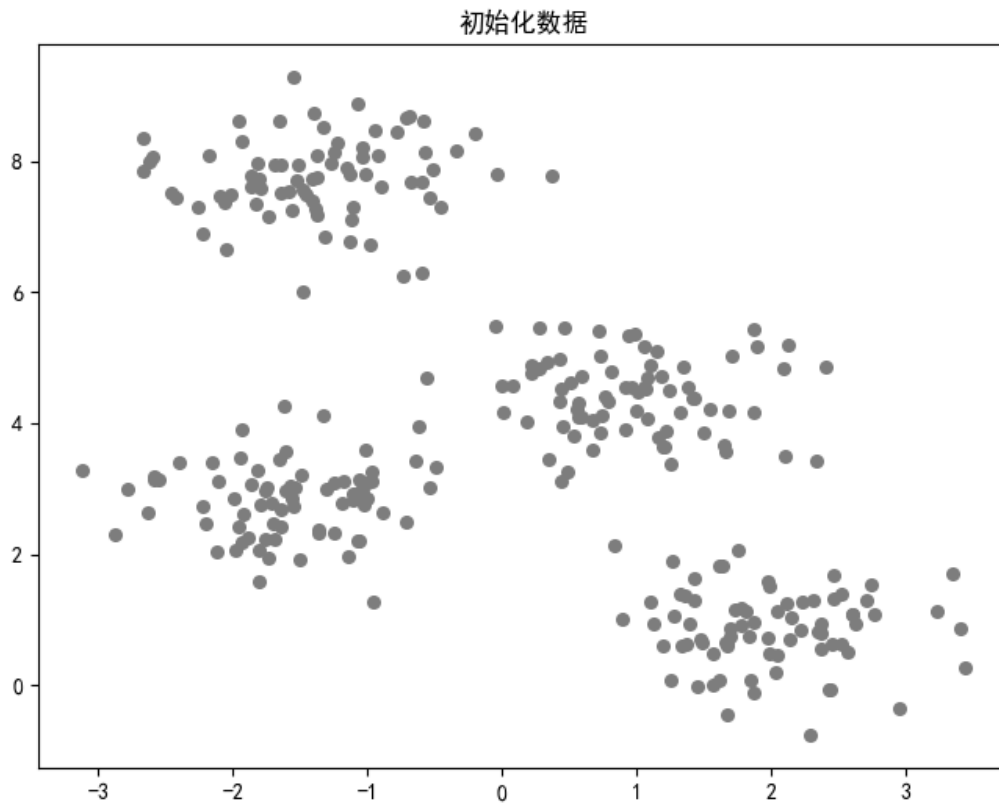
np.random.seed(42) # 随机种子
# 生成数据
X, y_true = make_blobs(
    n_samples = 300, # 样本数量
    centers = 4, # 中心数量
```

```

cluster_std = 0.6, # 簇离散度
random_state = 0,
)

plt.figure(figsize = (8,6))
plt.scatter(X[:,0], X[:,1], s = 30, c = 'gray')
plt.title("初始化数据")
plt.show()

```



初始化中心

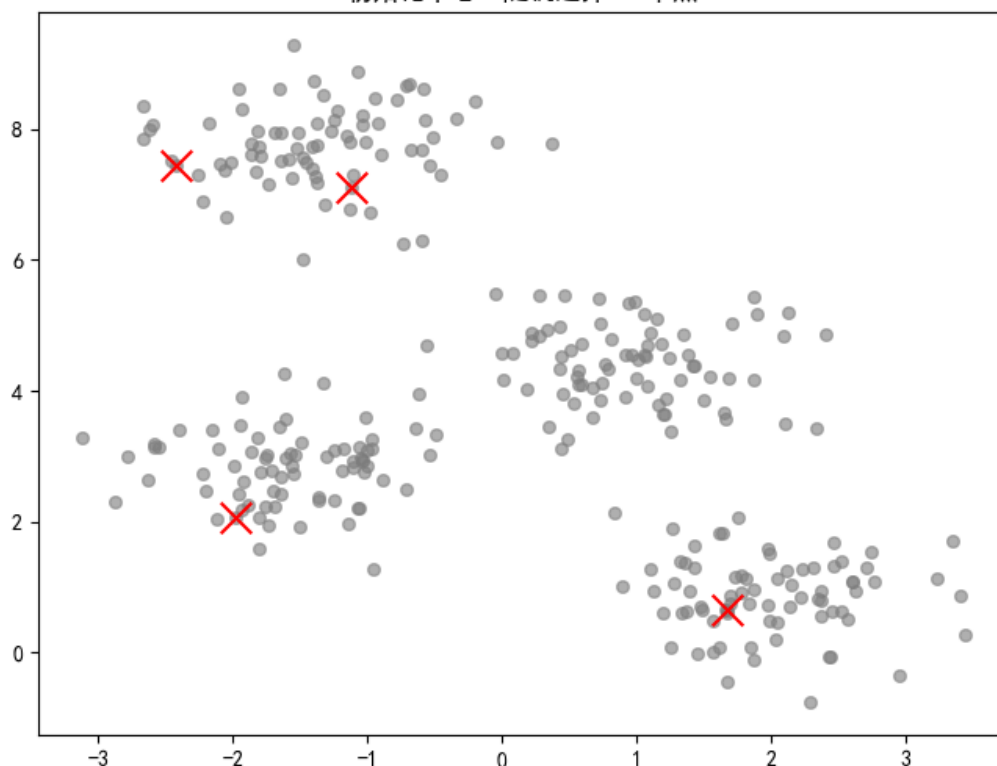
```

# 随机选择 k 个初始中心
k = 4
initial_indices = np.random.choice(X.shape[0], k, replace = False)
centroids = X[initial_indices]

plt.figure(figsize= (8,6))
plt.scatter(X[:,0], X[:,1], s=30, c='gray')
plt.scatter(centroids[:,0], centroids[:,1], s=200, c='red', marker='x', edgecolors='black')
plt.title("初始化中心: 随机选择 k 个点")
plt.show()

```

初始化中心：随机选择 k 个点



逐轮迭代

```
from scipy.spatial.distance import cdist
# 设置组图画幅
fig, axes = plt.subplots(6,2, figsize = (16, 36))

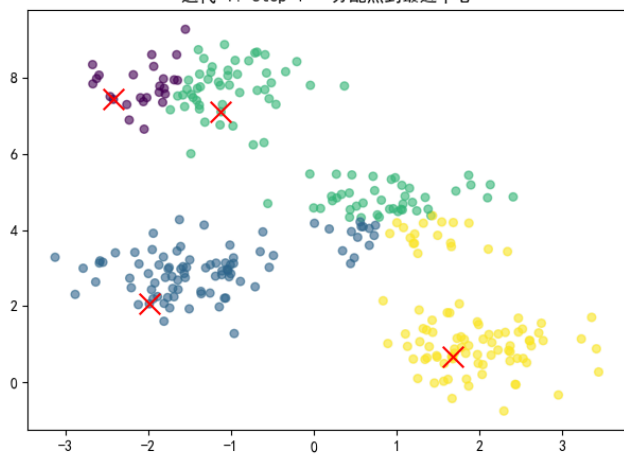
# 设置迭代上限
max_iters = 15
for i in range(max_iters):
    # ---Step 1 分配中心---
    distances = cdist(X, centroids) # 计算各样本点至各个中心的距离
    labels = np.argmin(distances, axis=1) # 按距离最近的中心分配标签
    ax = axes[i][0]
    ax.scatter(X[:,0], X[:,1], s=30, c=labels, cmap='viridis', alpha=0.6)
    ax.scatter(centroids[:,0], centroids[:,1], s=200, c='red', marker='x', edgecolors='black')
    ax.set_title(f"迭代 {i+1} : Step 1 - 分配点到最近中心")

    # ---Step 2 更新中心---
    new_centroids = np.array([X[labels == j].mean(axis=0) for j in range(k)]) # 计算各簇均值
    ax = axes[i][1]
    ax.scatter(X[:,0], X[:,1], s=30, c=labels, cmap='viridis', alpha=0.6)
    ax.scatter(new_centroids[:,0], new_centroids[:,1], s=200, c='red', marker='x', edgecolors='black')
    ax.set_title(f"迭代 {i+1} : Step 2 - 更新中心到簇均值")

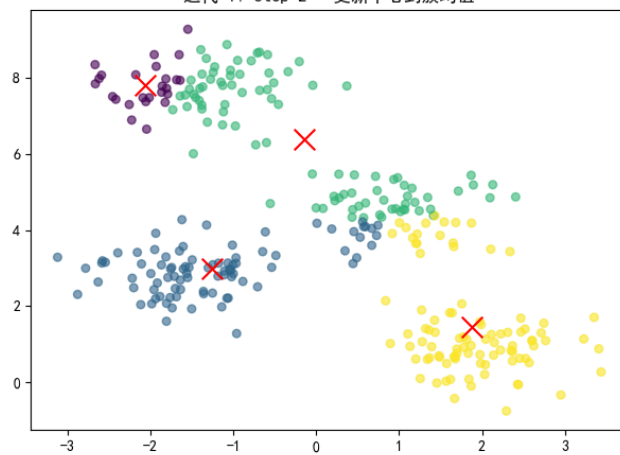
    # ---Step 3 检查收敛---
    if np.all(centroids == new_centroids):
        print(f"算法在 {i+1} 次迭代后完全收敛！中心不再移动。")
        break
    else:
        centroids = new_centroids

plt.suptitle("K-Means 逐轮迭代", fontsize = 12, fontweight = 'bold')
plt.tight_layout()
plt.show()
```

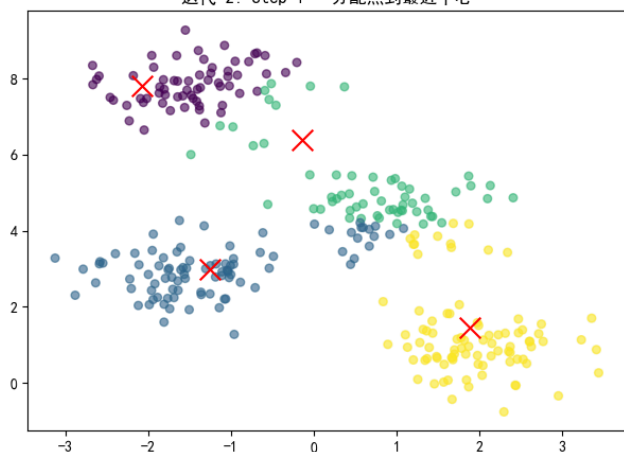
迭代 1: Step 1 - 分配点到最近中心



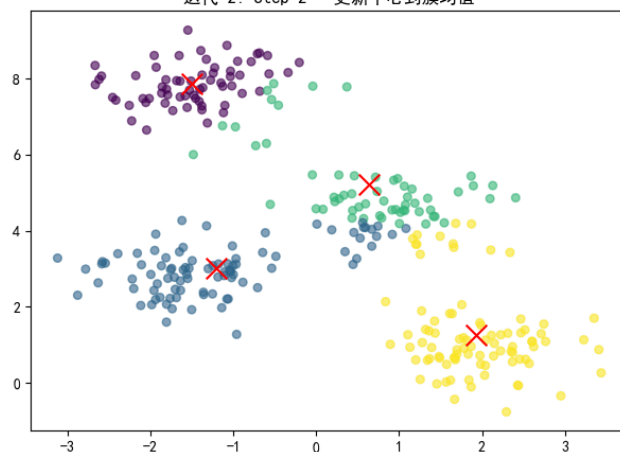
迭代 1: Step 2 - 更新中心到簇均值



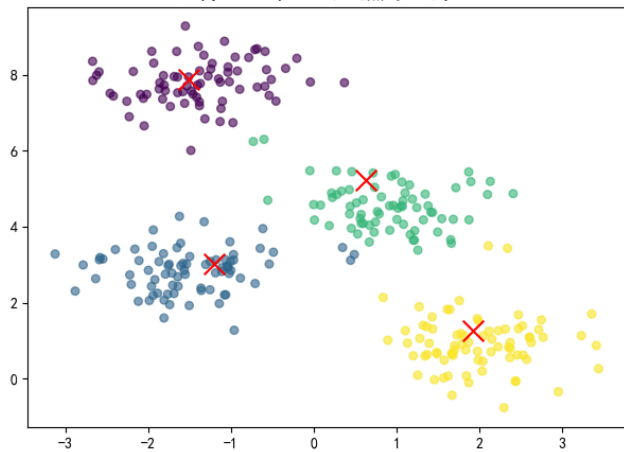
迭代 2: Step 1 - 分配点到最近中心



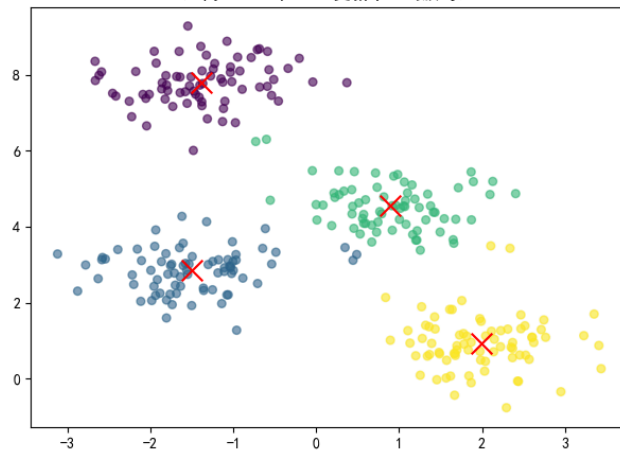
迭代 2: Step 2 - 更新中心到簇均值



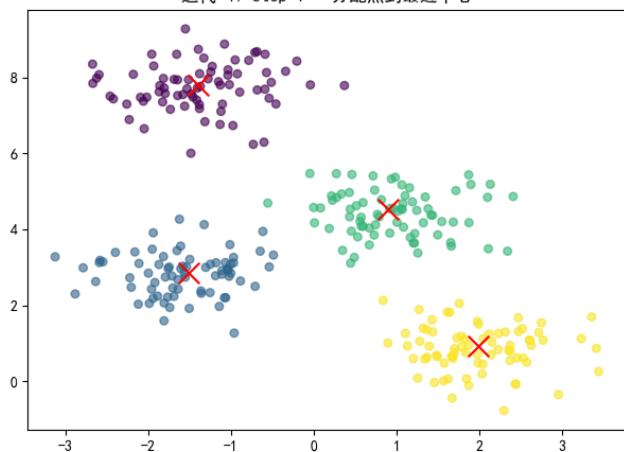
迭代 3: Step 1 - 分配点到最近中心



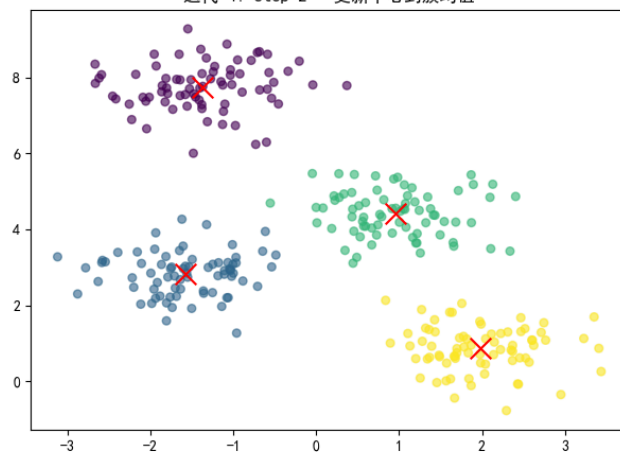
迭代 3: Step 2 - 更新中心到簇均值



迭代 4: Step 1 - 分配点到最近中心

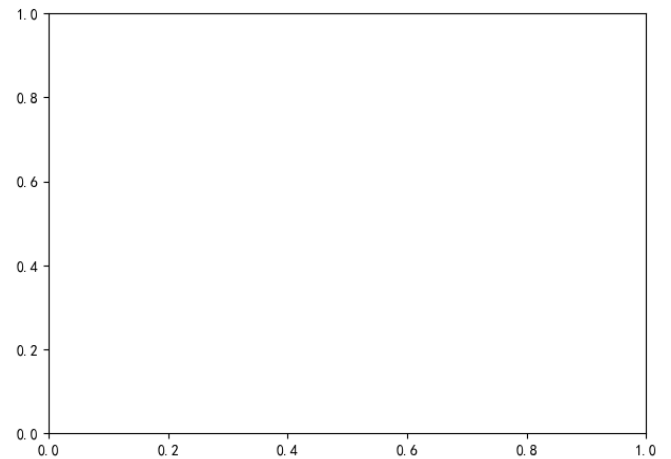
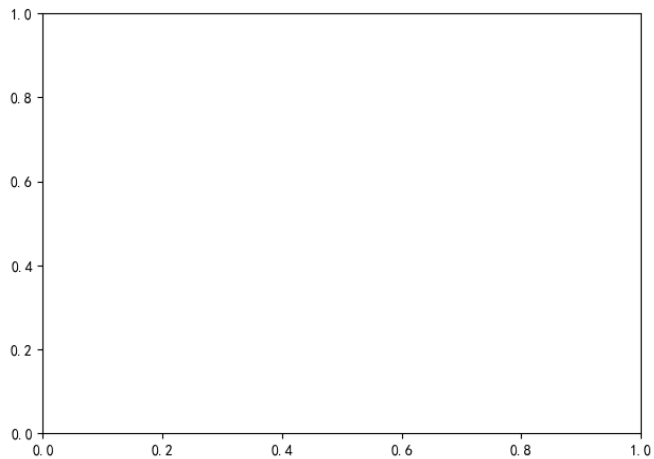
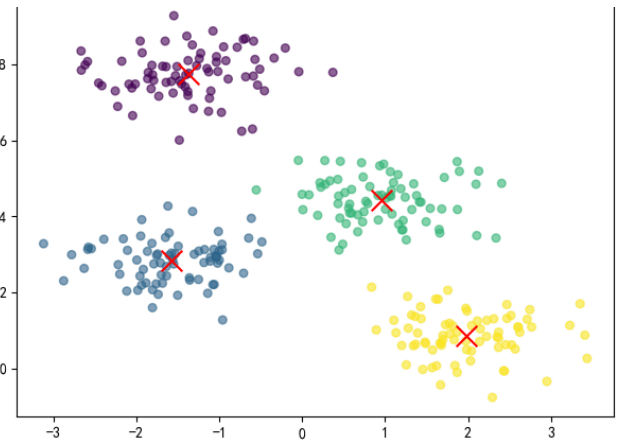
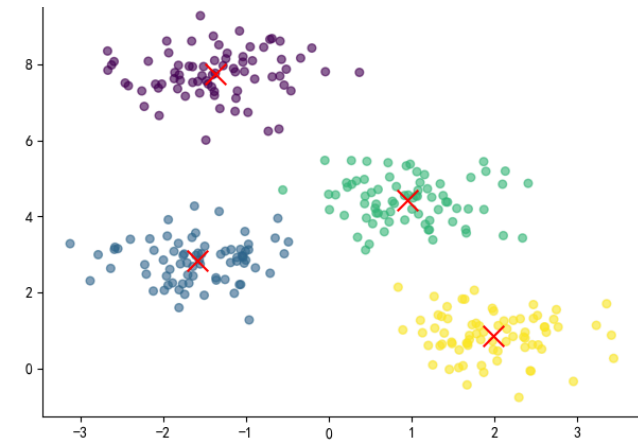


迭代 4: Step 2 - 更新中心到簇均值



迭代 5: Step 1 - 分配点到最近中心

迭代 5: Step 2 - 更新中心到簇均值



复现过程与[原作笔记演示](#)不同，仅迭代5次完成收敛

优化

初始质心选择 K-Means++

随机选择初始质心可能导致算法陷入较差的局部最优解

例如前述演示过程，如果多个初始质心定位靠近，那么可能15次迭代后仍未收敛

不同的初始化方式，最终得到的聚类结果可能存在明显差异

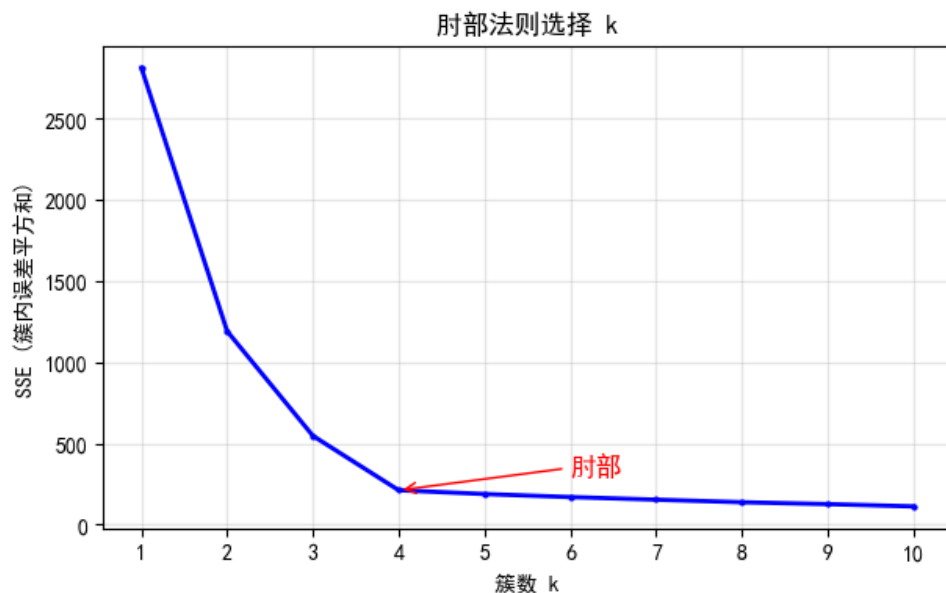
- K-Means++
 - 先随机选取一个样本点作为第一个质心
 - 选择下一个质心时，优先考虑距离当前已有质心较远的样本点
 - 重复这一过程，直到选出 K 个初始质心

肘部法则选择 K 值

K-Means 一大问题是如何预先指定簇数 K 效率更高

- 依次尝试不同 K 值，计算对应的 SSE
- 随着 K 增大，SSE 会持续下降
- 当 K 突破某个值后，SSE 下降幅度明显减小，曲线趋于平缓

斜率显著减小、曲线趋于平缓的拐点 称为肘部，是 K 值的较优选择



可视化

结合 K-Means++ 和肘部法则优化

```
from sklearn.datasets import make_blobs
# 1. 生成数据
X, y_true = make_blobs(
    n_samples = 300,
    centers = 4,
    cluster_std = 0.8,
    random_state = 42,
)

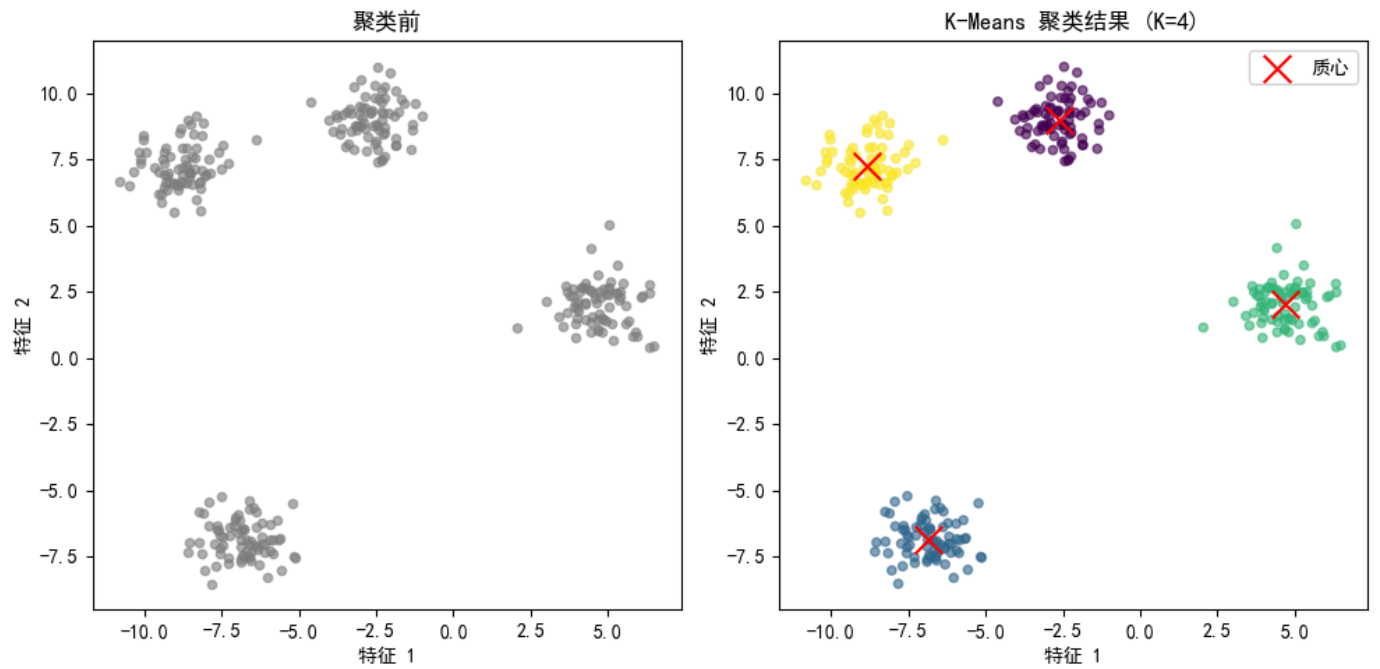
from sklearn.cluster import KMeans
# 2. 运行 K-Means
kmeans = KMeans(
    n_clusters = 4,
    init = 'k-means++',
    n_init = 10,
    random_state = 42,
)
y_pred = kmeans.fit_predict(X)
centroids = kmeans.cluster_centers_ # 获取质心

import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
# 3. 可视化
plt.figure(figsize = (10,5))
## 聚类前
plt.subplot(1, 2, 1)
plt.scatter(X[:, 0], X[:, 1], c='gray', s=20, alpha=0.6)
plt.title('聚类前')
plt.xlabel('特征 1')
plt.ylabel('特征 2')

## 聚类后
plt.subplot(1, 2, 2)
plt.scatter(X[:, 0], X[:, 1], c=y_pred, s=20, cmap='viridis', alpha =0.6)
plt.scatter(centroids[:, 0], centroids[:, 1], c='red', s=200, marker='x', edgecolors='black',
            linewidths=1.5, label='质心')
plt.title('K-Means 聚类结果 (K=4)')
plt.xlabel('特征 1')
plt.ylabel('特征 2')
plt.legend()
```

```
plt.tight_layout()
plt.show()

print(f"SSE: {kmeans.inertia_: .2f}")
print(f"迭代次数: {kmeans.n_iter_}")
```



- SSE: 362.47
- 迭代次数: 3

常见应用

聚类算法虽然是一种无监督学习方法，不依赖任何标注数据，但在实际业务中却有着非常广泛的应用价值。它的核心作用是从无标签的数据中自动发现隐藏的结构和规律，进而为决策提供支持。

应用领域	典型场景	聚类的核心作用
客户分群	电商用户 RFM 分层	自动识别用户价值层级，支撑精准运营
图像处理	颜色量化、图像分割	压缩颜色空间，划分图像语义区域
文档聚类	新闻 / 论文自动归类	将海量文本按主题自动归组，辅助检索与推荐
异常检测	网络入侵、信用卡欺诈	将远离所有簇中心的样本标记为潜在异常
数据预处理	连续特征离散化分桶	自适应分桶，增强特征表达能力
推荐系统	协同过滤的冷启动	将新用户 / 物品归入相似簇，快速生成初始推荐

聚类算法常作为数据探索、特征工程和系统冷启动等环节的重要组成部分，与其他模型配合使用，从而在更大范围内发挥作用。

图片压缩

使用 K-Means 把彩色图片的多种颜色，压缩成 4 种、16 种、64 种颜色，实现图片瘦身、简化色彩

```
from sklearn.cluster import KMeans
from skimage import io
import numpy as np
import matplotlib.pyplot as plt

# 1. 读取图片并归一化
image = io.imread('your_image.jpg') # 替换为你的图片路径
image = image / 255.0 # 归一化到 [0, 1]
```

```

h, w, c = image.shape
pixels = image.reshape(-1, 3) # 展平为 (h*w, 3) 的像素矩阵

# 2. 对不同 K 值进行压缩
fig, axes = plt.subplots(1, 4, figsize=(20, 5))
axes[0].imshow(image)
axes[0].set_title(f'原图 ({len(np.unique(pixels, axis=0))} 种颜色) ')
axes[0].axis('off')

for idx, k in enumerate([4, 16, 64]):
    km = KMeans(n_clusters=k, n_init=10, random_state=42)
    labels = km.fit_predict(pixels)
    compressed = km.cluster_centers_[labels].reshape(h, w, c)

    axes[idx + 1].imshow(compressed)
    axes[idx + 1].set_title(f'K={k} ({k} 种颜色) ')
    axes[idx + 1].axis('off')

plt.suptitle('K-Means 图像颜色压缩', fontsize=16)
plt.tight_layout()
plt.show()

```

K-Means 图像颜色压缩



总结优劣

优点

- 思想简单，容易理解
流程清晰，不断“分配样本”和“更新质心”
- 计算效率高
适合大规模数据和工程应用
- 易于实现
代码相对简单，是最常见的聚类入门算法之一
- 结果可解释性强
每个簇都有一个中心，比较容易理解每个类的特征
- 需要预设簇数 K
簇数需要预先指定，选择不当会严重影响结果（可用肘部法则、轮廓系数改善）
- 对初始值敏感
初始质心选择不当，容易陷入局部最优，甚至无法收敛（可用 K-Means++ 改善）
- 只能发现凸形/球形簇
基于欧氏距离的度量假设簇是球状分布，无法处理环形、月牙形等非凸形状
- 对噪声和离群点等异常值敏感
极端值会显著拉偏质心位置（可用 K-Medoids 替代）
- 特征尺度敏感
不同量纲的特征会导致距离计算被大量纲特征主导，使用前需要标准化

实用改进

1. 使用 K-Means++ 初始化
让初始质心尽量分散，显著降低算法陷入较差局部最优的概率

2. 多次运行并选择最优结果
让算法从不同初始质心出发，选择目标函数 (SSE) 最小的结果 (如 `n_init=10`)
3. 使用 Mini-Batch K-Means 提高效率
在大规模数据场景下，每轮迭代只使用一个小批量样本来更新质心，显著提高运行速度

其他常用聚类算法

1. 中心点聚类 K-Medoids
与 K-Means 取均值不同
K-Medoids 强制选取 **实际存在的数据点** 作为质心
如此增强对噪声和极端值的稳健性
2. 基于密度的聚类 DBSCAN
不需要预先指定簇数 K
通过样本的 **密度** 划分簇
能发现任意形状（环形、月牙形）的簇
并且自带异常点检测能力
3. 层次聚类 Hierarchical Clustering
不需要预先指定簇数 K
通过计算节点之间的距离，自底向上 (或自顶向下) 构建一棵 **聚类树**
适合需要多层次分类的场景

12. PCA 降维

本节内容

1. PCA 与 离散问题
2. 降维方法对比
3. PCA 直观理解与例子
4. PCA 数学原理
5. 代码演示
6. PCA 与随机森林对比
7. 总结

维度灾难

当特征维度不断增长时，会出现一系列问题，这些问题称为 **维度灾难**

1. **计算成本急剧增长**
特征越多，模型训练和预测时需要计算的参数量就越大，导致严重的算力和时间开销
2. **数据稀疏性与过拟合风险**
在高维空间中，数据点之间的距离分布变得极其稀疏。
为了让模型学到有效的模式，所需的数据量会呈指数级增加。
如果数据量不足，模型极易记住噪声，导致严重的过拟合。
3. **距离度量失效**
在高维空间中，数据点之间的距离趋于均匀化。
当维度足够高时，任意两个点之间的距离都趋于相等。
这使得基于距离的算法（如 KNN、K-Means）几乎失效。
4. **多重共线性问题**
许多特征之间往往存在强相关性（例如“房屋面积”与“房间数量”）
这不仅无法提供额外信息，还会干扰线性模型对特征权重的评估。

传统的降维方法

为了应对数据特征过多，需要对数据预处理，减少特征数量，称为**降维**

1. **特征选择 Feature Selection**
直接从原始特征库中挑选出最有价值的特征子集，剔除无用或冗余的特征。
 - 常见的方法
 1. 过滤法（如方差阈值、相关系数）
 2. 包装法
 3. 嵌入法（如 L1 正则化）

- 缺点

非此即彼的粗暴操作，直接丢失某些特征可能会导致部分有用信息的永久丢失

2. 特征提取 Feature Extraction

将高维的原始特征映射（转换）到一个低维的新特征空间中，新特征通常是原始特征的组合

- 优点

在降低维度的同时最大程度保留原始数据的信息

- 代表算法

PCA 主成分分析

主成分分析 PCA

主成分分析 (Principal Component Analysis, PCA)

是一种无监督的线性降维算法

将高维数据投影到低维空间，同时尽量保留数据中存在的**最大方差 (即信息量)**

PCA 就是在数学空间寻找最佳角度的算法，最大程度展示全貌

这些角度即所谓的主成分

✗ 不好的角度：正上方拍摄



从正上方看，只看到一个圆，
丢失了壶嘴和把手的信息。



✓ 绝佳的侧面角度

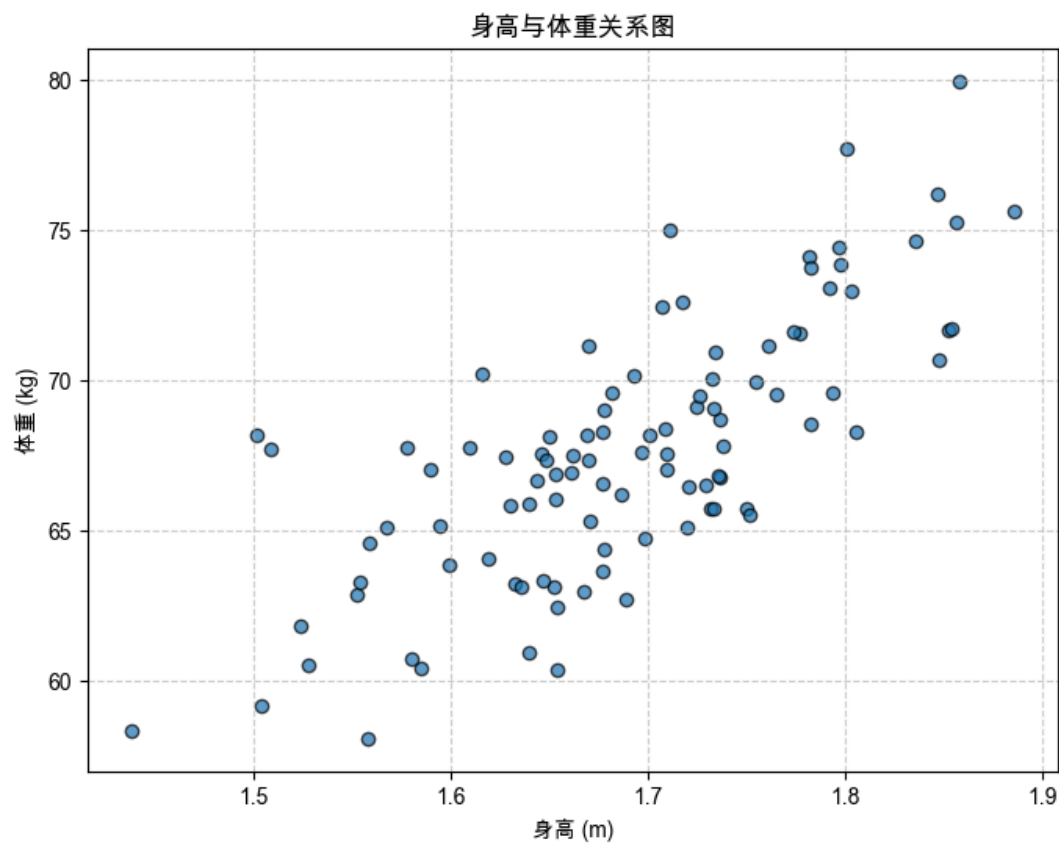


选择一个绝佳的侧面角度，
可以清晰地展示茶壶的全貌。

身高体重示例

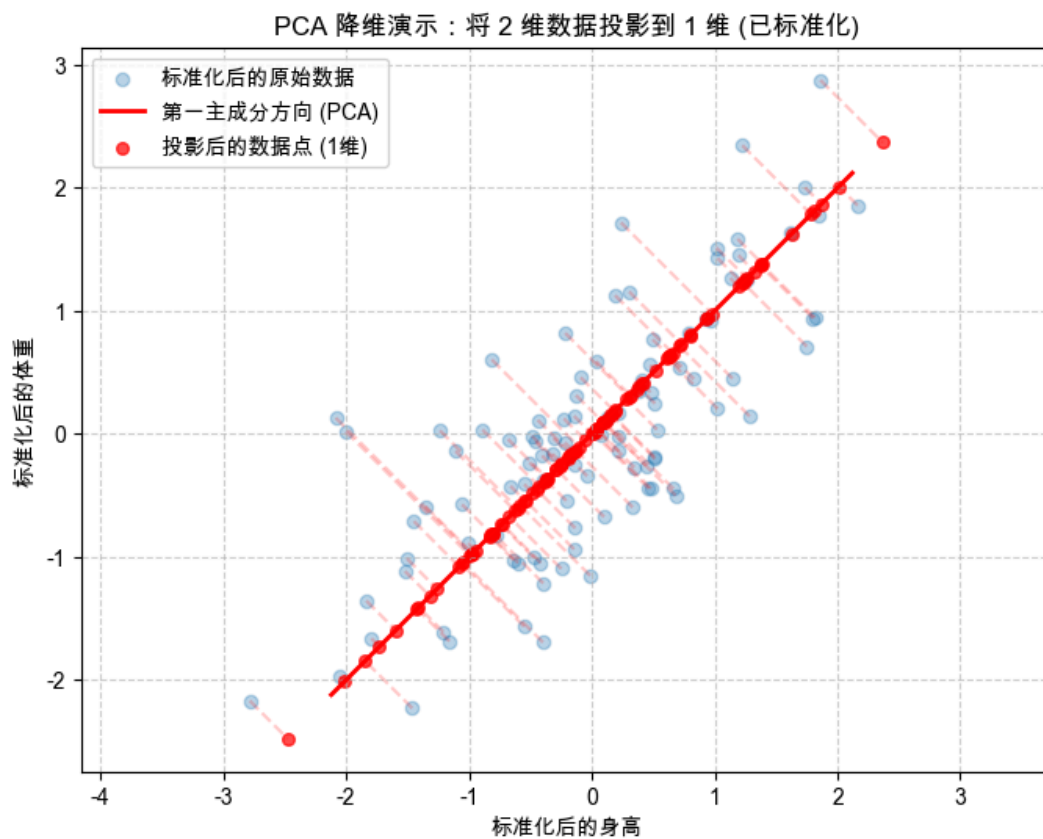
假设构建一个健康风险预测模型，数据集中有身高和体重两个特征

把这批数据画到二维坐标系上，横轴是身高，纵轴是体重



- 身高越高，体重往往越重
- 数据点大致自左下至右上的对角线排布

身高与体重这两个特征存在强烈的 **正相关关系**，携带重叠的信息



PCA 会找到最具代表性的对角线反映数据走势，然后将所有数据点投影到该线上

- 投影之后，每人只需一个特征数值就能表示，即从二维降至一维
- 这个新特征近似于 **体重指数 BMI**

PCA 能在缺乏医学知识的情况下，纯粹通过数据的数学结构自动发现该组合特征

[!TIP]
PCA 不是简单地扔掉某个特征，而是将多个相关特征融合成一个新的综合特征。
这个新特征保留了原来两个特征的大部分信息，同时消除了它们之间的冗余。

房屋地段示例

假设构建一个二手房价格预测模型，数据集中有以下 6 个特征

特征	说明
地铁距离	距离最近地铁站的距离
公交线路数	周边 1km 内的公交线路数量
商场数量	周边 2km 内的商场数量
学校评分	周边学区学校的综合评分
医院等级	周边最近医院的等级（1-3级）
公园面积	周边绿地公园的总面积

问题梳理

直接使用 6 维特征的问题

- 多重共线性
地铁距离和公交线路数高度相关，模型很难判断到底是哪个特征在起作用
- 特征冗余
6 个特征中大量信息重复，徒增模型复杂度
- 难以解释
模型给出地铁距离的权重是 -0.23，但这个数字对业务人员来说很难理解

背后的隐藏概念：地段
这 6 个特征表面各不相同，但背后都与“地段好坏”密切相关。
内部存在大量的正相关关系，携带着高度重叠的信息。

压缩结果

PCA 自动分析相关性，将 6 个特征压缩成少量综合成分

1. 第一主成分 PC1 ≈ 地段综合分
 $PC1 = 0.45 \times (-\text{地铁距离}) + 0.42 \times \text{公交数量} + 0.40 \times \text{商场数量} + 0.38 \times \text{学校} + 0.36 \times \text{医院等级} + 0.35 \times \text{公园面积}$
 - 赋予每个特征权重，可能解释原始数据约 70% 的方差信息
2. 第二主成分 PC2 ≈ 生活便利 vs 自然环境
 $PC2 = 0.52 \times \text{公园面积} + 0.48 \times \text{公交数量} - 0.43 \times \text{商场} - 0.40 \times \text{地铁距离}$
 - 捕捉另一维度差异，可能解释额外约 15% 的方差信息
郊区宜居型 vs 市区商业型

这两项主成分合理解释大约 85% 的信息
把 6 维特征压缩成 2 维，大幅降低了复杂度
同时业务人员也能更直观地理解这两个综合指标的含义。

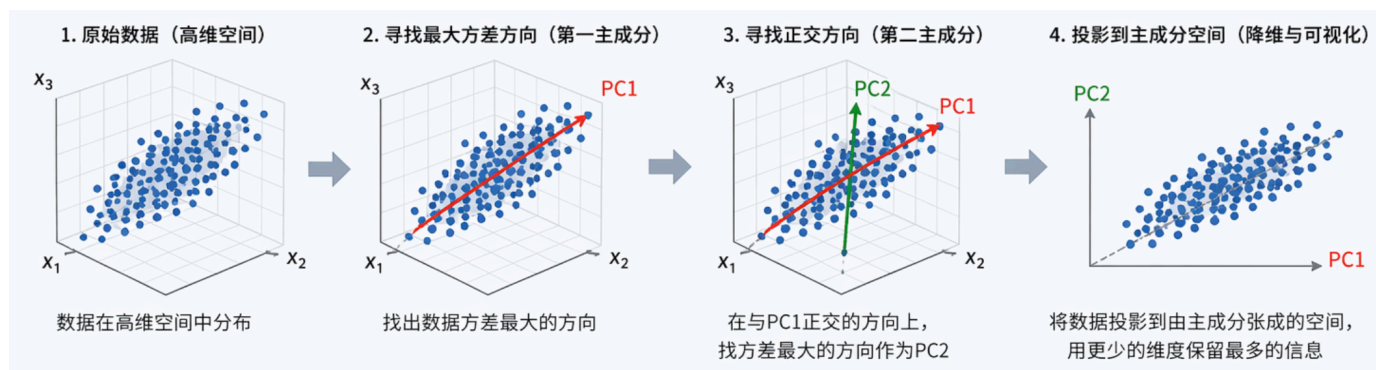
[!TIP]
PCA 通过数学方法把隐藏因子挖掘出来，用少数维度替代大量冗余特征

PCA 的数学原理

- 方差 (Variance)
每个样本减平均值，平方后再平均
数据越分散，方差越大
- 协方差 (Covariance) 两个变量各自减去均值后相乘再平均
正负反映两个变量是否同步变化

PCA 的底层逻辑建立在协方差矩阵和特征值分解的基础上。

假设有数据集矩阵 X ，包含样本 n 个，特征 d 个，即矩阵大小为 $n \times d$ ，将其降维至 k 维



数据去中心化

将每个特征的数据进行去中心化处理，即减去该特征均值，处理后新的均值为零

这能消除不同特征量纲和量级带来的影响

$$x_i^{(j)} = x_i^{(j)} - \mu_j$$

- μ_j : 是第 j 个特征的均值

假设测量5个人的身高、坐高、体重三项指标

人	身高(cm)	坐高(cm)	体重(kg)
A	160	90	55
B	170	95	65
C	180	100	80
D	150	85	45
E	175	98	72

去中心化后的数据矩阵 X 为

$$X = \begin{bmatrix} -7 & -3.6 & -8.4 \\ 3 & 1.4 & 1.6 \\ 13 & 6.4 & 16.6 \\ -17 & -8.6 & -18.4 \\ 8 & 4.4 & 8.6 \end{bmatrix}$$

计算协方差矩阵

协方差用来衡量两个变量之间的相关性

计算去中心化后数据矩阵 X 的协方差矩阵 C

$$C = \frac{1}{n-1} X^T X$$

- $\frac{1}{n-1}$: 贝塞尔修正 Bessel's correction

[!TIP]

通过样本数据推断总体特性时，协方差矩阵的无偏估计需要贝塞尔修正

简单的 $\frac{1}{n}$ 会低估方差

- $\frac{1}{n}$: 有偏估计
- $\frac{1}{n-1}$: 无偏估计

得到协方差矩阵 C 是 $d \times d$ 的对称矩阵

$$C = \begin{bmatrix} 145 & 73.5 & 165.25 \\ 73.5 & 37.3 & 83.7 \\ 165.25 & 83.7 & 190.3 \end{bmatrix}$$

- 对角线元素
是各特征的方差
 1. 身高 145
 2. 坐高 37.3
 3. 体重 190.3
- 非对角线元素
是特征之间的协方差
 1. 身高体重 165.25
 2. 身高坐高 73.5
 3. 坐高体重 83.7

计算特征值与特征向量

对协方差矩阵 C 进行特征分解，求解所有特征值 λ 和对应的特征向量 v

$$Cv = \lambda v$$

- v : 特征向量，是新的坐标轴方向，即**主成分**
- λ : 特征值，代表数据投影到对应特征向量上的方差大小，即**主成分的重要性**

解特征方程得到 3 项特征值

$$\lambda_1 \approx 334.93, \quad \lambda_2 \approx 37.67, \quad \lambda_3 \approx 0$$

和对应的单位特征向量

$$v_1 \approx \begin{bmatrix} 0.592 \\ 0.315 \\ 0.741 \end{bmatrix}, \quad v_2 \approx \begin{bmatrix} -0.793 \\ 0.522 \\ 0.314 \end{bmatrix}, \quad v_3 \approx \begin{bmatrix} 0.147 \\ 0.793 \\ -0.592 \end{bmatrix}$$

- v_1 分量均为正值，且身高和体重系数较大。综合反映身高、坐高、体重共同增减的“体型大小”维度
- v_2 身高系数为负，坐高体重系数为正，捕捉“胖瘦比例”（身高相同时偏胖或偏瘦）的维度
- $\lambda_3 \approx 0$ 说明第三数据有强冗余，数据实际几乎完全落在二位平面上

选取主成分并构建投影矩阵

将算得的特征值从小到大排序: $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$

选取前 k 个最大的特征值对应的特征向量，将它们按列拼接得到 $d \times k$ 的投影矩阵 W ， k 为降维后的目标维度

对于 k 的选定，通常根据累计解释方差率决定。选出前 k 个最大的特征值相加，再除以特征值总和，找出能超过阈值(如95%)的最小 k

$$\text{解释方差率} = \frac{\lambda_i}{\sum_{j=1}^n \lambda_j}$$

$$\arg \min_k \frac{\sum_{i=1}^k \lambda_i}{\sum_{j=1}^n \lambda_j} \geq \text{threshold (like 95\%)}$$

上述示例中

- 总方差和 $\sum_{j=1}^n \lambda_j = 334.93 + 37.67 + 0 = 372.6$
- PC1 解释率 $\frac{\lambda_1}{\sum_{j=1}^n \lambda_j} = \frac{334.93}{372.6} \approx 89.9\%$
- PC2 解释率 $\frac{\lambda_2}{\sum_{j=1}^n \lambda_j} = \frac{37.67}{372.6} \approx 10.1\%$
- PC3 解释率 $\frac{\lambda_3}{\sum_{j=1}^n \lambda_j} = \frac{0}{372.6} = 0$ 由于 PC1 与 PC2 累计解释率到 100%，因此选择 $k = 2$ ，将数据降至二维形成投影矩阵 W

$$W = [v_1 \quad v_2] = \begin{bmatrix} 0.592 & -0.793 \\ 0.315 & 0.522 \\ 0.741 & 0.314 \end{bmatrix}$$

数据投影

将原始中心化数据矩阵 X 乘以投影矩阵 W ，得到降维后的新数据矩阵 $Y(n \times k)$

$$Y = XW$$

$$\begin{bmatrix} -7 & -3.6 & -8.4 \\ 3 & 1.4 & 1.6 \\ 13 & 6.4 & 16.6 \\ -17 & -8.6 & -18.4 \\ 8 & 4.4 & 8.6 \end{bmatrix} \begin{bmatrix} 0.592 & -0.793 \\ 0.315 & 0.522 \\ 0.741 & 0.314 \end{bmatrix} = \begin{bmatrix} -11.50 & 1.03 \\ 3.83 & -1.62 \\ 17.6 & -2.77 \\ -22.61 & 2.69 \\ 12.68 & 0.68 \end{bmatrix}$$

- 原始 5×3 矩阵经过 PCA 降维后用 2 个新特征即可完整表达原始信息
- PC1 体型大小：从左到右大致是从瘦小到魁梧 ($D \rightarrow A \rightarrow B \rightarrow E \rightarrow C$)
- PC2 胖瘦比例：在同一体型水平上区分相对胖瘦
- 3 个高度相关的物理量被压缩为 2 个独立主成分，消除冗余便于二维可视化

鸢尾花 PCA 应用

鸢尾花数据集包含 150 个样本，4 个特征

1. 花萼长度 sepal length
2. 花萼宽度 sepal width
3. 花瓣长度 petal length
4. 花瓣宽度 petal width

加载数据

```
from sklearn.datasets import load_iris
# ===== 加载数据 =====
iris = load_iris()
X, y = iris.data, iris.target
target_names = iris.target_names
feature_names = iris.feature_names

print(f"原始数据维度: {X.shape}") # (150,4)
```

- 原始数据维度: (150, 4)

标准化

```
from sklearn.preprocessing import StandardScaler
# ===== 标准化 =====
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

分析特征相关性

```
import numpy as np
# ===== 分析特征相关性 =====
corr_matrix = np.corrcoef(X_scaled, rowvar=False)
print("\n特征相关系数矩阵:")
print(np.round(corr_matrix, 3))
```

- 特征相关系数矩阵:

```
[[ 1.    -0.118  0.872  0.818]
 [-0.118  1.    -0.428 -0.366]
```

```
[ 0.872 -0.428  1.      0.963]
[ 0.818 -0.366  0.963  1.    ]]
```

PCA 降维

```
from sklearn.decomposition import PCA
# ===== PCA 降维 =====
pca = PCA()
pca.fit(X_scaled)

print("\n各主成分特征值（方差）:", np.round(pca.explained_variance_, 4))
print("各主成分方差解释比:      ", np.round(pca.explained_variance_ratio_, 4))
print("累积方差解释比:          ", np.round(np.cumsum(pca.explained_variance_ratio_), 4))

# 输出主成分载荷（特征向量），理解每个主成分的含义
print("\n主成分载荷矩阵（每列是一个主成分方向）:")
loadings = pca.components_.T
for i, fname in enumerate(feature_names):
    print(f"    {fname:30s}: PC1={loadings[i,0]:+.3f} PC2={loadings[i,1]:+.3f}")

# 降到 2 维
pca_2d = PCA(n_components=2)
X_2d = pca_2d.fit_transform(X_scaled)
```

- 各主成分特征值（方差）
[2.9381 0.9202 0.1477 0.0209]
- 各主成分方差解释比
[0.7296 0.2285 0.0367 0.0052]
- 累积方差解释比
[0.7296 0.9581 0.9948 1.]
- 主成分载荷矩阵（每列是一个主成分方向）
sepal length (cm) : PC1=+0.521 PC2=+0.377
sepal width (cm) : PC1=-0.269 PC2=+0.923
petal length (cm) : PC1=+0.580 PC2=+0.024
petal width (cm) : PC1=+0.565 PC2=+0.067
- **PC1 解释 72.96% 方差**
petal length 花瓣长度(+0.58)
petal width 花瓣宽度(+0.57)
sepal length 花萼长度(+0.52)
三者都有较大正载荷，PC1 代表花的整体大小，数值越大，花越大
- **PC2 解释 22.85% 方差**
几乎只由 sepal width 花萼宽主导 (+0.923)
其他特征贡献很小，可以理解为 PC2 专门代表 花萼宽度
- 仅用 2 个主成分就捕获了 95.8% 的信息，同时从可视化图中能清晰看到三个类别的分布

可视化

```
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# ===== 可视化 =====
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# --- 左图：碎石图 ---
components = range(1, 5)
```

```

axes[0].bar(components, pca.explained_variance_ratio_,
            color='#3498db', alpha=0.8, label='Individual')
axes[0].plot(components, np.cumsum(pca.explained_variance_ratio_),
            'o-', color='#e74c3c', linewidth=2, markersize=8, label='Cumulative')
axes[0].axhline(y=0.95, color='gray', linestyle='--', alpha=0.7)
axes[0].set_xlabel('Principal Component', fontsize=12)
axes[0].set_ylabel('Explained Variance Ratio', fontsize=12)
axes[0].set_title('Scree Plot - Iris Dataset', fontsize=13)
axes[0].set_xticks(components)
axes[0].legend()
axes[0].grid(True, alpha=0.3)

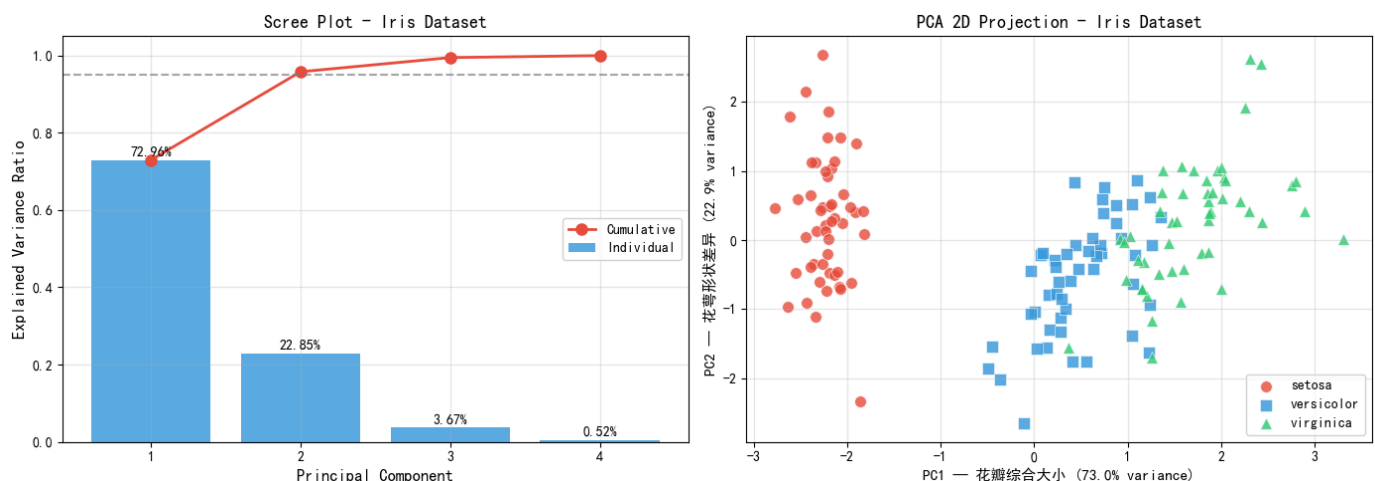
# 标注每个柱子的值
for i, v in enumerate(pca.explained_variance_ratio_):
    axes[0].text(i + 1, v + 0.01, f'{v:.2%}', ha='center', fontsize=10)

# --- 右图: 2D 投影散点图 ---
colors = ['#e74c3c', '#3498db', '#2ecc71']
markers = ['o', 's', '^']
for i, (color, marker, name) in enumerate(zip(colors, markers, target_names)):
    mask = y == i
    axes[1].scatter(X_2d[mask, 0], X_2d[mask, 1],
                   c=color, marker=marker, label=name,
                   alpha=0.8, edgecolors='w', s=70, linewidths=0.5)

axes[1].set_xlabel(
    f'PC1 — 花瓣综合大小 ({pca_2d.explained_variance_ratio_[0]:.1%} variance)',
    fontsize=11)
axes[1].set_ylabel(
    f'PC2 — 花萼形状差异 ({pca_2d.explained_variance_ratio_[1]:.1%} variance)',
    fontsize=11)
axes[1].set_title('PCA 2D Projection - Iris Dataset', fontsize=13)
axes[1].legend(fontsize=10)
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
# plt.savefig('iris_pca.png', dpi=150, bbox_inches='tight')
plt.show()

```



随机森林对比PCA

维度	随机森林特征选择	PCA
是否监督	需要标签	无监督
是否线性	可捕捉非线性	纯线性
解释性	高（原始特征）	低（新主成分是线性组合）
适用场景	业务诉求强、可解释需求	纯性能、特征高度相关场景

维度	随机森林特征选择	PCA
阿里典型案例	用户画像、风控规则	图像 embedding、CTR 预估

[!TIP]
先用随机森林筛掉明显无关特征，再用 PCA 做二次降维，效果往往 1+1>2

PCA 优劣

优点

- 1. 无需人工干预
- 2. 有效消除特征之间的共线性
- 3. 有效降低数据噪音，缓解过拟合
- 4. 常用于高维数据探索性的可视化工具

缺点

- 1. 线性假设局限
PCA 假设数据的主成分是线性的。
对于非线性结构数据，传统 PCA 效果不佳，需要 Kernel PCA 或 t-SNE、UMAP 等非线性降维算法。
- 2. 丧失可解释性
降维后的新特征是原始特征的线性组合，失去了原本的业务含义。
例如无法再解释鸢尾花数据集中 PC1 代表的是花萼还是花瓣。
- 3. 对异常值敏感
异常值会显著影响方差的计算

结语 机器学习总结

本节内容

- 1. 机器学习全貌
- 2. 核心概念辨析
- 3. 监督学习详解
- 4. 模型训练与评估
- 5. 集成学习
- 6. 无监督学习
- 7. 工业实践工具
- 8. 案例演示
- 9. 深度学习前瞻
- 10. 结尾

机器学习全景

三类任务

- 1. 监督学习 有标签：从输入学习到答案的映射
 - 回归：房价、销量、温度
 - 分类：垃圾邮件、流失预测
 - 线性/逻辑回归、树模型、KNN、SVM
- 2. 无监督学习 无标签：发现数据自己的结构
 - 聚类：K-Means、DBSCAN
 - 降维：PCA、特征提取
 - 异常检测、生成建模
- 3. 强化学习 在动态环境中试错，用奖惩塑造策略
 - Agent / Environment
 - Action / Reward / Policy
 - 游戏、机器人、推荐策略

上述任务通用如下流程，进行模型训练及问题解决

- 1. 定义目标与指标

2. 清洗数据与特征
3. 训练、验证、测试
4. 处理拟合问题
5. 上线监控与重训

[!TIP]

机器学习不是算法清单，是“用数据解决问题”的完成工程方法

核心概念

机器学习在学什么？

- 传统编程
人工阐明规则，机器按规则执行
规则 + 数据 → 结果
- 机器学习
给机器历史样本，让其自动提取规律
数据 + 答案 → 模型

完成模型训练后，机器将习得的规律用于新样本预测：房价、销量、点击、欺诈、流失、图像类别等。

数据、特征、标签、模型

1. 数据进入模型
 - 样本
一条记录，一个用户，一套房子，一封邮件
 - 特征
描述样本的输入变量
 - 标签
模型理想的输出预测
2. 算法与模型的区别
 - 算法
学习方法，如线性回归、决策树
 - 模型
学习结果，如一组权重、一棵树
 - 同一种算法，在不同数据上训练得到不同模型

[!TIP]

机器学习的第一步，是将问题翻译成“特征 → 标签”的学习任务，而非挑选算法

监督学习：有答案的学习

训练数据既有输入，也有正确答案

模型需要学习从输入到标签的映射

回归 Regression

预测连续数值

- 适用对象
 1. 房价
 2. 销量
 3. 收入
 4. 温度
- 常用模型
 1. 线性回归
 2. 决策树回归
 3. 随机森林回归
 4. 梯度提升决策树 GBDT

分类 Classification

预测离散类别

- 适用对象
 1. 垃圾邮件
 2. 流失用户
 3. 猫狗
- 常用模型
 1. 逻辑回归
 2. KNN
 3. 朴素贝叶斯
 4. SVM
 5. 树模型

典型算法

1. 线性/逻辑回归
 - 简单易解释
 - 训练速度快
 - 线性回归输出连续数值
 - 逻辑回归输出离散类别
 - 是深度学习的基础
2. 决策树
 - 通过系列判断规则进行预测
 - 直观易解释
 - 容易过拟合
 - 是随机森林和 GBDT 的基础
3. KNN
 - 根据最近的 K 个邻居投票
 - 思想简单
 - 预测速度慢
 - 对尺度敏感
4. 朴素贝叶斯
 - 基于概率和独立性假设
 - 常用与邮件分类、新闻分类、情感分析
5. SVM
 - 寻找最大间隔分类边界
 - 核函数可以处理非线性
 - 适合处理中小规模数据
6. 树模型集成
 - 随机森林、XGBoost、LightGBM、CatBoost 是结构化数据强基线

模型训练：从业务问题到可用模型

1. 定义问题
 - 明确业务目标、预测对象、成功指标
2. 准备数据
 - 清洗缺失值、异常值
 - 编码类别特征，构造业务特征
3. 划分数据
 - 划分训练集、验证集、测试集
 - 按需使用交叉验证
4. 训练优化
 - 定义损失函数
 - 使用梯度下降或树模型分裂规则优化
5. 评估调参
 - 关注泛化能力
 - 处理欠拟合、过拟合与类别失衡
6. 上线迭代
 - 部署
 - 监控数据漂移
 - 回收反馈
 - 周期性重训

损失函数与梯度下降

损失函数

衡量预测值和真实值的差距

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

梯度下降

沿着损失下降最快的方向更新参数

$$\theta := \theta - \eta \nabla J(\theta)$$

- $:=$ 表示计算机编程中的赋值
- $\nabla J(\theta)$: 损失函数 $J(\theta)$ 对 θ 的偏导数向量

[!TIP]

训练模型 = 定义目标 + 衡量错误 + 不断调参

模型评估

数据集

1. 训练集
用于学习参数
2. 验证集
用于选择模型与调参
3. 测试集
用于最终评估泛化能力

回归指标

1. MAE
平均绝对误差
2. MSE / RMSE
缩放极端误差
3. R^2 解释方差比例

分类指标

1. Accuracy
整体准确率
2. Precision / Recall / F1
3. ROC / AUC / 混淆矩阵

拟合问题

1. 欠拟合
学的太少
训练误差高，测试误差也高
 - 增加特征或特征组合
 - 使用复杂模型
 - 减少正则化，增加训练次数
2. 过拟合
记住噪声
训练误差低，测试误差高
 - 增加数据或数据增强，稀释噪声影响
 - 降低模型复杂度、剪枝
 - 正则化、早停、交叉验证

[!TIP]

模型选择不是追求训练集上取得高分，而是**平衡拟合能力和泛化能力**

集成学习：让模型组成团队

Bagging: 并行投票

- 有放回采样，训练多个模型
- 分类投票，回归平均
- 代表算法：**随机森林**
- 核心价值：降低方差，缓解过拟合

Boosting: 串行纠错

- 多棵树相继串行训练
- 后面的模型专注于纠正前面的错误
- 代表算法：**GBDT**
- 核心价值：降低偏差，提升精度

GBDT 家族：结构化数据强基线

GBDT 通过多棵树串行累加，每轮拟合当前模型最需要修正的方向

1. XGBoost
 - 正则化控制复杂度
 - 一阶 + 二阶梯度
 - 工程优化成熟
2. LightGBM
 - 直方图算法提速
 - Leaf-wise 生长
 - 适合大规模数据
3. CatBoost
 - 原生类别特征
 - 排序提升减少偏移
 - 对称树推理快

[!TIP]

使用表格数据先建立树模型基线，通常是稳妥的起点

无监督学习：从无标签数据中发现结构

1. K-Means 聚类
 - 预先指定 K 个簇
 - 样本分配到最近的质心
 - 更新质心，直到收敛
 - 应用：用户分群、图像压缩、异常检测
2. 其他聚类思路
 - K-Medoids
 - 处理异常值更稳健
 - DBSCAN
 - 基于密度，不用预设K
 - 层次聚类
 - 形成多层次聚类树

PCA 降维：保留信息压缩维度

高维数据会有维度爆炸问题

1. 计算成本高
2. 数据稀疏性
3. 距离失效
4. 过拟合

PCA 用少数主成分代表主要信息

- 找到数据变化最大的方向
- 把原始特征组合成新坐标轴
- 按解释方差选择前 K 个主成分

适用场景

- 高维数据可视化
- 去除冗余和共线性
- 降低后续模型训练成本

[!TIP]
PCA 的代价是降低可解释性
新特征更像数学坐标，而非直观的业务字段

工业实践工具

AutoML 不只是“一键训练模型”，更是一组覆盖数据探索、预处理、调参、实验管理和质量检查的工具链

需求	首选工具	替代方案	替代场景
快速了解数据全貌	fg-data-profiling	sweetviz	对比两个数据集
快速得到强健基线	AutoGluon	PyCaret	在Jupyter逐步交互
规范预处理流程	Sklearn Pipeline	-	必用核心，没有平替
智能搜索超参数	Optuna	Ray Tune	多机多卡分布式调参
实验记录与管理	MLflow	W&B	团队协作及深度学习场景
检测标签错误	Cleanlab	人工抽检	数据质量遇到瓶颈
处理类别失衡	imbalanced-learn	class_weight	数据量小、任务简单

[!TIP]
先用 AutoGluon / PyCaret 建立强健基线，再用 Pipeline / Optuna / MLflow 把流程工程化

从机器学习到深度学习

传统机器学习提供“从数据到模型”的完整框架，但是依赖人工设计特征

1. 传统机器学习
- 适合结构化表格数据
 - 强依赖特征工程
 - 可解释性强、训练成本低
 - 树模型在业界表现突出
2. 深度学习
- 使用多层神经网络
 - 自动学习多层抽象特征
 - 擅长图像、语音、文本、生成任务
 - 需要更多数据、算力和工程体系